

VECTOR

A Language for the Elite!

• APL94 Conference Abstracts	7
• Ziemann reviews J Release 2	65
• Pearson on the PLUS III GUI	75
• Smith on the Control Structures	84
• Piper on Objects and APL	96
• Lescasse on PLUS III Programming	109

... plus

• 16-page Educational Supplement	29
----------------------------------	----



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

Vol.11 No.1 July 1994

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, in workspaces from I-APL, APL*PLUS, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the Vector TrueType font, available free from Vector Production), and Winword-2.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 1994-95

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£14	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	10	5
Overseas Corporate	£135	10	
Sustaining	£430	50	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates are £250 per full page, £125 for half-page or less (there is a £75 surcharge per advertisement if spot colour is required).

Deadlines for bookings and copy are given under the Quick-reference Diary. Advertisements should be booked with, and sent to: Gill Smith, Brook House, Gilling East, YORK YO6 4JJ. Tel: 0439-788385. Cserve: 100331,644

Contents

		Page
Editorial: a Language for the Elite!	Anthony Camacho	3
APL NEWS		
Quick Reference Diary		5
APL94 Abstracts		7
British APL Association News: EGM Report, AGM Report, Annual Accounts, Regulations		13
News from Sustaining Members		21
The Education Vector	Ian Clark	29
REVIEWS SECTION		
APL Product Guide	Gill Smith	45
Jim Weigang's <i>APL Notes</i>	Walter Spunde	57
APL*PLUS/PC Version 11	Adam Weber	61
J Release 2 (Iverson Software Inc.)	Dave Ziemann	65
APL*PLUS III Gui Implementation	Duncan Pearson	75
... and Control Structures	Adrian Smith	84
RECENT MEETINGS		
James Wheeler demonstrates PLUS III	notes by Anthony Camacho	92
David Piper on Objects and APL		96
Namespaces and Dyalog APL Version 7	notes by Adrian Smith	101
GENERAL ARTICLES		
To Be, Or Not To Be —		
That is the Gazodenplatz	Peter Merritt	108
Windows Development with APL*PLUS III	Eric Lescasse	109
The BAA Membership Database	Nicholas Small	119
TECHNICAL SECTION		
Hackers' Corner: Native Files without Tears	Adrian Smith	126
Technical Correspondence		129
At Play with J - Control Structures	Gene McDonnell	136
A Windows-driven Menu Driver	Jan Karman	139
Index to Advertisers		143



SOLITON ASSOCIATES

SHARP APL: the high performance choice

SHARP APL is superior in the rapid development of mission-critical applications which meet the ever-changing demands of our customers.



***SOLITON provides SHARP APL
for MVS and UNIX with:***

- Superior productivity
- Ease in managing shared-file multi-user applications
- Powerful cooperative processing
- High performance DB2 interface
- Applications for end-users and programmers
- Responsive support services

Visit us at our APL94 booth, September 11 – 15 in Antwerp, Belgium

For more information, telephone or FAX:

SOLITON ASSOCIATES LIMITED

44 Victoria Street, Suite 2100
Toronto, Ontario, Canada M5C 1Y2

Tel: (416) 364-9355 FAX: (416) 364-6159

In Europe

Soliton Associates Limited of Canada
Groot Blankenberg 53, 1082 AC Amsterdam
The Netherlands
Tel: +31-20-646-4475 FAX: +31-20-644-1206

In the U.S.

Soliton Associates Incorporated
1100 University Avenue, Suite 111
Rochester, New York, USA 14607
Tel: (716) 256-6465 FAX: (716) 256-6469

or via Internet to: sales@soliton.com

Editorial: a Language for the Elite!

by Anthony Camacho

I have spent over ten years saying self-deprecatingly of APL "No, it's not really very difficult. Anyone can do it who tries. Chinese looks worse but five-year-olds in China can manage it without difficulty." and similar anodyne things.

My recent attempts to explain J have made me realise that "APL is easy" just isn't true. There is an industry-wide wish to sell all products on their ease of use. Most products are not easy to use — even the latest like Access. There is no easy way to specify algorithms. We have tried to sell APL on ease of use too. What we have been doing is to sell a micrometer as a ruler or a microscope as a magnifying glass. APL and J cannot be used properly by fools. They are not simple tools.

Why not try the other tack? We all know that the opposite is true. The APLers we admire are exceptionally intelligent people. The APL we abhor is written by plodders. Skill shows more in APL than in any other language. APL is used for the Board reports and so APLers have to be cleverer, more trustworthy and more tolerant of the Directors' whims. Only the best people use APL. Most programmers in other languages cannot adapt to use it properly.

A good APL program nowadays processes a file at a time and an array at a time; the other languages process a record at a time or a value at a time. APLers think in parallel; other languages require their programmers to think in loops. APL programmers think of functions such as average or remove duplicates as a phrase; to other languages these are subroutines. APLers routinely use six-dimensional arrays; other languages rarely manage three dimensions. APLers have component files and (in nesting or boxing) the equivalent for arrays; other languages don't have these constructs.

Since the quality of an APL system depends more on the design than on the coding, APLers have to have a wider range of skills than other DP professionals. There are no APL coders. All APLers are at least systems analysts as well as programmers. APL productivity can be ten or twenty times what is achievable in other languages but only one in ten programmers will ever be able to do APL.

Ability to program in APL at any level is a hint at excellence; being able to program well in APL is a sure sign of it.

APL is the language for the elite!

10 Years Ago Editorial from Vector 1.1

by Robert Bittlestone

Rerun Bittlestone Editorial from 1.1

Actual Size

EDITORIAL: Back to BASICS?

by Robert Bittlestone

This is the first issue of VECTOR: a new journal for a new generation of computer users. Although VECTOR is aimed at those who know and use APL, it's also very much intended for those who don't yet but might. So we make no apology for including articles which look at APL from the viewpoint of a non-APLer — including that of the general manager who'd like to know why APL makes sense without necessarily learning it personally.

APL is different from all other computer languages in many ways. These differences are part of what gives the language its strength, but also part of what causes it to be criticised. One of the most interesting points about its users is that there are virtually no cases of reverse conversion. Once you become committed to APL, it seems that you are rarely lured away from it. This kind of loyalty is praiseworthy indeed but also has a dangerous tendency to promote insularity. Events might just be passing APL by.

It's almost as if there are two kinds of computing. There is the world out there of COBOL and FORTRAN and BASIC and everything else. This world has its bad points — very slow project development and severe design regimentation being two of them. However, it also has its good points, which include a civilised approach to word processing, imaginative use of on-screen colour graphics, the advent of new user-friendly devices such as the "mouse", the use of a keyboard and character set that is well-nigh universal, and language availability on almost all computers.

Then there is the world of APL. Here we find some bad points too — extremists who confuse complexity with clarity, dreadful legends of undocumented and unsupportable systems, and obscure non-portable techniques required to cope with standard problems such as lower-case output. However, we also find a great many good points, such as elegant structures, powerful and appropriate array handling, flexible and sophisticated intrinsic features and a relational mentality that accelerates design as well as implementation.

I hope I have by now established the notion that VECTOR is not going to be a sycophantic journal. We aim to stimulate by criticism as well as by praise. If we sometimes infuriate, then be sure it is for a purpose. Write us a letter to object: but above all, keep the dialogue going. We have in our hands perhaps the most powerful tool yet devised for harnessing the power of the computer for human needs. So please let's agree to concentrate on honing its cutting edge, rather than laying out rococo designs for its handle.

Quick Reference Diary 1994

Date	Venue	Event
July 18-22	Swansea	APL in Business
Sept 11-15	Antwerp	APL94
October 28th	IEE	BAA Meeting: Vendor Forum

British APL Association meetings are normally held in the IEE, Savoy Place.
Nearest tube outlets: Temple or Embankment.

APL Training Courses for 1994

Level	Company	Days	Dates
Intro APL	MicroAPL	1	Jul 6, Sep 7, Nov 9
Intro APL2	MicroAPL	1	Aug 3, Oct 5, Nov 30
APL*PLUS	MicroAPL	1	Jul 20, Sept 21, Nov 23
Advanced APL	MicroAPL	1	Jul 13, Sept 14, Nov 16
APL GUI Design	MicroAPL	1	Aug 17, Oct 19, Dec 14
Programming Techniques	MicroAPL	1	Aug 24, Oct 26

If you would like to have your courses or seminars listed in Vector, please contact Gill Smith with the details.

Dates for Future Issues of VECTOR

	Vol.11 No.2	Vol.11 No.3	Vol.11 No.4
Copy date	2nd Sept 94	2nd Dec 94	3rd March 95
Ad booking	9th Sept 94	9th Dec 94	10th March 95
Ad Copy	16th Sept 94	16th Dec 94	17th March 95
Distribution	October 94	January 95	April 95

APL Programmer

For the finest minds in APL development

**Adaytum Systems is one of the UK's leading software houses
specialising in financial management software.**

The Product

We are pioneers in multi-dimensional application development. Our flagship product KPS is a multi-dimensional business planning system written in Dyalog APL under DOS/Windows.

The Position

A programmer is required to join our development team to take the current highly successful product from DOS/Windows into Unix/Motif, OLE2/ODBC and client server over the next 1½ years. Initially, the position will be based in either the South West or the South East.

The Requirement

The position demands an analytical and logical mind capable of thinking conceptually in multiple dimensions. You will have demonstrable skills for dealing with complex requirements and coding them into working systems swiftly and accurately.

Experience

Two years experience in APL/DOS/Windows is essential. UNIX/Motif/C++/SQL and networking is highly desirable, as is some familiarity with Object Oriented Programming. Ideally you are a Mathematics Graduate with a 1st class or 2.1 degree.

This is an outstanding opportunity to make a significant contribution to the development of a unique, state of the art Financial Management Software System. Salary £25k - £35k commensurate with experience.

Please write with CV to
Susan Strother, Adaytum Systems
13 Great George Street, Bristol BS1 5RR.

ADAYTUM
software

APL94 ABSTRACTS

APL94 will be held in Antwerp,
BELGIUM, 11-15 September 1994

APL – the Language and its Applications

Opening session: How symbols and notations may affect the mind and brain Dr. Paul J. Lewi, Janssen Research Foundation, Beerse, Belgium. Computer science produces powerful symbols and notations. Psychoanalysts proposed a model for the mind. Neurophysiology begins to reveal the molecular structure of the brain. The brain appears as a gigantic parallel computer that continuously rewires itself: plastic hardware. Patterns, when reinforced, grow into synaptic connections between neurons: sticky software. Symbols are the first links in the chain of encoding that maps the world into our brains. They mediate between the observer and the observed. Primitive people and alchemists understood the charm of symbols. Modernist artists have rediscovered their magic. Present-day scientists are regaining their alliance, most recently in fractal geometry and chaos theory. An understanding of symbols and notations is essential for our intellectual, if not mental, health. Because they embody our hopes, beliefs and fears, they record the progression and fate of mankind.

Plenary Presentations

APL in Finance Arthur Whitney, USA. A fast integrated APL has a lot to offer Finance. Currently, most decision support is done with a combination of spreadsheets, relational databases and C routines to glue them together and write analytics. The spreadsheets and RDBMSs are limited in speed and representation — and C is too low-level for high productivity in writing analytics. This paper is about a modern APL, Analytic Platform, which extends the spreadsheet and relational data models and uses C for things that C is good for — namely, intricate system or time-critical functions. Thus Analytic Platform uses C the way C programs used to use machine code. The list/vector processing language in Analytic Platform is called K. An earlier version of the Analytic Platform called A+ has been in use at Morgan Stanley for 6 years.

APL in Japan Kyozeke Saigusa, Japan. Dr. Saigusa presents a personal survey of the history of APL usage in Japan from 1971 to the present day. His survey, focusing mainly on IBM APLs, reflects the general acceptance of APL in Japan. Dr. Saigusa describes many significant applications, along with some technical problems unique to Japan. After showing the beginning, the growth, the blooming and the decline of APL use the discussion ends with an optimistic vision for the future and a request to better understand the specific requirements of APL for the Japanese market.

The History of APL2 Jim Brown, USA. Dr. Brown wants to remind us of the evolution of APL2 and the many contributions which served to develop the language: "Someone said that creativity is the art of remembering what someone has said but forgetting that someone else said it. In this paper I examine the origins of the major ideas that formed APL2 and try to remember who said them. Hundreds of people contributed their thoughts and criticisms. By mentioning some key contributors I don't mean to slight the others. APL2 was the work of many people and I was the organizer and integrator."

Closing session: Acquisition and Processing Data from Normal and Cancer Cells Dr. Ivar Giaever, Nobel Prize, Rensselaer Polytechnic Institute, USA. It is common today to control an experimental setup using a dedicated personal computer which is used to both acquire and process the data. Many computer programs exist for this purpose, but APL can easily be adapted to do this task. In this talk, a novel system will be described that can obtain morphological data from cells in tissue culture. When mammalian cells are cultured on small gold electrodes, they will alter the impedance of the electrodes. By measuring the impedance, it is possible to get quantitative data with regards to both the dynamic and static behaviour of the cells. During the experiments, the impedance is plotted on the computer screen, and afterwards the data is analyzed using Fast Fourier Transform, Hurst Rescaled Range Analysis or other mathematical techniques.

Accepted papers

A Risk Management Prototype using Object-Oriented APL M. Gfeller. After a quick introduction to Financial Risk Management, I show that Object-Oriented Technology (OOT) fits well with the demands of Risk Management. I then outline the analysis, design, and implementation of a prototype system for Risk Management using both OOT and APL. I describe a Dyalog APL implementation of OOT, and conclude by wondering whether APL is really suitable for OOT.

A Rating System for Qualification Analysis with Application to Personnel Evaluation A. Kononov. An APL-based expert opinion processing system provides a statistical evaluation of information and prepares both an assessment of attributes sufficiency and a textual description or testimonial.

Geometric Data of Cross-sections of Beams H. Besier. This paper describes a program that calculates the geometrical characteristic values of cross-sections of beams with complex shapes (bars or thin-walled profiles with any number of holes in the profiles). After a short introduction describing the calculations involved for moments of area, we will introduce the capabilities of every component of this program and we will make a survey of the whole system via a flow chart. At the end of the paper we will summarize the main features of the program and mention some possible extensions.

Simulation of a Mail Warehouse: An APL2 Solution for a Large Company A. Skomorokhov. The paper describes an APL2 solution to the problem of mail and document distribution in a large (33,000 employees) company. A simulation of a fully-automated warehouse for delivery of business papers is presented and the difficulties of simulation software design are considered.

Time Series Forecasting Using Neural Networks T. Kolarik and G. Rudorfer. Artificial neural networks are suitable for many tasks in pattern recognition and machine learning. In this paper we present an APL system for forecasting univariate time series with artificial neural networks. Unlike conventional techniques for time series analysis, an artificial neural network needs little information about the time series data and can be applied to a broad range of problems. However, the problem of network "tuning" remains: parameters of the back-propagation algorithm as well as the network topology need to be adjusted for optimal performance. For our application, we conducted experiments to find the right parameters for a forecasting network. The artificial neural networks that were found delivered a better forecasting performance than results obtained by the well-known ARIMA technique.

A Parallel Correlation-based Algorithm in J learns Neural Network Connections A. Skurikhin and A. Surkan. This paper describes and demonstrates a J language implementation of a correlation-based algorithm as it is applied to learning feed-forward and recurrent neural networks. Alopex, the learning algorithm, can be used as a tool for solving a wide

range of problems for complex systems including: modelling, prediction, diagnosis and pattern recognition. Instead of relying on error gradients, the Alopex algorithm learns on the basis of local correlations between the changes in individual weights and the changes of a global error measure. The algorithm is useful for optimising networks having arbitrary transfer functions distributed over its individual neurones and for minimising a large class of error measures. Alopex has the advantage that it updates the weights in a network simultaneously (i.e. in parallel) requiring only local computations. This permits a complete parallelisation of the algorithm represented in the J language implementation of this demonstration. This J program for the algorithm is applied to the real-world econometric problem of predicting the gas sendout or consumption value one or more days after the end of the time series record.

Knowledge Representation in Expert Systems in a Linguistic Form J. De Meyer. A good expert system needs to be as close as possible to the knowledge and data manipulation of the expert himself. Often experts use linguistic instead of numerical values. When input data are mostly qualitative and are based on subjective knowledge of experts, the Fuzzy Set Theory is a solid mathematical model to represent and manipulate these data. APL arrays, scalars, vectors, matrices, operands, etc. provide powerful means for implementing fuzzy sets. In this paper we will show the capabilities of APL to represent knowledge in a linguistic form.

An Implementation of Prediction Calculation Using APL and Clipper I. Luneva, S. Obratsov and A. Shirkovich. A complex algorithm for prediction calculation based on non-linear algebraic models has been implemented using Clipper for the user-interface and APL*PLUS for calculations. A currency exchange rate system has been implemented as a DOS application for Windows with dynamic data exchange.

APL and Economic Education A. Buzin. The teaching of various Socio-Economic models formerly viewed as 'bourgeois theories' is enhanced by the use of APL in the development of elementary mathematical models including function research, matrix algebra, interactive analysis, and graphic interpretations.

VDE: A Virtual Data Engine for APL J. Merrill. This paper discusses the development history of a data-handling mechanism known as VDE, which stands for Virtual Data Engine. My client, a major international bank, has been able to take advantage of the ease with which a flexible APL-based design can be extended into areas not even imagined during initial planning.

Is APL a Competitive Language in the International Olympiads in Informatics I. Mironov and V. Bargatchev. After participating in the International Olympiads for Informatics the authors the authors evaluated the effectiveness and appropriateness of APL in the solution of the same set of problems.

The Impact of APL on First Year Mathematics W. Spunde. In 1993, the Australian Federal

Government's Committee for the Advancement of University Teaching (CAUT) funded a project to develop an integrated computationally rich first year mathematics course built on the use of APL as an integral part of the instruction. This article outlines how the standard syllabus was amended to respond to the available computing power for elucidating mathematical topics and what mathematical skills were improved or neglected by the novel presentation.

Application of an APL Formula Manipulation System to Different Models of Computer M. Kikkawa, K. Sugawara and Y. Washio. In the course of studying the application of APL to high school education to help students in their understanding of mathematical concepts, many teachers encountered difficulties in that the results of the study could not be used in their classes because the school had provided them with models of computer which cannot run APL. The authors and their co-workers found a way to solve such a difficulty by porting the study file from NihonGo APL to t-APL. This paper discusses problems encountered in the classrooms, the new system applied to three popular models of personal computers, the actual use of the APL study file in eight schools in learning factorisation, and evaluation of both teachers and students.

Report of the Kronstadt Project B. Shipigel and T. Guryeva. During the APL92 international conference, held in St. Petersburg in July 1992, a conjunction of several independent incidents led to a fruitful and interesting APL educational project at the school of Kronstadt.

Probabilistic Studies of I CHING with the Help of APL P. Luksha. I Ching, the ancient Chinese fortune-telling book, is the basis of the whole Chinese culture. It consists of 64 8-line pictographs called hexagrams, and commentary texts for them. The author presents mathematical problems connected with I Ching and provides their solutions via APL functions.

Useful Extensions of Matrix Divide N. Thomson. This paper shows two generalisations of the use of the APL2 domino function. Domino has two roles, namely the solution of non-degenerate systems of linear equations, and the estimation of linear least squares fits. The duality between these is exploited to extend domino, using Newton-Raphson iteration to find roots of non-linear systems of equations on the one hand, and to perform non-linear curve fitting on the other. APL2 operators are very appropriate, and the J approach is also described. Simple idioms give consequent fitted values and Analysis of Variance calculations. A compact tool-kit is given summarising in parallel columns the APL2 and J code given in the paper. The brevity of the latter speaks for itself.

Alberth's Method for the Parallel Iterative Finding of Polynomial Zeros T. Chi Chen. Alberth's method for iteratively finding polynomial zeros using simultaneous interacting guesses is seen to be a special case of Newton's method with parallel anticipatory implicit deflation, changing the single polynomial problem into that for a system of rational functions. It subjects all

iterates to the same iterative process, and discourages coincident convergences, at the cost of greater computing labour and slow convergence to multiple zeros. Serial methods interspersed with standard deflation prove much faster though suffer from uneven roundoff error accumulation. Factors affecting convergence include initial guess choices, problem symmetry, and sluggish clusters. The experimental APL program PAIDAX installs the proposed asymmetric initial guesses and immediate updating strategy. Tests show that the favoured scheme does provide welcome asymmetry and speedup, though convergence to multiple zeros remains slow.

Statistical Simulation and Illustration Using the APL2/AGSS Computing Environment P. Lewys. The purpose of this paper is to show that multi-factor statistical simulations can now be very easily and conveniently implemented in the APL2/AGSS computing environment. In particular, this environment has all of the necessary statistical methodology and the colour graphics facilities one needs to delineate the effect of various factors in a statistical simulation. Moreover the main reason for using a batch language like FORTRAN — computing speed — is no longer a deterrent to using an interactive environment like APL/AGSS. With this APL/AGSS version of SIMTBED, most statistical concepts can be illustrated interactively on-line while teaching if a fast microprocessor is available. Several examples are given.

Analysing Time-Series in APL A. Sykes. This paper considers the provision of time-series software for statistics students to use. Specifically it discusses some appropriate graphical output, and the fitting of ARIMA models using a version of Marquardt's Non-Linear Least Squares algorithm.

Maps of Science Construction using APL and Graphics B. Makeev and A. Zueva. Maps of Science are produced using APL and graphics. These graphic representations disclose the interlinks between branches of atomic science and the rate of interpenetration between these branches.

The APL Theory of Human Vision G. Langlet. Vision is considered as a mechanism that deals with information processing by a living organism. The conjunction of several hypotheses led to a single theoretical model of quasi-automatic integration of information, which processes compression and orthogonal transforming, without involving any classical calculation; the model that was worked out completely with APL as the main research tool, will be exposed in APL also. It is based on special properties of matrices in modulo-2 integer algebra; the paper covers: considerations on information coding, explanations about the role of patterns formed by retina receptors, about the behaviour of the electrons within the main information-processing molecule (retinal) and about the transmission of information in synapses as an annex.

APL for Statistical Research of Turbulence Structure during the Dust Storm A. Mukanova. The Aral dust storms are statistically analyzed. Special attention is

given to experimental and theoretical investigations of desert aerosol transport problems for quantitative description of dust transfer from source to precipitation areas by wind.

LISEB: A Language for Modelling Living Systems with APL2 P. Bottoni, M. Mariotto and P. Mussio. The paper discusses the design and implementation of LISEB, a class-based language built on top of APL2 to respond to challenges posed by modelling living systems from a medical point of view. LISEB capitalises on several features of APL and on some lessons learned from its history. Living systems are modelled as open systems in which concurrent mobile autonomous agents interact. Modelling of these properties required extensions of traditional object-oriented paradigms and of their previous translations under APL: a) every object performs a sequence of actions dynamically modified to adapt to circumstances; b) a new policy of message management is introduced uniformly encompassing broadcast and directly addressed communication. An example of a simulation using LISEB illustrates these concepts.

A New Graphical Interface for APL Dick Bowman. Historically, APL has had two significant advantages over many other problem-solving computational tools. The first of these is that APL's language elements have a consistency and breadth that remains unchallenged, at least for the tasks for which those elements were designed. It may be arguable that some widening of scope beyond these linear algebra origins is long overdue, but that is beyond the scope of this paper. The other historic advantage of APL, its provision of an interactive problem-solving environment, has long since been eroded. The purpose of this paper is to explore some possible ways in which the face that APL presents to its users could usefully be enhanced. These ideas are presented from the viewpoint of an APL user and there is no consideration of the practicality of their implementation into viable APL interpreters.

A New Development in the Technique of APL Modelling using APL*PLUS and a C++ Compiler D. Gusev and I. Pospelov. A compiler that can convert non-interactive APL*PLUS programs into machine-executable code uses C++ as an intermediate language. The compiled code is machine-independent and runs three times faster than the original APL program.

An Object-Oriented APL D. Selby. Object-oriented programming has become an important, and accepted part of the computer software industry. Nearly every new operating system that has recently arrived or is scheduled to arrive soon is object-oriented in nature. If APL2 is to continue to be a mainstream computational environment, it is very important that it becomes object-oriented. Finding a way that APL2 may sit comfortably in these new operating systems is important. Unfortunately, in both implementations and current programming teaching philosophy, existing APL2 does not easily offer the end-user the ability to adopt this new approach. This paper proposes some small changes to the basic APL2 language definition which will permit objects to be defined and used. It also suggests some

major changes to the end-user's programming methodology.

Restriction Riddance: More Power to Parentheses F.H.D. van Batenburg. This paper proposes to enhance the power of parentheses. Instead of restricting parentheses to expressions that yield an argument of a function, as in "(2-3)+6", I propose to use parentheses also for arguments of operators. If operators are allowed to have not only functions, but also parenthesised expressions as arguments, APL expressions will be more powerful. It will be shown that this proposal enables the programmer to make various types of expressions more readable and even make it possible to experiment with different types of structured programming constructs.

Programming in J/Windows C. Burke. J has been available as shareware for the last four years. The core language has been largely complete from the outset, but it is only comparatively recently that the implementation (at least for the PC) has included features needed for serious application development. The most significant change was the introduction of Windows support in release 6, which for the first time allowed full screen applications to be written in J. Following the release of 6.2, which included many additions to the Windows support and a much better session manager, I started using the Windows version of J in small applications, for which I would normally have used APL. Since then I have switched over to using J for almost all development, except for client work where APL is specifically required. I believe that J is largely comparable to commercial APLs for application development and will soon be superior. In this paper, I will describe programming the J/Windows interface, and discuss some practical considerations in using it. The focus throughout will be on Windows — in what follows, for 'J', read: 'J/Windows'.

TAGS: Trains, Agendas and Gerunds K. Iverson and R. Hul. Trains, agendas, and gerunds have been available in J for some time. This paper reviews their definitions and uses, and presents some new extensions that enhance their utility.

Posters

APL and Demographic Models Umberto Piasentin, Maria Stovall, David Lanari. A demographic projection model for regional planning will be presented. The base year population of the region (about one million inhabitants) was disaggregated into age/sex categories (one hundred annual age classes by each sex). The APL language was chosen for its strength to develop the model and to realise a friendly interface for the planner. The model can be used not only for a neutral projection of the population growth but also for a simulation of important changes in the demographic system (migration, fertility,...).

APL as Multi-lingual Communication Tool: A Small Machine Translation System for English-Esperanto and Japanese-Chinese Toshio Nishikawa. A small machine translation system was developed in APL. It demonstrates that APL is capable of manipulating character string data as well as numerical data, applied even to machine translation; and simultaneously it attracts APLers' interests to APL as an international communication tool. For the time being, as a prototype of machine translation, we installed the easiest language pairs, English-Esperanto and Japanese-Chinese translations.

Building Science Models in APLomb Ian A Clark. APLomb is an educational software construction set for the Apple Macintosh. It is not just one product but a range of products based on interconnecting parts. The parts, called "kompos" (derived from "component"), provide ready-made screen furniture such as gauges, arrays of icons, coloured pictures, sounds and sprites. A powerful function-definition language (ISO-standard APL) makes the kompos do useful things. You can liken APLomb to a toy construction set based on snap-together parts.

MERCIA Software Limited

APL Programmer

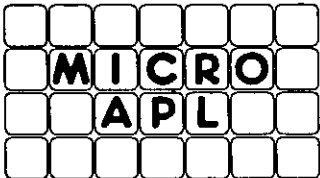
Mercia Software is a growing company specialising in materials management systems. Our LOGOL product is the UK's leading PC and UNIX based system for finished goods inventory control, and is increasingly becoming the number one choice world wide.

As a result of new developments in Graphical user interfaces and Client / Server technology we are seeking technically excellent people to join our small but highly motivated team. As well as having a sound background in APL and its use in large OR based systems, you will also be skilled in Windows and Relational Database Management Systems.

A knowledge of some (if not all) of the following environments, as well as a desire to learn new techniques, is what we are after.

APL * PLUS II / III Dyalog APL C++
MS DOS and Windows UNIX and X-Windows / Motif etc.
Paradox, Oracle or any other mainstream database

We would like to hear from you if you have at least 3 years experience in these fields, and want to get involved in a great product with a future aimed at providing practical business solutions using the latest in IT. The post is Birmingham UK based and attracts a competitive salary (dependant on experience), profit share scheme, as well as a challenging and enjoyable working environment.

—		APL Preferences	▽
APL.68000			
Platforms		Versions	
☒ RISC System/6000		☐ Level I	
☒ Apple Macintosh		☒ Level II	
☒ Commodore Amiga			
☒ Atari ST			
APL for GUIs			

MicroAPL Ltd., South Bank Technopark, 90 London Road,
LONDON SE1 6LN, UK

Tel: 071 922 8866 Fax: 071 928 1006
Applelink: microapl Internet: microapl@applelink.apple.com

British APL Association News

Minutes of Extraordinary General Meeting 2pm Friday 25 March 1994 at the IEE

Membership Rates The proposal, to reduce the membership rates for overseas members to reflect that Vector is the sole advantage of membership, was approved *nem. con.*

Membership Year The proposal, to allow the committee flexibility in determining the membership year for new members, was accepted *nem. con.*

Cheque Authorisation The proposal was to allow the committee to authorise new signatories to the Association bank account. An amendment was proposed by the Secretary, seconded by George MacLeod and passed *nem. con.*, that the words "member of the Management Committee or employee of the Association" in the proposal be changed to the word "person". The amended proposal was approved *nem. con.*

Date of AGM The proposal, to change the regulations to reflect that the AGM must be in May to comply with BCS regulations, was accepted *nem. con.*

EGM Regulations The proposal, to correct a typographical error in the original regulations with regard to the EGM, was accepted *nem. con.*

Report of Annual General Meeting 2pm Friday 27 May 1994 at the IEE

Agenda

- Minutes of the AGM 11 June 1993
- Minutes of the EGM 25 March 1994
- Chairman's Report
- Management Committee Officers' Reports
- Audited Statement of Accounts
- Proposals for Alterations to the Regulations
- Proposals for Alterations to the Fees
- Report on the Election of Officers for 1994/1995
- Appointment of Auditors for 1994/1995

The Minutes of the 1993 AGM were agreed.

The Minutes of the EGM are reported in this issue of Vector.

The Chairman reported a small increase in membership. Also that negotiations with Oxford University Press concerning the proposed publication of a fully refereed Journal are proceeding satisfactorily. Also that arrangements for the Swansea Conference will appear in Vector 11.1.

The Chairman then made the presentation of the Outstanding Achievement Award to Adrian Smith. He noted Adrian's much appreciated book, widely circulated software (in particular free APL PostScript and TrueType fonts), and the APL Typewriter; also the close support he gives to the production of Vector. Adrian responded with two well chosen cartoons from *The Guardian* on the overhead projector.

The Treasurer circulated the Statement of Accounts. A member asked for comment on the excess of expenditure over income and was told that the Management Committee had a policy to distribute some of their reserves to pay for projects supportive of the aims of the Association.

Alterations to the Regulations and Fees were agreed at the EGM.

The Chairman reported that there had been no requirement for a postal ballot as the nominations for Committee posts had been uncontested. As there had been no nomination for the post of Treasurer, the Committee had asked Nicholas Small if he would serve another year. He agreed to be co-opted.

1994-95 Committee

Chairman	Dr Alan Mayer
Secretary	Sylvia Camacho
Treasurer	Nicholas Small
Journal Editor	Anthony Camacho
Activities	Duncan Pearson
Education	Dr Ian Clark
Technical	Jonathan Barman
Projects	George MacLeod
Publicity	David Eastwood
Recruitment	Mark Harris

Summary of Annual Accounts 1993/94

	1993/94 (R&P) £	1993/94 (I&E) £	1992/93 (I&E) £
Income/Receipts			
Subscriptions	14736	14344	14979
BCS services	0	0	1152
Bank interest	2145	2276	4584
Vector advertising (incl. VAT)	4985	5795	4056
Conferences & seminars	2188	2618	0
Other	1719	219	1285
Total receipts	25774	25251	26056
	-----	-----	-----
Expenditure/Payments			
Meetings	2793	2270	2526
Administration	3340	2731	2959
BCS services	396	396	1152
Vector production and despatch	13890	14627	13815
Education Vector supplement	3395	3429	3396
Outstanding Achievement Award	500	500	500
Projects	3648	3546	1023
Conferences & seminars	4567	3096	0
Other	3832	3934	1173
Total payments	36360	34528	26544
	-----	-----	-----
Assets summary:			
Bank and other balances		63485	74072
Debtors		6543	5510
Creditors		(6117)	(6393)
Net assets		63911	73188
		-----	-----

Notes

1. Pence figures have been omitted, so columns may not add exactly.
2. The value of stocks of Vector and the Software Library have not been assessed, nor has the value of the Association's computing hardware and software.

Treasurer's Remarks on the Annual Accounts.

For 1993/94, figures are shown both as income and expenditure, i.e. revenues strictly relating to the activities of that year, and as receipts and payments, i.e. what goes in and out of our bank account. The comparative figures for 1992/93 relate to income and expenditure.

In arriving at income and expenditure from receipts and payments, the item **Other** under Receipts was adjusted for late payments not identifiable with specific invoices; from **Conferences & seminars** under Payments was subtracted the advance payment for *APL in Business*. The item **Other** under Payments includes the cost of a portable PC.

British APL Association: Regulations

1. Name

The Group shall be called *The British APL Association*.

2. Affiliation

The Association is and shall be affiliated to the British Computer Society as a Specialist Group.

3. Objects

- a) To promote the use of APL;
- b) To develop awareness and competence in APL;
- c) To represent the interests of Association members with other bodies;
- d) To contribute to the development of the language and definition of international standards.

4. Constitution

The British APL Association shall consist of:

- a) President and Vice Presidents;
- b) Chairman, Secretary and Treasurer;
- c) Other elected officers;
- d) Individual fee paying members;
- e) Individual fee paying student members;
- f) Corporate members;
- g) Sustaining members;

5. Members in Good Standing

A member in categories 4(d), 4(e), 4(f) or 4(g) is in good standing with the Association by virtue of having paid all membership fees due to the Association for the current year. Any membership which is not renewed within the first four months after becoming due is deemed to have lapsed.

6. Appointed Officers

- a) A President may be appointed at the discretion of the Management Committee.
- b) A maximum of four Vice Presidents may be appointed in recognition of service to the APL community at the discretion of the Management Committee.
- c) Appointed Officers may serve for a period of one year with up to two consecutive renewals. Non-contiguous periods of office are permitted with no limit.
- d) Appointed Officers shall have the right to attend free of admission charge all General Meetings of the Association and all events staged by the Association.

7. Elected Officers

- a) The Elected Officers shall be Chairman, Secretary, and Treasurer who must be individual fee paying members in good standing.
- b) Other officers may be elected to fill posts created by the Management Committee.
- c) It is the duty of the Elected Officers to attend meetings of the committee.
- d) Election: The officers shall be elected by postal ballot - to serve from 1 May in the year of election until the following 30 April. Individual members will be sent one ballot paper; corporate and sustaining members will be sent five to be cast by five users of APL where possible.
- e) Casual Vacancy: A vacancy occurring during the term of office may be filled by an appointment by the Management Committee.
- f) The occupant of any post who has held it for three consecutive years is not eligible for election to that post in the fourth year.

8. Management

- a) The affairs of the Association shall be managed (subject to the control of the AGM) by a Management Committee comprising:
 - i) Appointed Officers
 - ii) Elected Officers
- b) Co-option: The Management Committee may co-opt members as required, normally to hold office until the following 30 April.
- c) The General Committee of the Association shall comprise the Management Committee together with members of Sub-Committees and Working Parties which from time to time be set up by the Management Committee.
- d) Meetings: The Management Committee shall meet at least four times in its year of office and frequently enough to carry out the business of the Association properly.
- e) Notice: At least fourteen days notice of the place, date and time of meetings shall be given to each member of the relevant committee.

- f) Quorum: The business of the Association may be transacted by not less than half the members of the Management Committee.
- g) In the absence of the Chairman, the Management Committee shall elect one of its members to take the chair for the meeting.
- h) Voting: In determining a question by vote of the majority of members present, each having one vote, the chairman of the meeting shall have a second or casting vote.
- i) Standing Committees: The Management Committee is empowered to set up Standing Committees to deal with matters that require continuity. A member of the Management Committee shall sit on each Standing Committee.
- j) Sub-Committees and Working Parties: The Management Committee may set up at any time sub-committees and working parties responsible to the Management Committee which shall appoint a Chairman and provide appropriate terms of reference.

9. Annual General Meeting

- a) Each year the Association shall hold an AGM in May.
- b) Notice: The Secretary shall send, at least 28 days before, notice of the date, time and place of the AGM, together with the Agenda, to all members of the Association. For this purpose a notice printed in the official publication of the Association shall be considered sufficient.
- c) All members have the right to attend the AGM for which there shall be no attendance charge.
- d) Agenda: The following items shall be included:
 - i) Minutes of the previous AGM;
 - ii) Minutes of any EGM held since the previous AGM;
 - iii) Chairman's Report;
 - iv) Management Committee Officers' Reports;
 - v) Audited Statement of Accounts;
 - vi) Proposals for Alterations to the Regulations;
 - vii) Proposals for Alterations to the Fees;
 - viii) Ratification of Auditors.
- e) Nominations: A Nominating Committee will draw up a list of Candidates for each elected post; additional candidates may be nominated by a proposer and seconder. All candidates, proposers and seconds must be members in good standing. Closing date for nominations is 1 February.

- f) Voting: Every question at the AGM shall be decided by a majority of the votes cast. Individual members of the Association each have a single vote. The accredited representative of each corporate and sustaining member have five votes which may be split as desired. If a vote is tied, then there shall be a recount; if this fails to resolve the matter the candidate with the least votes is removed and the vote retaken, this process continuing until there is a decisive vote - or it cannot be continued, in which case the Chairman has a casting vote.

10. Extraordinary General Meeting

- a) An Extraordinary General Meeting (EGM) shall be convened on a resolution of the Management Committee or within five weeks of receipt of the Secretary of a requisition signed by no less than one quarter of the current membership in good standing stating the business to be transacted at the meeting.
- b) Regulation 9, except subparagraph (d) and (e) shall apply.
- c) An EGM shall transact only such business as is specified in the resolutions or requisitions concerning it.

11. Finance

- a) The Association shall follow the BCS Financial Guidelines as issued from time to time insofar as they apply to Specialist Groups.
- b) Bank Account: In accordance with the BCS Financial Guidelines, the Association shall have at least one Account (Account A) at Lloyds Bank, Langham Place Branch, used for normal running expenses. Other accounts at the branch may be used for special events or investment funds.
- c) The financial year shall start on 1 May.
- d) The Chairman is responsible to the BCS for submitting draft budgets, recording ongoing expenditure and capital expenditure separately for each year by 30 November in the preceding year.
- e) The Chairman is responsible for making available to the BCS a receipts and payments statement at the end of every financial year (30 April) in respect of the Association's normal operations and special events; this statement to be included in the BCS annual accounts subject to audit by the BCS auditors.
- f) All cheques drawn on the Association's bank accounts must be signed by any two of Chairman, Secretary, Treasurer and any person selected by the Management Committee for that purpose.
- g) The accounts of the Association shall be audited each year by an auditor appointed by the Committee.
- h) All income and property of the Association from whatever source derived shall be applied solely to the promotion of the objects of the Association.
- i) No member of the Association shall receive payment for his services as a member.

- j) The Management Committee shall appoint dates in the year to be membership renewal points.
- k) Membership fees are due initially on joining the Association and subsequently at the latest membership renewal point in the following full year and on that date thereafter for the duration of the membership.

12. Dissolution

In the event of the winding up or dissolution of the Association any surplus assets remaining after discharge of liabilities shall automatically vest in the BCS.

13. The British Computer Society

- a) The Association shall be governed by the regulations of the BCS as these apply to Specialist Groups of the BCS. Where it is considered that a regulation of the Association is in conflict with a BCS regulation governing Specialist Group activities, the BCS rule shall apply.
- b) The Chairman of the Association must be a Fellow, Member or Associate Member of the BCS.
- c) Other Elected Officers of the Association should normally be members of the BCS.
- d) The Chairman of the Association is ex officio a member of the BCS Specialist Groups Board and is thereby eligible, if nominated and elected, to be one of three representatives of the Specialist Groups Board appointed to the Council of the BCS.
- e) The Chairman or his representative is expected to attend four Specialist Group Board meetings each year.
- f) The Association must advise the Vice President (Specialist Groups) of those members of the Management Committee of the Association who are not members of the BCS.
- g) All members of the Management Committee shall abide by the code of conduct relating to members of the BCS.
- h) The Association may use the BCS name to enhance the reputation of its own activities but must not bring the BCS into disrepute.
- i) No member of the Association may speak on behalf of the BCS without proper authority from the BCS.

News from Sustaining Members

Compiled by Gill Smith

Insight Systems

Since we introduced our *SQAPL Client* product a year ago and gave APL developers access to most multi-user SQL databases on the market, the most frequent request for an enhancement to the product has been for it to support local databases or file systems such as Access, dBase or Paradox. We are therefore pleased to announce that version 2.0 of SQAPL supports Microsoft ODBC as an alternative to SequeLink under Microsoft Windows and in the OS/2 Windows box (WINOS2). ODBC drivers are available for almost all PC file formats. Many client/server database vendors also provide ODBC drivers for their own middleware products (such as Oracle's SQL*NET). If you only need access to a single client/server database, a proprietary middleware solution from the database vendor may be an acceptable solution — but SequeLink remains the best solution in heterogeneous environments, and the only solution if you have client workstations running Unix and OS/2.

We are also pleased to announce that we have entered into redistribution agreements with both Dyadic Systems and Manugistics for the ODBC version of SQAPL 2.0. Contact your regular supplier if you want to use this product with APL*PLUS III/Windows or Dyalog APL/W version 7. Until further notice, we are still handling distribution of SQAPL for SequeLink under Windows, OS/2 and Unix, for APL2, APL*PLUS II and Dyalog APL.

Whether you use SQAPL with SequeLink or with ODBC drivers, the APL function calls are identical across all APL systems and environments in which SQAPL is available. You can easily prototype an application under Windows using a local database, and put it into production running APL under Unix to access DB2 on a mainframe. Unfortunately, although the SQAPL function calls are the same, the "local" ODBC drivers we have tested so far have been quite different when it comes to data types. Data types which have fundamentally the same internal representation can have quite different names from one driver to the next. For example, in testing ten or so drivers for PC files (dBase, Btrieve, CSV, TXT and Excel drivers from Microsoft and Q+E), we came across VARCHAR data types named VARCHAR, MEMO, NOTE, TEXT and LONGCHAR — usually (but not always) requiring a maximum length declaration in CREATE TABLE statements. You will also find data types with the same name, which have subtly different internal data types.

We are more than a little surprised and disappointed by these differences between ODBC drivers. The differences may not be of great importance to the "personal" APL user — but they will be rather irritating to the software professional who needs to be able to provide portable, scalable solutions. Our strategy to cope with the situation is to provide an Entry Level product for the personal APL user, available immediately at low cost or bundled with an APL interpreter. Within a few months, we will provide *Professional Edition* with a number of performance and portability features important to the professional software developer. Besides hiding the differences in data type naming and handling, we will try to deal with some of the most irritating differences in transaction processing (are cursors closed after a commit?) — to make it as easy as possible to write portable applications.

In the slightly longer term, we will be transferring our attention to the server side of our product. The two most important goals are to port the SQAPL Server to the mainframe and support IBM APL2 and possibly Sharp APL under MVS. We will develop an ODBC compatible version of the SQAPL Server, so that APL applications can act as ODBC servers for any ODBC-enabled client application (including MS Query, Powerbuilder and other popular prototyping and analysis tools).

Although our customers still see SQL database access as their most pressing need, it is the SQAPL Server which we think is the important product in the longer term. Although we now derive an important part of our income from the sales of APL to SQL interfaces, we make no secret of the fact that we have found very few (APL) applications where SQL had any real functional advantage over APL component files. We have seen customers move multi-dimensional timeseries structures into relational databases and be rewarded with a loss of flexibility, and a hundred- or thousand-fold increase in resource consumption. Ten users of a sales forecasting system which used to consume a small fraction of the machine when using APL component files can consume all of a big mainframe when the data moves to DB2 and they all generate the dynamic SQL statements required to multiply the sales forecast by unit price and currency rate(s), then total it on three dimensions while converting the frequency from monthly to quarterly. The resulting two-dimensional table will most often need a lot of afterburning before it is ready for presentation. Remember that retrieval is the strong side of SQL. After all, the people who designed the language named it a Query language — and anyone who has tried to write non-trivial data entry applications using SQL will agree that it is aptly named.

We believe that the SQAPL Server can provide a more attractive solution. Instead of moving the data to an SQL database, you can turn the sales forecasting

system into an ODBC or SequeLink server, which can be used by a frontend coded in APL or called from most other tools on any kind of workstation. You can even put a simple SQL frontend up which allows people to use SQL on virtual tables which are generated by APL. In this way, the APL data and services can become integrated with mainstream applications without sacrificing the advantages of APL. We feel that there are a lot of opportunities open to APL as an application server. We can provide computation engines or specialized database servers which are not only more flexible than relational databases, but also outperform them by one or two orders of magnitude for the types of data that APL is good at handling.

Manugistics Inc

Manugistics Vector Sustaining Member News

A great deal has been happening in Rockville since our last sustaining member news. The most exciting of which has to be the tremendous success we've had so far with APL*PLUS III for Windows. We officially began shipping version 1.1 in late April, although we had been selling pre-release versions for nearly two months to customers who couldn't wait and wanted to take APL*PLUS III for a spin. To be honest, even we were a little surprised at how quickly we reached the 1,000 units shipped mark. Next stop 2,000.

Our development group is busy putting the finishing touches on version 1.2, which we hope to be shipping by the end of August. Version 1.1 has been such a stable system that we have been able to spend the bulk of our time putting more features in 1.2 instead of fixes. Top on the list of new features for 1.2 is support for Visual Basic Custom Controls (version 1). We are also adding the necessary object classes to APL*PLUS III to allow developers to implement MDI style applications and `□NA` and `□WCALL` are being modified so both will support 16 bit DLLs directly, so you won't have to "thunk" any longer.

While building APL*PLUS III, our developers became so enamoured with syntactic colouring, we decided to put it in version 1.2. This really is a nice feature and along with control structures, makes reading APL code much easier. We are also enhancing the product's graphics capability by adding a "Picture" class, for persistent graphics, and a new "chord" operation to the "Draw" method. Help and prompting tools are also being added to assist the developer in implementing status line and pop-up help messages and a few aesthetic changes to the session manager are liable to sneak in as well including support for the classic APL keyboard.

We see APL*PLUS III evolving into a modular system, with add-on packages being offered by Manugistics as well as other parties. At the time of this writing, an ODBC module, developed by Insight Systems, is about to go into beta test. When commercially released, the product will be available through Manugistics' regular distribution channels. Please contact Manugistics or your local APL*PLUS re-seller for more information.

Last week we held another successful annual international dealer conference. Resellers from various countries came to Rockville for 4 days of speeches, round-table and one-on-one discussions and networking. We want to thank all of our friends for making the trip and participating this year and we hope to see even more next time.

As always, Manugistics welcomes all suggestions, feedback and inquiries from Vector readers. To facilitate this, we have added the following two Internet addresses. In the United States, e-mail us at aplsales@manu.com. Outside the United States, use the address intl@manu.com.

MicroAPL Ltd

In the last issue of Vector, we announced details of an upgrade to our Macintosh versions of APL68000 Level II. We have now completed work on Version 3 of APL68000 Level II for the Macintosh and it represents a major enhancement to the leading APL2-compatible interpreter for the Apple Macintosh family.

'Native' Power Macintosh Support

With the advent of the new RISC-based Power Macintosh systems, Apple has dramatically improved the potential performance of Macintosh systems, *provided* applications are re-engineered to run 'native' on the new PowerPC processors. For over two years, MicroAPL has been working closely with Apple on tools for converting 68000-family applications to the PowerPC. One of the fruits of this work is that the full power of the Power Macintosh is available to APL applications running under Version 3 of APL68000 Level II. This provides one of the fastest APL interpreters available for a small computer, particularly for code that makes heavy use of floating point.

New Object-Based GUI Programming

Included in Version 3 is a new system function `□WI` which provides a simple, consistent and object-based method of controlling user-interface elements such as buttons, list boxes, pop-up menus and pictures. Objects are created using `□WI`, and their behaviour is defined by setting *properties*. And to help APL users

develop cross-platform applications, the implementation of *ØWI* is largely compatible with Manugistics' APL*Plus III for Windows, making it straightforward to move applications between the Macintosh and Windows PCs.

The object classes supported by *ØWI* are Window (or Form), Document, Dialog, Static text, (or Label) Button, Radio button (or Option button), Check box, Edit field, List, PopUp menu (or Combo), Icon, Picture, Scroll bar, Line, Rectangle, RoundRect, Arc, Movie and Timer.

New Callback-Based Event Handling

In addition to the *GETEVENT* function previously available, Version 3 of APL.68000 Level II implements callback-based event handling. A 'callback' is an APL function (or other expression) which is run when a given event occurs (for example, if a Button is pressed).

To give you full control of event handling, callbacks are run during the *ØWE* (wait for event) system function, to which you pass a timeout value as the right argument. This allows applications to continue processing events during lengthy calculation or other processing. You also have full control over whether APL's default event handling takes place, which together with an event triggering feature makes it possible to write APL custom controls.

Play Quicktime Movies from APL

Amongst the object classes supported by APL.68000 Level II is the Movie object. This allows an application to take advantage of Apple's Quicktime movie technology, with the possibility of multi-media applications under APL control.

Improved Programming Environment

Many enhancements have been made to the user interface, further developing APL.68000 Level II's position as one of the most user-friendly APL programming environments. These include:

- New Windows' menu, with multiple sessions identified by workspace name, and improved dialog design.
- Support for Apple Events — multiple workspaces may be launched simultaneously at start up, and double-clicking on a workspace icon during APL execution opens a new APL task.
- Improved File Menu with Import and Export options to exchange Transfer format files with other APLs
- New Print menu items allow print of workspace listing as well as contents of any APL window.

- Improved editor facilities including new find/replace facility.
- Closer conformance to Apple's System 7 user-interface guidelines.
- Removal of limits on number of edit windows open simultaneously (up to overall limit of 99 open windows).
- Up to nine APL tasks can run simultaneously!
- New user preferences dialog for startup configuration.

100% Upwards Compatibility

In implementing the new features in APL.68000 Level II, we have retained 100% upwards compatibility for existing APL applications — even if you have written 680x0-family auxiliary processors and want to run them on a Power Macintosh!

Royalty-Free Packager Included

MicroAPL pioneered the concept of royalty-free runtime APL licences, allowing you to distribute your APL applications without paying licence fees. Version 3 of APL.68000 Level II now makes this even easier by including as standard a 'Packager' which turns your APL workspace into a standalone Macintosh application.

Business Graphics Workspace

The new PlotMac workspace allows you to create simple business graphics with a few easy APL commands, automatically scaling and labelling axes. You can create line and scatter plots; histograms; and pie charts. The graphs can be shown on-screen, printed or exported to other applications.

Free APL Toolkit

Finally, we have included a copy of the public domain APL Toolkit produced by the Toronto APL SIG which contains a wide variety of useful APL utility functions that will run under all types of APL interpreter.

Requirements

APL.68000 Level II Version 3 runs on all models of Apple Macintosh, including running in native mode on the PowerMac range, and requires Apple System 7.0 or later. APL.68000 Level II Version 3 will automatically use the hardware coprocessor on those 68000-family Macintosh computers which support 68881/2 operations.

Soliton Associates Limited

Viewpoint, Soliton's APL-based end-user report generator, previously available only for Sharp APL MVS, is now available for Sharp APL UNIX.

DBIUtils, the *Viewpoint* Application Programming Interface, provides APL access to *Viewpoint* data, and allows integration of *Viewpoint* into custom APL applications.

As well as operating upon its own internal databases, *Viewpoint/UNIX* can operate upon external databases, including DB2. Via *NSVP*, Soliton's *Network Shared Variable Processor* product, *Viewpoint/UNIX* can access mainframe data, including DB2.

Dyadic Systems Ltd

Dyadic is pleased to report the availability of Dyalog APL/W Version 7 that was announced at the end of April. The new release contains a number of improvements and additional features that were not included in the April announcement.

One of the most important additions is the implementation of a built-in Grid object, which has proven extremely popular among audiences that have seen it demonstrated. Originally Dyadic had planned to rely on third-party custom control objects to satisfy the requirement for a spreadsheet-like editor. However, Grid objects designed for Visual Basic turn out to be highly scalar-oriented and not particularly convenient for APL. Dyadic therefore decided to develop its own built-in Grid object which is tailored specifically for the APL language.

The Dyalog APL Grid looks and behaves like a typical spreadsheet. The body of the spreadsheet is (naturally) a matrix, and individual cells can be numbers, character vector or character matrices. This makes it rather more than a simple numeric editor as it will handle the more commonly used nested array structures too. You can specify your own column titles and row titles or you can let the Grid label them with typical spreadsheet defaults. You can also disable the titles to provide a simple scrolling list. The Grid supports 'in-cell' editing or lets you use an external edit field. This is done by providing an association between the Grid and an Edit object; indeed it is possible to associate different Edit objects with different cells. Once this association has been done, APL manages the input entirely automatically; and, because the Edit object now supports Numeric, LongNumeric, Currency, Date and Time fields, APL performs most of the routine validation and formatting for you. The Grid also allows colours and fonts

to be specified at the individual cell level, so you can readily customise its appearance.

The second major new feature is the provision of an Auxiliary Processor that allows you to access databases that support the Microsoft ODBC standard. Such databases include Microsoft Access, dBase, Paradox, Oracle, Ingres and Sybase. The Auxiliary Processor has been developed for Dyadic by Insight Systems who are specialists in this field. Insight are also the author of SQAPL which provides similar database connectivity using the SequeLink product, and which is already in production use at a number of large Dyalog APL sites. The ODBC interface from Insight Systems is included with Dyalog APL/W Version 7 at no extra charge and is fully supported by Dyadic and the author. When ODBC drivers begin to appear for Unix systems, Dyadic intends to provide the same functionality for the Unix implementations of Dyalog APL.

Another addition is the provision of a DOSFNS DLL together with a set of complementary APL functions. These functions perform many of the standard tasks of `□CMD` without the overhead of starting a new DOS command processor.

Vector Back Numbers

Back numbers of Vector are available from:

British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO6 4JJ

**REDUCED
PRICE**

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

THE EDUCATION VECTOR

July 1994

Editor Ian Clark

This Education Vector has been reprinted from VECTOR Vol.11 No.1. VECTOR is the Quarterly Journal of the British APL Association. For more information about the British APL Association, please contact: Anthony Camacho, 11 Auburn Rd, Redland, BRISTOL, BS6 6LS Tel: 0272-730036.

Contents

Editorial	Ian Clark	30
Jot-Dot-Floor	Ian Clark	31
J: a Second Lesson	Donald B. McIntyre	36

Ian Clark
IAC/Human Interfaces,
9, Hill End, Frosterley,
Bishop Auckland,
Co. Durham DL13 2SX.

Tel: 0388-527190
Email: clark.i@applelink.apple.com
or 100021.3073@compuserve.com

Editorial

by Ian Clark

Readers of Education Vector will join with me in expressing their gratitude to the outgoing editor, Alan Mayer, who has staunchly edited EV since taking over from Alan Sykes in October 1991. It falls to me as the new Education Officer to occupy (I won't say "fill") the gap left by the two Alans. Since Alan Mayer has been elected Chairman of the 1994-95 Committee, I know that I don't have to "fill the gap" in the substantive sense, since I can rely on his continued assistance and encouragement. The same goes for Adrian Smith who would have been content to typeset this edition of EV along with the rest of VECTOR with only infinitesimal input from me.

However, encouraged by both Alan and Adrian to put my mark on this little publication, I've decided to introduce my first innovation, a column of jottings on rock-bottom educational matters. Since it's simply got to have a techie APL title, what better than "Jot-Dot-Floor"?

EV wants to appeal to new readers, even at the risk of telling existing readers what they know perfectly well already. You will begin to see certain elementary facts and well-known sources being quoted time and again, just for the benefit of newcomers. Your editor plans to grasp the IQ-knob firmly and turn it down to just below 99, earning just above zero academic kudos in the process. He plans to spoon-feed the APL novice and those new readers who aren't sure yet whether they want to become APL novices or not. Deep studies will be welcome, as ever, but always with an eye to publishing just the simple parts, giving a contact address for extended information, or even offering it for publication in the main body of VECTOR. Here in EV you're firmly in the nursery. Please write and say what you think of this policy as the months go by. This isn't a new departure, but we get the impression it's what EV should be doing more and more.

This issue carries *The Second Lesson* of Donald McIntyre's introductory J tutorial. Some APLers see the J language as a sort of cuckoo in the nest and may wonder what justifies devoting the major part of an issue of EV to it. For my part I'm delighted to do so. I had wondered in the past whether a J tutorial would not have been better placed in the main body of VECTOR, but I'm convinced that EV is the right place for it. For one thing, it ensures that the two lessons of the tutorial come in a handy, easy-to-distribute form, fit for dissemination to a wide audience. For another, I suspect that career-APLers are going to be the last people to appreciate the advantages of J for educational exposition — and that's

something that will only emerge as more and better educational resources using J get published.

For readers who have only just joined us for this issue, EV forms a bound-in supplement to VECTOR, the official journal of the British APL Association, but EV is sent by itself to a separate mailing list of teachers and educationalists, to whom it primarily addresses itself. The subscription to EV is free. Provided you are a teacher, lecturer or bona-fide educational professional, you don't have to be a member of anything except EVML — the Education Vector Mailing List! Therefore, if you don't already receive your own copy of EV, or know of someone who ought to, simply write to me and I'll put it right.

Jot-Dot-Floor

by Ian Clark

The National Curriculum is Dead - Long Live the National Curriculum. A consultation process now begins, from which whatever emerges may bear little resemblance to the contents of the set of white folders that everyone in the educational trade in England, and not a few teachers, have zealously collected and studied. There's hardly an educational supplier who hasn't got the existing structure of *Key Stages* and *Levels of Attainment* off backwards, at least in their own subject, because unless a supplier could state exactly what boxes in a pupil's grid of AT levels could be ticked-off as a result of the pupil running the product, he hadn't a ghost of a chance of selling to British schools. The NC defined a market which was created by an Act of Parliament, just like vehicle insurance or personal pensions. In today's Britain the best sort of market to be in is one in which people have to buy your product by law, because nobody has any money to buy anything else.

Someone once buttonholed me and said wouldn't it be nice if EV could publish some classwork, running on almost-free software like I-APL, which a teacher could simply set before the class and then tick off the boxes on the attainment-tracking grid — or, better still, have I-APL do it automatically. Dream on, MacDuff. However one good thing about the now-obsolete NC (1991) Order documents was that they gave some neat projects, or "programmes of study" as they called them, plus examples, to meet the so-called "statements of attainment". In future issues of EV we shall discuss these examples in the Mathematics and Science curriculum in the idiom of I-APL. This will go some

way towards MacDuff's Dream, or would do if they weren't going to change it all. Another teacher tells me I couldn't choose a worse time to run a series on the National Curriculum, but is it? Presumably the new NC will only be a delta-change on the old one (a minus-delta one hopes!) so a discussion of the existing one will not be completely wasted. And there's also the promised consultation. Ah, well.

Another subject of perennial interest to mathematics teachers it seems is prime numbers. A paper accepted for APL94 is entitled "*The First One Hundred Million Prime Numbers and APL with Applications in Public Key Cryptography*" by Prof. Henriques. It seems that the purer the mathematics the spookier its applications nowadays. Ted Emms' article "*A Note on Primes*" (Education Vector 10.4) has prompted a contribution by Guido Leeten of an on-line investigation into complex integer primes, i.e. numbers of the form $a|b$ where a and b are integers. Just the stuff for a Key Stage 4 project. Here's some likely questions: If a and b are primes, is $a|b$? And vice-versa? Is prime factorisation still unique? Does the set of complex integer primes contain the real primes? Watch out for Mr Leeten's researches in a future issue.

One of the things *Mathematica* claims to do well is handle very large numbers. Most computer languages top-out at 2^{64} and APL is no exception. In fact I-APL (our pet APL interpreter in EV) only gives you 10 significant figures. Which puts a ceiling on classroom investigations — or so you might think... (shush, experts, I'm not talking to you). However, whole numbers don't really get interesting until they get big. So what do we do? Trash I-APL and go and spend £450 or so on *Mathematica*? Not at all — at least not for that reason. Here's a cautionary tale from the distant past.

A certain Chinese sage once performed a signal service for his emperor and was invited to name his price, provided the Lord High Treasurer agreed. Now the LHT was a stingy chap, so calling to mind the unfortunate experience of his friend the Pied Piper, the sage took a chessboard and set one grain of rice in the first square, two in the second square, four in the third square and 8 in the 4th. He then invited the LHT to fill the remaining squares, doubling the previous square each time — and the total would be his fee. The LHT, not possessing his own copy of I-APL, promptly agreed, and lived (though not very long) to regret it. The moral is twofold: (1) get your copy of I-APL without delay, and (2) don't sign any deal until you've costed it in real figures.

How much rice did the sage ask for? A binary numeral consisting of 64 ones, or (using I-APL) $64 / 1$. Each one represents a square, and of course each successive one is worth double the previous. What's that as a real number?

```

      2164/1
1.844674E19

```

which is more than all the rice in China, even at a milligram a grain.

But wouldn't it be nice to see all 20 digits of that. Especially by a general procedure that handles numbers up to the limit imposed by workspace capacity.

Define G as

```
G:⊠'Z←Z ADD10 Z'
```

Then running G 64 times starting with $Z←1$ gives one more than the required number, as a vector of decimal digits:

```
1 8 4 4 6 7 4 4 0 7 3 7 0 9 5 5 1 6 1 6
```

Why *one more* than the required number? Well, what you see at each iteration is the number of grains on the next square. The first square has $1=2*0$ grains, the second has $2*1...$ and the 64th has $2*63$. But the sage has submitted an expense claim for all of these added together, which just so happens to be $(2*64)-1$. Why? Because when you add 1 to a binary number 111...1 all the bits flip over and you get 1000...0.

Also it is well known (to those of us smart enough to add up a series) that for any X besides 2 and (natural) N besides 64 the series: $+/X*⊠N$ sums to: $(1-X*N)÷(1-X)$ provided $⊠IO=0$. Write these expressions into a function:

```

      ∇ Z←X TRY N;⊠IO
[1]   ⊠IO←0
[2]   (1-X*N)÷(1-X)
[3]   +/X*⊠N
      ∇

```

and try different values of X and N . Does it work for fractional X ? I'm not so dumb as to ask if it works for fractional N .

Come to think of it — and APL is meant to be a tool of thought — why *doesn't* it work for fractional N ? Anybody care to try developing a theory of fractional series? If Mandelbrot can have fractional dimensions, why not? It would be the biggest thing in Functional Analysis since infinite series.

Meanwhile, back at the nursery, here's a fairly robust version of *ADD10*:

```

V ADD10[[]]V
V Z←X ADD10 Y;CARRY;R
[1]  R adds 2 decimal vectors treated as long integers
[2]  R←~1+1,(pX)[pY R--the greater length (never null)
[3]  Z←φ(R+φX)+(R+φY) R--the straight sum
[4]  →(~V/CARRY←(10≤Z),0)/0 R--stops recursion if all CARRY=0
[5]  Z←(10|Z) ADD10 (~1+CARRY)←CARRY R--drop leading 0 from CARRY
V

```

also (to save you entering *G* sixty-four times) a function *REPT* to be used like this: 'G' *REPT* 64

```

V ZZ REPT ZM;ZI
[1]  Repeats given line ZM times
[2]  ZI←0
[3]  LOOP:→(ZM<ZI+ZI+1)/0
[4]  +ZZ
[5]  +LOOP
V

```

Is there anyone out there using Macintoshes in class? Send me a blank disk with an SAE (or the cash equivalent) for a copy of my big-number workspace, plus a copy of I-APL/Mac to run it. PC users can be accommodated too (the workspaces are byte-compatible).

How does *ADD10* work? Let's simply say that it adds the two numerals *X* and *Y* (having made them the same length). This ends up with a vector *Z* possibly containing "digits" greater than 9, but

$10|Z$

fixes that, and

$CARRY←(10≤Z),0$

computes the ones to carry. *ADD10* then calls itself to add *CARRY* into *Z* and that's all there is to it.

How's your long multiplication? Anybody want to try coding *MUL10*? *MUL2* is a bit easier (pun intended) and can be written to illustrate how the computer does it, especially if you use logic primitives instead of addition. Say, wait a minute! I've got AND, OR, NOT, NAND and NOR on the keyboard, but where's that vital logic function XOR, or non-carrying addition, as it used to be quaintly called?

You know, for years I didn't know the answer to that. At APL parties I'd blush and bashfully change the subject. Then some kind friend whispered "Have you tried not-equals?". And the moral to that is: it's all there on the keyboard — somewhere (if you can find it). The chaps who developed APL weren't born yesterday. Last half-century maybe...

As for finding things on the keyboard, my PowerBook has APL.68000 stickers on its keys. I do get some dirty looks during customer visits. It's almost as bad as sporting a baby's rattle with satanic symbols on it. Did you know that you can get a flexy transparent keyboard cover for most major PCs? It fits snugly over and between the keys and you simply type through it. It's meant to protect the keyboard from dust, but it's marvellous for sticking APL symbols on. When you leave the computer you peel it off and nobody knows your wicked secret. For a blank cover to stick your own symbols on (price £9.50) phone HCS Global Computer Supplies on 0800-252252, quoting stock-numbers 5801 onwards for IBM keyboards.

Practise Safe Keyboard-Entry, I say. Actually I-APL/Mac doesn't need one. There's a little on-screen palette you click for the symbol. Hold down {option} and it tells you what the symbol does, too. Using IAPL/Mac I haven't typed an APL symbol for years (other vendors please copy).

While on the subject of keyboard entry, isn't it amazing how vulnerable computers still are to mis-spelling, especially of surnames. I still bifurcate on mailing lists (very painful!). Years ago the Readers Digest was reputed to have one of the best weighted-matching systems around for reconciling duplicate names and addresses. Other good ones were owned by the Police National Computer and the Mormon Church. Most of these were adaptations of the Russell-Soundex system, invented around the turn of the century for USA census work, I believe. See also Sylvia Camacho's article, "*What's in a Name?*" (VECTOR 10.4, pp78-82). Now the Levenshtain Distance is a measure of how close two words are in spelling. It is defined as the minimum number of single-letter insertions, deletions or substitutions needed to turn one word into another. It would make a good fuzzy-match APL function to handle misspelt words, if you can program the algorithm efficiently (all the methods I've come across find the shortest diagonal route across a grid). Send solutions to me, or functions for slightly different metrics which would serve almost as well.

Ian Clark, alias Clarke.

J: A Second Lesson

by Donald B. McIntyre

A First Lesson (*Education Vector*, Vol.10 No.4, April 1994, pp.18-32) gave examples of J's parts of speech (*nouns, verbs, adverbs, and conjunctions*), verb trains (*hooks and forks*), and the important concepts of *items* and the *rank* of verbs and nouns. These were illustrated as they applied to the computation of a similarity table, which is the basis of cluster analysis.

In this Second Lesson we illustrate (a) some ways of entering tables of data, (b) selecting and re-ordering data, and (c) the use of inner products (*i.e.* matrix multiplication).

Consider first two small tables, *t1* and *t2*, of data on dairy products. The rows (*items*) are samples analysed for Energy (calories) and Calcium (mg).

t1	t2
165 285	165 285
330 570	330 259
30 52	30 15
50 86	50 12

t1 can be entered in any of the following ways, and *t2* similarly:

```
t1=. > 165 285; 330 570; 30 52; 50 86
x=. 165 330 30 50 [ y=. 285 570 52 86
x,:y          NB. Laminare
165 330 30 50
285 570 52 86
t1-: x,.y      NB. Append.  Matches t1
1
t1-: |: 2 4$x,y  NB. Transpose after reshape
1
```

The verb *\$* reshapes its right argument as specified by the left argument. The verb *|:* *transposes* a table; *i.e.* it interchanges rows and columns, turning the table on its side.

With so small a table as *t1* you might notice that a simple relationship is evident when you divide each item by the first item. Use "1 to specify that the division is to apply to the rank-1 cells of both arguments:

```

t1 %"1 (.t1      NB. {. is head. % is divide
  1          1
  2          2
0.181818 0.182456
0.30303 0.301754

```

Define a verb as either a hook or a fork to produce the same result:

```

h=. %"1 {.
f=. [ %"1 {.      NB. [ returns its left argument

```

The rows are all multiples of any one row. Noting that the rows are not independent, a mathematician would say that *t1* has a *rank* of 1. It is important to note that the word *rank* can have two meanings: it may mean the *span*, or number of linearly independent rows or columns; or it may mean (as in J) the number of items in the shape of a noun, the last *k* axes of an array determining its *rank-k cells* or *k-cells*. The context should make it clear which meaning is intended.

The rows of *t1* are not independent because the first row corresponds to 1 cup of milk and the other rows to varying quantities of the same milk. Given the composition of a cup of milk, the table can be completed by knowing the volumes of the other items. The table-builder (***/*)* is the mathematician's *outer product*:

```

cup=. 165 285
vol=. 1 2 0.182 0.303
vol */ cup
165      285
330      570
30.03    51.87
49.995   86.355

```

Table *t2* is very different, because the second row is clearly not a multiple of the first. The first row of *t2* again corresponds to a cup of milk. The second row represents something with as many calories as in 2 cups of milk but with only about half the calcium. We cannot obtain it merely by taking a different quantity of milk.

No row of *t2* is a simple multiple of any other; every row is a blend of two others. There is *more information* in *t2* than in *t1*. A mathematician would say that *t1* has a span of 1 and *t2* has a span of 2. Taking rows 0 and 2 (J uses zero origin) as end-members we can use the *matrix divide* (*%.)* verb to solve two sets of linear equations and find the proportions required to blend rows 0 and 2 to produce rows 1 and 3. For example, to get row 2 we blend rows 0 and 2 in the proportions 0.4642 8.447, found by solving the equations:

$$\begin{aligned} 165p + 30q &= 330 \\ 285p + 15q &= 259 \end{aligned}$$

$$\begin{aligned} m &= .0 \ 2 \{t2 \\ y &= .1 \ 3 \{t2 \end{aligned}$$

The verb *from* $\{$ takes items from the right argument as specified by the left argument. Because both m and y must be transposed for use in the equations, and the result must be transposed to match $t2$, the verb we use is

matrix divide under transpose

$\% . \quad \& . \quad | :$

This is analogous to doing multiplication by the addition of logarithms; first take logarithms of both arguments; then add the logarithms; and finally take the antilog.

plus under log

$$\begin{aligned} 3 + 8. \wedge . \ 4 \\ 12 \end{aligned}$$

$$\begin{aligned} Jw &= .y \ \% . \& . | : m \\ 0.464198 \quad & 8.44691 \\ _0.0641975 \quad & 2.01975 \end{aligned}$$

We can show that these results are correct by an *inner product* (matrix multiplication); i.e. multiply the rows of w by the columns of m and sum the products.

		cal Ca				
M	L	m	165	285	Milk	
			30	15	Light cream	
		y	330	259	Half-and-half	
			50	12	Heavy cream	
		w				
		0.464198	8.44691			
		_0.0641975	2.01975			

In J we use the *dot product* conjunction ($\cdot$) to define the *inner product*. However, because $/\cdot$ would be a J digraph, a space must separate the two symbols:

```

ip=. +/ .*
w ip m
330 259
30 15

```

The two verbs `+/` and `*` are composed by the dot-product conjunction to give the new verb `ip`. The syntax (`f=. g .h`) allows us to extend the pattern of the mathematician's *matrix multiplication* by replacing `g` and `h` by verbs (either primitive or defined) other than `+/` and `*`.

`w` gives the weights required for blending the items of `m` to produce the items of `y`. The items of `m` are the *end-members* (Milk and Light Cream), which when blended in the proportions (`.w` give what in California corresponds to the legal definition of Half-and-half. Because the minimum percentages of milkfat allowed are 3.5 for whole milk, 11.7 for half-and-half, and 20.0 for light cream, we can legally make half-and-half by blending equal quantities of milk and light cream, each of minimum standard. In `t2`, however, the units of volume are 1 tablespoon for cream but 1 cup for milk and half-and-half; so that, there being 16 tablespoons in a cup, we expect the proportions to be 0.5 cup of milk and 8 tablespoons of light cream. The proportions computed differ slightly from these expectations because energy and calcium are not perfectly correlated with milkfat, and the values in `t2` may correspond to milk products above the minimum quality prescribed by law.

In *J: A First Lesson* (*The Education Vector*, Vol.10, No. 4, April 1994, pp.17-29) we saw how to compute a similarity table by determining the distances between each of six points in 4-dimensional space. We did this in order to introduce some notions, such as *rank* of nouns and verbs, with wide applicability in programming. Having now introduced the more specialised pattern of the *inner product*, we have an alternative way of computing a similarity table.

First define a verb that computes the squares of differences between *items* (rows of a table are its *items*). The conjunction *atop* (`@`) composes this verb, `sqd`, from the verbs *square* (`*`) and *minus* (`-`):

```
sqd=. *: @ -
```

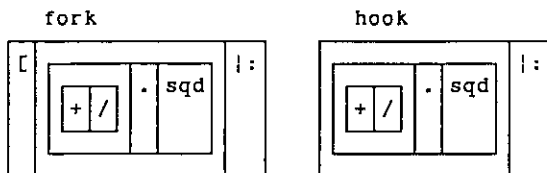
Then use `sqd` to form a new kind of inner product, `ssd`, that follows the pattern of matrix multiplication (rows of the left argument combine with columns of the right argument) to give the sums of squares of the differences:

```
ssd=. +/ .sqd
```

The right argument of the verb `ssd` will be the transpose of the left argument. Now complete a train of either three verbs (a *fork*) or two verbs (a *hook*):

```
fork=. [ ssd |:
hook=. ssd |:
```

These are equivalent to the following:



The table (*J: A First Lesson*) defining six points in 4-dimensional space (Herbert Hellerman's data) is:

```
Jhh=. >1 4 3 1;2 1 3 5;1 1 4 1;3 8 2 4;2 3 3 2;0 2 4 2
1 4 3 1
2 1 3 5
1 1 4 1
3 8 2 4
2 3 3 2
0 2 4 2
```

The squares of the distances are then computed by either the hook or the fork:

```
hook hh
0 26 10 30 3 7
26 0 18 52 13 15
10 18 0 66 7 3
30 52 66 0 31 53
3 13 7 31 0 6
7 15 3 53 6 0

(fork -: hook) hh
1
```

Returning to the milk example, consider some ways in which we might rearrange and otherwise modify the contents of a table. Begin by appending the end-member items to the computed blends:

```

]n=. m,w ip m
165 285
30 15
330 259
50 12

```

Recover t2 by permuting the items of n as defined by a permutation vector; *i.e.* interchange rows 1 and 2:

```

p=. 0 2 1 3
p{n
165 285
330 259
30 15
50 12

```

The *standard cycle representation* of the permutation is distinguished by *boxed* data, which in this case show that items 2 and 1 are interchanged while items 0 and 3 remain unchanged:

```

C. p


|   |   |   |   |
|---|---|---|---|
| 0 | 2 | 1 | 3 |
|---|---|---|---|

(p{n) -: (<2 1) C. n
1

```

Consider a different permutation:

```

p=. 0 2 3 1
p { n
165 285
330 259
50 12
30 15
C. p


|   |   |   |   |
|---|---|---|---|
| 0 | 3 | 1 | 2 |
|---|---|---|---|


```

In this case item 0 remains in position 0; while items 3 1 2 cycle so that item 1 goes to position 3, item 2 goes to position 1, and item 3 goes to position 2.

```

(p{n) -: (<3 1 2) C. n

```

To understand standard cycle representation (C.) consult the *J Dictionary (Version 7, pp.93-94)* and experiment with the following boxed table. In the definition of f

note that when two verbs, say *u* and *v*, are composed as *u&v*, *v* modifies both arguments before passing them to *u*, *i.e.*

x u&v y is (*v x*) *u* (*v y*)

<"0 boxes rank-0 cells; <"1 boxes rank-1 cells (rows of a table). Ravel items (.,.) gives the table formed by ravelling each item, and therefore makes a table out of a list.

y = . 6 3\$'abcdefghijklmnpqr'

f = . , "1&(<"1)

ix = . (.,.1.6) *f y*

0	abc
1	def
2	ghi
3	jkl
4	mno
5	pqr

p = . 4 5 2 1 0 3
p{x

4	mno
5	pqr
2	ghi
1	def
0	abc
3	jkl

ic = . C. *p*

2	4	0	5	3	1
---	---	---	---	---	---

Item 2 is unchanged; items 4 and 0 are interchanged; and items 5, 3 and 1 are cycled.

(*c C. x*) - : *p C. x*

1

To standardise the cycle representation, "each boxed cycle is rotated to begin with its largest element; and the boxed cycles are put in ascending order on their leading elements."

(<0 4) C. x (<1 5 3) C. x]z=. (1 5 3; 0 4) C.x

4	mno
1	def
2	ghi
3	jkl
0	abc
5	pqr

0	abc
5	pqr
2	ghi
1	def
4	mno
3	jkl

4	mno
5	pqr
2	ghi
1	def
0	abc
3	jkl

```
z-: (1 5 3; 0 4; 2) C.x
1
z-: (1 5 3; 2; 0 4) C.x
1
z-: (2; 4 0; 5 3 1) C.x
1
```

As we have already seen, the verb *from* { selects items (rows) from a table. This verb also allows scattered indexing:

```
]y=. 4 6$'abcdefghijklmnopqrstuvwx'
abcdef
ghijkl
mnopqr
stuvwx
(<2 3){y
P
(3 1; 1 2; 2 0; 0 4){y
time
```

One verb is sufficient for selection; the right argument specifies the table and the left argument the cells.

Tables larger than those used in the examples can be read from a DOS file created by another program or entered by a word-processor. The verb *readmat* makes input simple. The verb can be entered and used simply as a tool. I hope, however, that after working through these first two Lessons readers will be able to understand the definitions with the help of on-line experiments and the following comments.

First read the DOS file and remove null, new line (carriage-return), and end-of-file characters. Then *cut* at the line-feeds (removing the line-feeds in the process) and *execute*. The result is a numeric table:

```
read=. 1!:1@<
k=. 0 13 26{a.
```

```
g=. #~ [: -. e.&k
f=. "._2
readmat=. [: f [: g read
```

For 1!:1 see J Dictionary (Version 7, Appendix E).

- a. is the *atomic vector* of all 256 possible characters.
- e. is *membership*, symbolised in logic by the Greek letter epsilon.
- ;. is *cut*, which in this case applies " . to the delimited segments. The _2 shows that the delimiter is the last item and that the delimiters are excluded from the result.
- ". *Do*; i.e. execute the character object.

So that the names `f g k` can be used again without affecting the verb `readmat`, we fix the definition and erase the named objects. `f .` is an adverb

```
readmat=. readmat f.
erase=. 4!:55@;:
erase 'f g k'
```

The argument for `readmat` is the DOS file identification, including the path if necessary, given within single quotation marks; e.g.

```
table=. readmat 'c:\mydir\mydata.dat'
```

Footnotes

The span, in the sense of the number of linearly independent rows or columns, cannot exceed the number of rows or the number of columns, whichever is smaller. Moreover the span of a matrix product cannot exceed the span of the tables that produced it. The matrix product of a table with its own transpose is always a square symmetric table (i.e. a table identical to its transpose), and therefore Jacobi's method can be used to find its eigenvalues. The span is then the number of non-zero eigenvalues. For further information see my: *Jacobi's Method for Eigenvalues: An illustration of J. Vector Vol.9, No.3, January 1993, pp.125-133*. Also D.C. Murdoch *Linear Algebra for Undergraduates*, John Wiley, 1957, pp.42, 46, 70; B. Noble *Applied Linear Algebra*, Prentice-Hall, 1969, pp.89, 127.

The examples of blending milk and cream are taken from my *Introduction to the Study of Data Matrices*, in: *Models of Geologic Processes, An Introduction to Mathematical Geology*, American Geological Institute, 1969, in which the computations were done in APL.

APL Product Guide - Fully Revised

compiled by Gill Smith

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

Pressure on space occasionally prevents us from printing the complete guide, however updates will always be listed. We do depend on the alacrity of vendors to keep us informed about their products. Anyone who is not included in the Guide should contact me to get their free entry — see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete APL Systems (Hardware & Software)
- APL Interpreters
- APL-based Packages
- APL Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world. The listed addresses for Overseas Associations have all been contacted, but at time of going to press, not all have replied - some of them may be obsolete!

All contributions and updates to the APL Product Guide should be sent to Gill Smith, at Brook House, Gilling East, York, YO6 4JJ. Tel: 0439-788385, Email: 100331.644@Compuserve.com

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,658	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
Interprocess Systems	APL2 Dev't Workstation	poa	Mainframe APL2 supported on a PS/2 via a co-processor card with 16Mb of memory running VM/ESA (370 mode). A complete system includes a PS/2, a P/370 co-processor card, and software licenses for VM/ESA, APL2, GDDM and the full line of Interprocess APL2 enhancements.
MicroAPL	IBM RS/6000	12,000+	POWER range of RISC systems running AIX. Dumb terminal or graphical interface.
	Aurora	20,000+	Multi-user APL computer using 68020 CPU. Std. configuration 2Mb RAM, 16 RS232 ports, 68 Mb hard disc, 720K diskette
Optima	IBM Compatible	poa	Complete PC-based station, APL interpreters & all support eq't

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Software	APL*Plus/PC Release 10	450	STSC's APL for IBM PCs & compatibles. Upgrades from earlier releases also available.
	Run-time	poa	Closed version of APL*Plus/PC which prevents user exposure to APL.
	APL*Plus II	1,395	All the features of mainframe APL*Plus for your 386PC!
	Run-time Dyalog APL	poa 1000-10,000	2nd generation APL for Unix systems
	APL2/PC	poa	IBM's APL 2 for the PC.
Atlantis Software	Analytic Platform (K)	poa	K is an APL-like language
Cocking/Drury	APL*PLUS PC Rel 11	250	STSC's full featured APL for IBM's and compatibles - Version 11 gives free runtimes.
	APL*PLUS III Windows	949	The new 32-bit native Windows APL*PLUS. Develop in Windows, and distribute APL applications with no runtime charges. Reasonable migration charges from APL*PLUS/PC or APL*PLUS II.

	APL*PLUS II for DOS	750	Now that APL*PLUS III for Windows is available, the facility for creating Windows applications in PLUS II has been removed, and the price reduced.
	APL*PLUS II for UNIX	poa	STSC's 2nd generation APL for all major Sparc and Risc Unix workstations.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers under VMS.
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL.
Dyadic	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
IAC/Human Interfaces			
	I-APL/Mac	13	Macintosh version of I-APL
I-APL Ltd	I-APL/PC or clones	8 - 11	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	11	As above
	I-APL/Macintosh	13	As above
	Iverson Software Inc		I-APL is the UK agent for all ISI products, including APLWIN and JWIN for PC and many other machines.
I-APL/ISI	APLWIN/386	50	Windows APL including manual
I-APL/ISI	JWIN/386	16	Including <i>Dictionary of J</i> and <i>Introduction to J</i> Please note the packing charge of £3 per order.
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	APL2 PC (US Version)	\$630	Product No. 5799-PGG. PRPQ Number RJ0411. Order from 1-800-IBM-CALL
	APL2 PC (European Version)	£348	Product No. 5604-260. Part number 38F1753. From all IBM dealers, including MicroAPL.
	APL2 for OS/2 Entry Edition	\$185	Part No 89G1556.
	APL2 for OS/2 Advanced Edition	\$650	Part No 89G1697. Contains all facilities of the Entry Edition plus: DB2 interface; co-operative processing TCP/IP interface; tools for writing APIs; TIME facility
	APL2 for Sun Solaris	\$1500	Product No. 5648-065.
	APL2 for AIX 6000	poa	Product No. 5765-012.
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390
	APL2 Application Env't Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
Insight Systems	APL*PLUS/PC	poa	APL systems marketed and supported ...
	Dyalog APL	poa	from: Dyadic, Manugistics, IBM
	APL2	poa	under: Windows, OS2 and Unix
Iverson Software Inc.	APL386	\$30	Sharp APL Release 20 for PC 386, 486 with graphics, and ability to operate under Windows.
	APLIPC	\$30	For PC under DOS
	APLIWIN	\$30	For 386/PC under Windows 3.1
	APL Reference Manual	\$30	Documentation for all the above.

	J System Kit	\$24	J 6.2 diskette with manual "J:Introduction and Dictionary"
	J Source Code	\$90	Full C source code plus 100-page book
MicroAPL	APL 68000 Level I	2000	First generation APL with numerous enhancements. Multi-user version (Unix, Mirage, MCS).
	APL 68000 Level II	2500	Second generation APL. Nested arrays, user defined operators, selective specification etc. Multi-user version (Unix, Mirage, MCS)
	APL 68000/X	1500-6000	Second-generation APL. Nested arrays, user defined operators, selective specification, etc. Multi-user AIX version with full OSF/Motif support.
	APL 68000 Level I Mac, ST, Amiga	87	First generation APL. Single user, full windowing interface, software floating point support.
	Mac, Amiga	260	First generation APL. Single user, full windowing interface, hardware floating point.
	APL 68000 Level II ST	170	Second generation APL. Full windowing interface, software floating point support.
	Amiga	260	Second generation APL. Full windowing interface. Hardware and software floating point support.
	Mac	520	Second generation APL. Full windowing Interface. Hardware and software floating point support.
Optima	APL*PLUS Rel 10	450	
	APL*PLUS II V 4.0	1395	
	APL*PLUS/PC	369	
	APL*PLUS II	950	
Soliton Associates	APL*PLUS II PC Developers Kit	poa	
	Dyalog APL	999	
Soliton Associates	SHARP APL for MVS	poa	for IBM MVS mainframes
	SHARP APL for Unix	poa	for IBM RS/6000 and Sun SPARC
Uniware	APL*PLUS/PC	495	STSC's full feature APL for IBM PC/XT/AT, Compaq, Olivetti.
	Run-Time	call	Closed version of APL*PLUS/PC which prevents user exposure to APL.
	APL*PLUS/UNIX	call	STSC's full feature APL for UNIX based computers
	APL*PLUS II	call	STSC's full feature APL for 386 machines.

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
APL-385	APL-385 FSM-385 DRAW-385 DB-385 GEN-385	50(PC), 125(mf)	including ... Screen development Screen design Relational W.S. Miscellaneous Utilities
The APL Group	Qualed	\$1500-4000	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
APL Software Ltd (mainframe)	RDS	poa	Relation Data Base System
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.

	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
Cocking/Drury (for VSAPL)	E'MENTS & SHAREFILE	poa	Component files, quad-functions & nested arrays for VSAPL under VM/CMS & MVS/TSO
	COMPILER	poa	The First APL compiler!
(for APL2)	SHAREFILE/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage
CODEWORK	HELM	poa	Decision Support system for top management. Developed in Italy over 7 years. Requires APL mainframe or APL*PLUS/II. Optional modules: EIS, Excel Interface, DTP output via LATEX, output on map background.
CYBEX AB	APL Graf/PC	290	Presentation graphics for APL*PLUS/PC (CGI)
	APL Graf II/PC	390	Presentation graphics for APL*PLUS II/PC (CGI).
	Utility Functions APL2	1900	For APL mainframe; incl. a very fast search.
	Utility Functions II/PC	130	Same package for APL*PLUS II/PC.
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
HRH Systems	APL Utilities	poa	Software to transfer workspaces between APL*PLUS and Sharp, and between APL*PLUS and I-APL. Software to import IBM .ATF files to APL*PLUS.
	APL*PLUS Utilities		Public domain software, unlock locked fns, a user-friendly alternative to locking, fns of mathematical physics, menus, and others.
IAC/Human Interfaces IAC/Graf		15	Graph plotting for I-APL/Mac
	IAC/Vox	15	Spoken APL characters for I-APL/Mac
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson. Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
Impetus Ltd	Impetus	poa	Corporate Modelling and Reporting System.
INFOSTROY	APL*PLUS/Xbase Interface (II/386 Version 2)	\$198	Complete package written in C. Comparable with the data, index & memo files of FoxPro, dBASE, & Clipper. Multi-user support. No DBMS license required.
	(PC Version 2)	\$98	As above for APL*PLUS/PC.
	(DLL Version 1)	\$198	The same in a DLL form! Gives your Windows applications all advantages of DLLs.
Insight Systems	IUTILS/XP	20-95	Cross-platform utility library including simple OS calls (DIR, COPY, DEL, RENAME) and DATE functions. For APL*PLUS II, APL2 and Dyalog APL under Windows, OS/2 and Unix.
	ASI	95	APL Spreadsheet Interface. "Device-Independent" spreadsheet driver supporting Excel, 123 and Quattro-Pro for Dyalog APL/W
	WinCom	95	Asynchronous comms package for Dyalog APL/W
	S2D,22D,X2X	poa	Advanced APL syntax analysis and conversion packages from Sharp and APL2 to Dyalog, and between any two APLs
	SQAPL Client	poa	Interface from APL*PLUS II, APL2 and Dyalog (Windows, OS/2 or Unix) to most SQL databases over most networks.

	SQAPL Server	poa	Makes APL*PLUS II, APL2 or Dyalog APL (Unix) available as SequaLink servers. Can be called from SQAPL clients or other applications such as Excel, C++, Smalltalk, Visual Basic.
Interprocess Systems	APL2 Development Workstation	poa	
	IEDIT	\$3000-5000	Full screen APL2 editor with immediate APL execution, and full-screen debugger
(mainframe)	AFM	\$6500-15300	High performance component and keyed file system (VS APL and APL2)
(PC)	AFM	\$175	Single user component and keyed files for APL2/PC.
	Enhanced Format	\$2575	A QuadFMT data formatter for VS APL and APL2
	PowerCode	\$2000	External functions for APL2
	CALL/AP	\$4700	For calling non-APL programs (VS APL and APL2)
	WSORG	poa	Full-screen Workspace Organizer for APL2.
JAD Software	JAD SMS	150-500	Software management system for APL*PLUS II based on hierarchical databases; includes full-screen interface and stand-alone functions. Price depends on number of users.
Lingo Allegro	FRESCO Business Graphics	200	Business Graphics DLL for Dyalog APL/W and APL*PLUS III
	GDDME	800	AP128 GDDM graphics emulation for Dyalog/W
	SQL	200	AP127 SQL Microsoft ODBC interface for Dyalog/W
Mercla	LOGOL 92	poa	Logistics management system for 386/486 & RISC computers. Sales Forecasting, Inventory Management, Master Scheduling, Distribution Requirements Planning, Sales & Operations Planning.
	TWIGS	poa	A modular library of tools to teach and explore state-of-the-art materials management concepts. Developed by R.G. Brown.
MicroAPL	MicroTASK	250	Product development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc
	MicroLINK	250	General device communications
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroPLOT/PC	250	For APL*PLUS/PC product
	MicroSPAN/PC	250	APL self instruction for APL*PLUS/PC
	STATGRAPHICS Rel 5	590	
Soliton Associates	LOGOS	poa	Application Development Environment
	MAILBOX	poa	Electronic Mail
	VIEWPOINT	poa	Report generator with interfaces to DB2 and MVS data
UNIWARE (for mainframe)	STSC's ENHANCEMENTS	poa	Quad-functions & nested arrays for IBM VSAPL
	STSC's SHAREFILE	poa	component files for IBM VSAPL and for IBM APL2
	TOOLS & UTILITIES	poa	Including FILEPRINT, FILESORT, FILECONVERT, FILEMANAGER(EMMA) STSC's database package
	EXECUCALC	poa	Mainframe spreadsheet compatible with VISICALC and part of LOTUS 1-2-3 under VSAPL/VM or TSO)
(for APL*PLUS/PC)	APL Debugger 2.1	FF1950 FF9750	A visual APL debugger to help develop applications (site license)
	Menus 3.0	FF2450 FF12250	Complete set of hierarchical menu utilities (site license)
	ETATGEN 2.0	FF1950 FF9750	Page layout report generator (site license)
	UNITAB 2.0	FF4550	An APL*PLUS spreadsheet-like data entry and validation system

		FF22750	(site license)
	UNI/ASM 3.0 (site license)	FF4950	Assembler utilities to speed up APL*PLUS/PC applications
	UNISTAT 5.1	FF2900	Data analysis add-on module for Statgraphics
(for APL*PLUS II)	UNIWARE Toolkit II 4.1	FF39000	(site license only). Relational database system and complete set of utilities for APL*PLUS II development
	APL Debugger II 2.1	FF2950 FF14750	A visual APL debugger to help develop applications (site license)
	Menus II 4.0	FF3950 FF19750	Complete set of hierarchical mouse-driven menu utilities (site license)
	ETATGEN II 2.0	FF2950 FF14750	Page layout report generator (site license)
	UNITAB II 2.0	FF6950 FF34750	An APL*PLUS spreadsheet-like data entry and validation system (site license)
	UNISTAT Plus 5.2	FF4300	Data analysis add-on module for Statgraphics
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9619067-07) including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace	APL Programming	poa	Short or long-term consultant available.
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
APL People	Consultancy	poa	Consultants available at all levels. Expertise in APL system design, project management, prototyping, financial applications, decision support systems, MIS, links to non-APL systems, documentation, etc.
Camacho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming; APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics: PC and mainframe
Cocking/Drury	Consultancy	175-275 275-350 300-450 400-600 450-750	Junior consultant Consultant Senior consultant Principal Consultant Managing consultant
David Crossley	Consultancy	poa	Broad experience in many APL environments
Peter Cyriax	Consultancy	100-150 120-200 160-300	Junior Consultant Consultant Senior Consultant
Dogon Research	Consultancy	poa	APL Systems consultancy, design, implementation, support, documentation and maintenance. All dialects with special emphasis on APL2 and Dyalog APL/W.
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.

E & S	Consultancy	poa	System prototyping: all types of information system, engineering software, graphics and decision support systems APL*PLUS/PC, APL2, Dyalog APL
General Software	Consultancy	from 120	
Greymantle Assoc Ltd	Consulting	poa	Company reporting, business graphics, Windows applications with Dyalog APL/W.
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Michael Hughes	Consultancy	poa	Consultant with 10+ years experience with various APL interpreters and C.
IAC/Human Interfaces	Consultancy	350	APL on Macintosh & PC. HCI design. VDU ergonomics: EC/Health & Safety compliance.
	Documentation	100-200	On-line assistance, product demos & mock-ups, manual writing; foreign language software localization.
	Training	poa	Using t-APL for courseware & distance learning materials; Mac programming in C, APL & HyperCard.
INFOSTROY	Consultancy	poa	APL*PLUS & Windows consultancy. Porting of software written in C into APL*PLUS.
Insight Systems	Consultancy	poa	Experts in APL conversions between any combination of: APL*PLUS, APL2, Dyalog APL and Sharp APL. We are also experienced right-sizers, comfortable with networks and relational databases (that also means when NOT to use SQL) and client/server development in APL, C and Visual Basic.
Intelligent Programs	Consultancy	poa	Systems development, enhancements, support.
	Documentation	poa	Preparation of new manuals, rewriting of existing materials.
	Training	poa	Training for APL experts through to non-technical system users.
JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
Kestrel	Consultancy	poa	All APLs, all environments. Design, analysis, coding, maintenance, documentation, training, interfacing.
Lingo Allegro USA	Consultancy	poa	General APL consultancy specializing in Prototyping, Migration, Mainframe to PC Downsizing, Performance Analysis, Troubleshooting, and Graphics.
MicroAPL	Consultancy	poa	Technical & applications consultancy.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
Optima	Consultancy	poa	A range of consultants with 3-15 yrs APL PC and mf experience.
Parallax Systems Inc	Consultancy	\$750	Introductory APL, APL for End-user & Advanced Topics in APL
QB On-Line	Consultancy	350	Specialising in Banking, Financial & Planning Systems.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Rex Swain	Consultancy	poa	Independent consultant, 20 years experience. Custom software development & training. PC and/or mainframe.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.
Unware	Consultancy (Senior)	FF/day 5000	Consultancy from people with at least 8 years APL experience.
	Consultancy (Senior)	FF/day 7500	Advice and training in Windows programming with APL*PLUS II
	Training	FF10000	5-day class on Windows programming with PLUS II version 4.0
Wickliffe Computer	Consultancy	poa	System design, consultancy, programming and documentation. Especially project management and decision support systems

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL People	Employment Agency	poa	Employees placed at all levels.
HMW	Employment	poa	Contractors and permanent employees placed.
HRH Systems	APL lessons		On-screen interactive APL lessons for APL*PLUS, TryAPL2, Sharp and I-APL — in English or French.
	The BBS\APL:	\$24 p.a.	703-528-7517, 1200-14400b, N-8-1, 24 hours. APL educational material is downloadable free. An additional 30 megs of APL software for APL*PLUS, PLUS II, IBM, Sharp & I-APL is available to subscribers (cost is \$24/yr). Selection available on disk for \$15 post-paid. Free on-disk catalogue.
I-APL Ltd	An APL Tutorial	3	45pp by Alvord & Thomson
	An Encyclopaedia of APL (2d Ed)	6	228pp by Helzer
	APL in Social Studies	3	36pp by Traberman
	I-APL Instruction Manual (2d Ed)	3	55pp by Camacho & Ziemann
	APL Programs for the Mathematics Classroom (Springer-Verlag)	16	185pp by Thomson
	J Dictionary	16	by Ken Iverson
	Programming in J	10	75pp by Ken Iverson
	Arithmetic	12	118pp by Ken Iverson
	An Introduction to J	8	47pp by Ken Iverson
	Tangible math	8	36pp by Ken Iverson
	Sharp APL Reference Manual	18	349pp by Berry
	APL Press Books	poa	A comprehensive selection of early APL literature
	<i>Please note there is a packing charge of £3 per order</i>		
Iverson Software Inc.	Programming in J	\$15	76pp.
	Tangible Math	\$12	34pp.
	Arithmetic	\$18	123pp.
Kestrel	Employment	poa	Permanent and contract, home and abroad. From individual placement to supply of complete project teams.
	Software Library	poa	Low-cost software distribution service; call for details.
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.
Soliton Associates	MVSLINK	poa	Interface from Sharp APL (Unix & MVS) to non-APL data and software in the MVS environment.
	SSQL	poa	High-performance DB2 interface for Sharp APL (Unix and MVS).

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM/SIGAPL	International	Quote Quad		
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$15
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Club Germany	Germany	APL Journal	Semi-annual meetings	DM60
Ass. Francophone pour la promotion d'APL	France	Les Nouvelles d'APL		
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
Chicago SIG	Chicago			
Hartford Group	Hartford, Connecticut, USA			
CPC US APL SIG (Capital PCUG)	Washington, D.C.	Capital PC Monitor	Monthly meetings, occasional classes	free

Danish SIG	Denmark		
Dutch APL Assoc.	Holland	-	Mini-congress, APL ShareWare Initiative
FinnAPL	Helsinki, Finland	FinnAPL Newsletter Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Japan APL Assoc.	Tokyo, Japan		
Melbourne APLUG	Melbourne, Australia	Quarterly meetings	free
New York SIG	New York, USA		
Potomac SIG	Washington DC, USA	Free monthly meetings	
Rochester APL	Rochester, New York		
Rome/Italy SIG	Roma, Italy		
SE APL Users Grp	Atlanta, Georgia	SEAPL Newsletter	
SOCAL	Southern California	Seminars	\$15 (\$5 students)
SovAPL	Obninsk, Russia		
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars SEK 75
SWAPL	Texas, USA	SWAPL	\$18
Swiss APL (SAUG)	Bern	Part of City SI-Info	SF60 (SI) + SF20 (SAUG)
Sydney APLUG	Sydney, Australia	Epsilon	Monthly Meetings
Toronto SIG	Toronto, Canada	Gimme Arrays!	Monthly Meetings, APL skills database, J SIG, Toronto Toolkit \$25

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
ACM/SIGAPL	Donna Baglio	ACM, 1515 Broadway, New York, NY 10036 USA Tel: (212) 626-0606 Email: baglio@acm.org
Active Workspace Ltd	Ross D Ranson	Moulsham Mill Centre, Parkway, Chelmsford, Essex, CM2 7PX. Tel: (0245)-496647; Fax: 0245-496648.
Adaptable Systems	Lols & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 589 5578 Fax: +61 3 589 3220
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel: 31-3474-2337, Fax: 31-3474-2342
APL-385	Adrian Smith	Brook House, Gilling East, York. Tel: 0439-788385 Fax: 0439-788194 Email: 100331.644@compuserve.com
APL Bay Area Users Group APLBUG	Lewis H. Robinson (Sec)	1100 Gough St, Apt 14A, San Francisco, CA 94109, USA Tel: (415) 928-2058 Email: frgp21a@prodigy.com
APL Club Austria	Erich Gall	IBM Österreich, Obere Donaustrasse 95, A-1020 Wien, Austria
APL Club Germany	Dieter Lattermann	Rheinstrasse 23, D-69190 Walldorf, Germany. Tel: (+49) 6227-63469 Compuserve: 100332,1461
The APL Group Inc	Stuart Sawabini	644 Danbury Road, WILTON, CT 06897 USA. Tel: (203) 762-3933 Fax: (203) 762-2108
APL People / Software	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
Association Francophone pour la promotion d'APL	Dr. Gérard Langlet	SCM, C.E. Saclay, F-91191-Gif sur Yvette, France. Fax: (33) 1 69 08 79 63
Atlantis Software	Arthur Whitney	1105 Harker Avenue, Palo Alto, CA 94301 USA
BACUS	Joseph de Kerf	Rocinberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Camacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS. Tel: 0272-730036. email: acamacho@dx.compulink.co.uk Reutemet (Sharp): ACAM
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS Tel: 0252-874697
Paul Chapman		51B Lamba Conduit Street, London: WC1N 3NB. Tel: 071-404-5401. Compuserve: 100343,3210
Chicago SIG	Larry Mysz	836 Highland Drive, Chicago Heights, IL 60411 Compuserve:73040,3032
Cinerea AB	Rolf Kornemark	Skyttegatan 25, S-193 00 Sigtuna, Sweden.

Cocking & Drury Ltd.	Romilly Cocking	180 Tottenham Court Road, LONDON, W1P 9LE Tel: 071436 9481 Fax: 071-438 0524
CODEWORK	Mauro Guazzo	Corso Cairoli 32, 10123 Torino, Italy. Tel: +39 11 885168 Fax: +39 11 812 2652
CPC UG	Lynne Startz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850, USA. Tel: 301-762-8372.
David Crossley		187 Le Tour du Pont, Quartier Le Mourre, 84210 ST DIDIER, France Tel: 90-66-08-87
CYBEX AB	Lars Wentzel	Gruvgatan 35B, S-421 30 V. Frölunda, Sweden. Tel: (46) 31-45 37 40. Fax: (46) 31-45 24 23.
Peter Cylrax Systems	Peter Cylrax	22 Hereford Road, London W2 4AA. Tel: 071-229-5344
Danish User Group	Per Gjerlof	Email: gjerper@inet.uni-c.dk
Datatrade Ltd.	Ian Tomlin	1 & 2 Sterling Business Park, Salthouse Road, Brackmills, Northampton, NN4 0EX. Tel: 0504-760241
Dogon Research	Dick Bowman	2 Dean Gardens, London E17 3QP Tel: 081-520-6334 Email: bowman@apl.demon.co.uk
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein. Tel: +31 3474-2337
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL. Tel: 0256-811125 Fax: 0256-811130
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. Tel: 0689-824741
FinnAPL		Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland
General Software Ltd	M.E. Martin	22 Russell Road, Northolt, Middx, UB5 4QS. Tel: 081-864-9537
Greymantle Associates	George MacLeod	Bartrum House, Ravens Lane, Berkhamsted, Herts, HP24 2DY Tel: 0442-878065
Hartford CT Group	Bob Pomeroy	Mass Mutual Life, 1295 State St, Maildrop F465, Springfield, MA 01111 Tel: +1 (413) 788 8411x2838
H.M.W. Trading Systems	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, London EC4Y 0HA. Tel: 071-353-4212; Fax: 071-353-3325
HRH Systems	Dick Holt	3802 N Richmond St, Suite 271, Arlington, VA 22207 Tel: (703) 528 7624 Internet: dick.holt@acm.org
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics., LE16 8QL Tel: 0538-770998
IAC/Human Interfaces	Ian A. Clark	9 Hill End, Frostarley, Bishop Auckland, Co. Durham DL13 2SX Tel: 0388-527190. Email: clark.i@applink.apple.com CS: 100021,3073
I-APL Ltd	Anthony Camacho (for queries, order forms)	11 Auburn Road, Redland, Bristol BS6 6LS. Tel: 0272-760036 email: acamacho@clx.compulink.co.uk Reutemet (Sharp): ACAM
	J C Business Services (for pre-paid orders only)	56 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU
IBM APL Products	Nancy Wheeler	APL Products, IBM Santa Teresa, Dept M46/D12, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: (408) 463-APL2 (=2752) Fax: (408) 463-4488 Email: APL2@vnet.ibm.com Cserve: GO IBMAPL2
Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD Tel: 0732-885126
INFOTROY	Alexei Miroshnikov	3 S. Tulenln Lane, St. Petersburg 191186 Russia. Tel: +7 812-3111611 Fax: +7 812-3153321 Email: aim@infostroy.spb.su
Insight Systems ApS	Morten Kromberg	Nordre Strandvej 119A, DK-3150 Hellebæk, Denmark Tel: (45) 42 10 70 22 Fax: (45) 42 10 75 74 Email: info@insight.dk
Interprocess Systems Inc.	Stella Chamberlain	11650 Alpharetta Highway, Suite 455, Roswell, Georgia 30076, USA Tel: (404) 410-1700. Fax: (404) 410-1773 Cserve: 70373,2676
Intelligent Programs Ltd	Mike Bucknall	9 Gun Wharf, 130 Wapping High St, London E1 9NH. Tel: 071-265-1120
Iverson Software Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel:(416) 925-6096 Fax: (416) 488-7559
JAD Software	David Crossley	580 Eyer Drive, #81 Pickering, Ontario, Canada L1W 3B7 Tel: (905) 837-1895 Fax:(905) 831-5172
Japan APL Association		23-2-302 Hiromichi, Adachi-ku, Tokyo 120, Japan
Kestrel Consulting	Mark Harris	Business & Technology Centre, Bessemer Drive, Stevenage, Herts SG1 2DX Tel:0438-310155 Fax:0438-310131
Lingo Allegro USA Inc.	Walter G. Fil	6749 South Westedge Ave., Suite K268, Portage MI 49002 USA Tel: (312) 203-4926 Fax: (708) 459-8501 Cserve: 71303,3224

Melbourne APL Group	Harvey Davies	CSIRO Div Atm Res, Private Bag No.1, Mordialloc, Victoria 3195, Australia Tel: +61 3 586 7574 Fax: +61 3 586 7600 Email: hld@dar.csiro.au
Mercia Software Ltd.	Gareth Brentnall	Holt Court North, Heneage Street West, Aston Science Park, Birmingham B7 4AX. Tel: 021-359-5096. Fax: 021-359-0375
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN Tel: 071-922 8866 Fax: 071-928 1006
Ellis Morgan		Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants. Tel: 0730-263843
New York SIG APL	Nestor Nelson	PO Box 138, NY 10185-0002, USA
Optima Systems Ltd	Paul Grosvenor	Airport House, Purley Way, Croydon, Surrey CR0 0XY Tel: 081 781-1812 Fax: 081 781-1999
Potomac APL SIG	John Martin	Computer Sciences Corp, 1100 West St, Laurel, Maryland 20707-3587 Tel: +1 (301) 497-2698 Fax: +1 (301) 498-8260 Email:jam@aom.org
QB On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Rd., Camberley, Surrey, GU15 1LB. Tel: 0276-855880 Fax: 0276-855301
Renaissance Data Systems	Ed Shaw	P.O. Box 20023, Park West Finance Station, New York, NY 10025-1510, U.S.A. Tel: (212)-864-3078
The Rochester Group Inc.	Robert Pullman	50 S. Union St., Rochester NY 14607-1828, U.S.A. Tel: 716-454-4360. Fax: 716-454-5430
Rochester APL	Gary Dennis	Sollton Associates, 1100 University Avenue, Rochester, NY 14607 Email: gsd@ipsalab.tor.sollton.com
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00100-Roma Trullo, Italy Email: marsac@vnet.ibm.com
SE APL Users Group	John Mangas	991 Creekdale Drive, Clarkston, GA30021 USA
Shandell Systems Ltd.	Maurice Shanahan	Chiltern House, High Street, Chalfont St. Giles, Bucks., HP8 4QH.
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 Tel: +1 (818) 222-2759 Fax: (818) 222-9250
Sollton Associates	Laurie Howard	Sollton Associates Ltd, Groot Blankenberg 53, 1082 AC Amsterdam, Netherlands Tel: +31 20 648 4475 Fax: +31 20 644 1206 Email:ljh@sollton.com
SovAPL	Alexander Skomorokhov	PO Box 5061, Obninsk-5, Kaluga Region, Russia Email:askom@apl2.obninsk.su
Rex Swain		8 South Street, Washington, CT 06793, U.S.A. Tel: 203-868-0131 Fax: (203) 868 9970
SWAPL	Stuart Yarus	PO Box 210367, Bedford, Texas 76095, USA Tel: (817) 577 0165 Compuserve: 73700.2545
SwedAPL	Gian Medri	Box 16181, S-103 24 Stockholm, Sweden Tel:+46 (8) 96 09 47
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: si@ifi.unizh.ch
Sydney APLUG	Rob Hodgkinson	PO Box 1511, Macquarie Centre, NSW 2113, Australia Tel:+61 2 257 5313
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: (818) 222-2759 Fax: (818) 222-9250
Toronto SIG	Marc Griffiths	PO Box 384, Adelaide St Post Office, Toronto Ontario M5C 2J5, Canada Tel: (416) 532-0843 Email: marc@utcc.utoronto.ca Cserve: 76260,3314
Unware	Eric Lescasse	Tour Neptune, Cedex 20, 92086 Paris la Defense, France, Tel: +33 1 47-78-78-00. Fax: +33 1 40-90-04-11
Wickliffe Computer Ltd	Nick Teller	76 Victoria Rd., Whitehaven, Cumbria, CA28 6JD. Tel: 0946-692588
Warwick University	Prof. Jeff Harrison	Dept of Statistics, University of Warwick, Coventry, CV4 7AL Tel: 0203-523369
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: (203) 872-7806

Jim Weigang's APL Notes

reviewed by Walter Spunde

The learning curve for APL, they say, is very steep: it is hard to get going, to feel competent using APL, and it seems to be true. Students doing the University of South Queensland mathematics course built on the use of APL (admittedly a course that throws them in at the deep end with $\circ$, $\star$ and $\times$ in the first week) all say the same thing: it was hard at the beginning. For instructors who want to use APL as a tool for teaching something else, there is a need for a book that provides not too much, but enough, information about APL — an easy reference, to satisfy the curious, to stimulate the adventurous, but one that won't daunt the slow or divert the insecure. Jim Weigang's *APL Notes* seems to fit the bill, but not without some reservations.

Its 164 pages of uncluttered A5 in a three-ring binder (after the style of early APL*PLUS reference manuals) are split into seven chapters dealing with: APL expressions and mathematical functions; building arrays; indexing and reduction; programs and workspaces; comparisons and selections; branching, looping and debugging; and finally, the remaining APL primitives and operations with files. There are a further 35 pages of appendices and a useful index which includes page references to APL primitives dealt with in the text. Like Dalois' *Introduction to APL*PLUS/PC* the book comes with a copy of Manugistics' student version APL*PLUS/SE, where $\square WA$ in a clear workspace is 180K. Alan Mayer's review of this book in Vector 10.2 pages 19-20, describes this system.

Weigang's style is conversational, enthusiastic and easy to read. His care with edge conditions in writing functions is meticulous. There is a nice explanation of why $M[; 1]$ and $M[; , 1]$ give different answers and the implications of writing $M[; -1 \uparrow \rho M]$ or $M[; (\rho M)[2]]$. The discussion of line-labels and local variables explains simply what will be happening in a function that calls other functions. His explanation of $\square CT$ makes clear the difference between $a=b$ and $0=a-b$. Equally pleasing is his discussion of identity elements for various operations illustrated by $+/\iota 0$ and $\times/\iota 0$, with the corollaries that $\times/\rho A$ gives the correct number of elements for any array and that $\lfloor/\iota 0$ returns the largest number the computer can represent. Unfortunately, he describes this number as infinity, having promised, in the introduction to the chapter, to display infinity. The reviewer did hope he would not say it three times [*Ed: What I tell you three times is true*] but, on page 147, he did. It will not endear him to mathematics teachers, who may find the usage unforgivable. After all, calculus and indeed the

real number system is based on a concept of infinity and infinite processes that cannot accommodate the constraints of a computer.

Mathematics instructors may also find his treatment of functions disappointing. He prefers to use the term **program** and introduces his first program with a niladic example using global variables, and then progressively improves the program. This is probably the correct approach for those who come to the study with the concept of a program as something which runs when invoked, but it avoids the emphasis on input→output that is so helpful in reinforcing the mathematical concept of a **function**.

It is not clear whether Weigang sees his reader as someone from a business or from a scientific background. Certainly he has included functions for solving quadratic equations and calculating integrals by Simpson's Rule.

A discussion of `□WIN` and associated system functions is one of those areas that must be passed over in a mathematics course but that students, once introduced to it, seek further information on, and it would surely give a lot of satisfaction to those from a business background. Unfortunately, this information is not there. A good case may well be made for teaching only language features and leaving system features alone, but when the book comes with a disc that places so many powerful facilities easily at the reader's fingertips, it seems a pity to pass over them without mention. Coupled to this, one may find advice on how to cope with older APL systems, the `∇` editor, `□` and `□`, and some of the supplied functions obviously written for these older systems, somewhat irrelevant and perhaps a little confusing for the student. With the APL*PLUS full screen Editor at hand, the discussion of system functions for native files will only be useful to the student when (s)he needs to manipulate file data from within a function. It seems an unlikely task for a beginner.

For those who use the standard keyboard, there are ample references to the location of particular symbols (which of course will be annoying to those instructors who use the APL*PLUS facility to re-define the keyboard so that, e.g., `<` appears on the `<` key, etc.).

The text has very few typographic errors, *leaning APL* being the exception. The index inexplicably omits the function `?` which nevertheless is used as roll on page 143 and mentioned in conjunction with `deal` (which is not explained) on page 155. One might have wished for a more detailed discussion of matrix multiplication (important both for business and mathematics students) and more of the sort of examples that make Helzer's Encyclopedia so rewarding to peruse. However, the language basics are there, and, if one wants a short and quick guide, one has no right to ask for too much. It is certainly a useful addition to the

literature and, despite its failure to exploit and describe the APL*PLUS system as fully as it might, it has been well enough received at the University of South Queensland to have it set as a text, together with study notes, for a diploma course in APL.

Comments on Review of APL Notes

by Jim Weigang

Editor's note: I sent Walter Spunde's review to Jim Weigang for comment and to ensure it contained no misunderstandings. Below is an edited version of jim's reply:

Jim feels that his use of infinity to mean the largest representable number would be quite understood in computer circles, though strict mathematicians find it incorrect. Jim aimed to make the book relatively neutral between the scientific or business audience. When someone learns APL, the book should be combined with problems from the reader's speciality. I am not generally qualified to provide these, and I mostly want to step out of the way and avoid supplying inappropriate examples. He continues:

"Walter is right about it being a pity that most of the APL*PLUS quad functions are omitted from the text. This was done partly to keep the book simple, and partly because I don't expect that all readers will use the APL*PLUS interpreter included with the book. But the next edition should have examples of the most useful APL*PLUS quad functions and a brief summary of all of them.

A few minor corrections: Although the interpreter is the same as the one included with Dalois' *Introduction to APL*PLUS/PC*, most of the other programs, including the utility workspace, user commands, and full-screen debugger, are unique to *APL Notes*. The value of $\square WA$ in a clear workspace depends on your computer's configuration. I get 300,000 bytes free using DOS 6 with a few resident programs. The del editor is mentioned only in a single sentence which says it is obsolete. (A brief summary of del editor commands appears in the reference card.) The descriptions of how to enter APL symbols are for the Unified keyboard, not the standard APL layout as Walter states. Yes, the question mark symbol (?) was inadvertently omitted from the index, although entries are provided for roll and random numbers. Roll is used in an example on page 141, not 143, but it is introduced on page 28.

The book sells for \$29 postpaid in the U.S. and Canada, \$37 postpaid elsewhere. Discounts are available for resellers and quantity purchases. Payment should be made by postal money order or a check drawn on a U.S. bank. Order from the author, Jim Weigang, at 68 Plateau Circle, West Springfield, MA 01089, U.S.A. Inquiries may be sent via e-mail to jimw@math.umass.edu."



RENAISSANCE DATA SYSTEMS

Enlightenment Thru Information Processing

Specializing in Books and Software on APL, J
and other Curiosities of merit.

HAVE YOU GIVEN A COPY OF IAPL OR J TO A TEACHER OR STUDENT?

Name: _____ Date: _____

Street: _____

City: _____ State: _____ Postal Code: _____

Country: _____ Telephone: _____

For a copy of our most recent catalog,
please send a self-addressed, legal-
sized envelope (stamped if from U.S.)
or mail this form to:

Renaissance Data Systems
Park West Finance Station
P. O. Box 20023
New York, New York 10025-1510

Learn APL

APL Shareware/Demos and Related Publications

APL and Mathematics

APL AS A TOOL OF THOUGHT Proceedings

Other Important Sources of APL Information.

APL History

APL References and Techniques

Special Subjects in APL

APL Interpreters and Software

J=: A Powerful Dialect of APL

Other Software for Use with APL

To Russia with Love and for APL92

Other Curiosities of Merit

**I-APL: An international voluntary project of individuals and
companies seeking to share their love of APL.**

APL*PLUS/PC V.11

reviewed by Adam Weber

Brilliant! At last an APL*PLUS/PC interpreter with a utility (and licence) to produce (and sell) executable files from workspace files. But why the need to produce EXE files? Basically it allows anyone to produce an application which they can then sell at a reasonable price without the need to supply the interpreter to go with it.

Installation

Installation was NOT easy on a 386SX. I was upgrading from V 5 and V 11 simply wouldn't install. Thankfully the installation manual came with manual installation instructions which told me (though vaguely) what I had to do!!! On a 486DX, installation was perfectly straightforward.

V 11 can be used windowed in Windows 386 enhanced mode or full screen in any other mode. It comes supplied with icons and PIF files.

The Interpreter

The interpreter actually comes in 3 versions:

APLNOG — Standard APL with no graphics APL — Standard APL with $\square G$ style graphics. APLX — Extended APL with $\square G$ style graphics.

The extended system provides a more user friendly interface using a session manager. This allows easy interface with hardware such as keyboard, printer and display. It also allows you to edit functions and data in separate, simultaneous sessions. However, for the user friendliness, you lose memory for workspace since the extended system is bigger than the others.

All three interpreters can use VDI graphics (Virtual Device Interface). This is a graphics programming standard. V 11 comes with a wide variety of drivers to use VDI graphics. There is only one problem — using VDI means loading huge TSR drivers that gobble up precious workspace memory. If you have enough UMBs you can put these drivers there, else use the APLNOG (the smallest interpreter) and don't use large arrays!

Run-Time Applications

V 11 comes with a utility which converts a workspace into an EXE file. Before this can be done, you must assign `□LX` to a function in your workspace. You must make sure that your workspace does not go into immediate execution mode. This will result in your EXE file terminating to DOS. You can also assign `□SA` to 'OFF' to return to DOS, but simply returning to immediate execution mode will do the same.

The next step is to 'bind' your workspace to a run-time engine. Four run-time engines are provided, one for the extended system and three different sized ones for the standard system. The difference in size is because each run-time engine supports different modules. The largest standard engine supports all standard APL functions. Below is a table showing which engine supports which modules:

MODULE	LARGE	MEDIUM	SMALL
BIG SCREEN SUPPORT	Y		
CMD	Y	Y	Y
FULL COMMUNICATIONS	Y		
PART COMMUNICATIONS		Y	Y
□COPY, □PCOPY	Y		
DETACHED INPUT/OUTPUT	Y		
DETOKENIZATION	Y	Y	Y
DOMINO FUNCTION	Y	Y	
FIND FUNCTION	Y	Y	
□DM	Y	Y	Y
□FDUP	Y	Y	
GRAPHICS FONT	Y		
□FMT	Y	Y	Y
FULL SCREEN EDITOR	Y	Y	
□G STYLE GRAPHICS	Y		
HELP FACILITY	Y		
APL KEYBOARD	Y	Y	Y
MOUSE SUPPORT	Y	Y	
USE OF OVERSTRIKE	Y	Y	
□NA	Y		
□UCMD	Y	Y	
□WIN	Y	Y	Y
□SOUND	Y	Y	Y
PRIMITIVE TRIG FUNCTIONS	Y	Y	Y
VIRTUAL WORKSPACE FACILITY	Y	Y	Y

To make your workspace into an EXE file you use the following command from DOS in whichever directory the BINDAWS.COM file is found:

```
BINDAWS startup(.aws) engine(.exr) applicationname(.exe)
```

Extensions are optional, as are paths. Your EXE file can have the same name as your workspace. The binding takes a couple of seconds. Note that this is not a compiling utility. It simply attaches a special version of the interpreter permanently to your workspace and then calls it an EXE file.

Before you can run the EXE file, you must serialise it just as you serialised the interpreters.

The EXE file will now run just like your workspace did, and at the same speed. You can also specify APL parameters after the filename on the command line just like when starting APL. Certain operations are disabled such as system commands, `ⓘSAVE`, `ⓘPSAVE`. Function definition is disabled or very restricted.

Unfortunately, the EXE file will use almost the same amount of memory as the workspace. This is due to the inefficient use of modules in a run-time engine. For example, if your workspace functions only use `ⓘG` graphics and nothing else, you have to use the largest engine, the only one which supports `ⓘG` graphics. This means you waste about 70K of memory because most of the engine sits there doing nothing! The only way around this is to ask Manugistics for a custom run-time engine! The other alternative is to choose a run-time engine, then write your code to fit that particular engine, rather than first writing your code any old way, then fitting an engine which provides all the required modules. For example, if you chose the smallest engine, you would have to define your own *DOMINO* function, rather than use the symbol in your code.

Once you make your EXE file, you can sell it, providing that you include the Manugistics copyright. Even applications using GSS device drivers are covered by the APL*PLUS PC System licence.

Other Features in V 11

V 11, compared with V 5, contains many new features, mainly those described above. It also has a few more `ⓘ` functions, as well as enhanced versions of existing ones. V 11 supports virtual workspaces if you wish. It still has its limits though. Functions such as `GRADE UP` are still limited to 64K arrays.

Conclusion

The ability to make EXE files is, itself, worth the upgrade if you are still using APL*PLUS PC. As an interpreter, it is probably years behind the 386 and windows versions, but it is quite good for DOS. This will probably be the last APL*PLUS PC version out.

Some things improve with age...

but software isn't always one of them. As computer technology advances, developers struggle to fit obsolete features into the modern environment. Yesterday's benefits can become today's limitations - sometimes the best starting point is a clean slate!



Release 2 for Windows from Iverson Software combines an incredibly powerful programming language with the latest in Windows technology. J's clean design and lack of special requirements makes for a smooth fit with other Windows software, providing the most flexible environment ever for application development. Use VBX controls for graphics, spreadsheets and a host of custom controls, ODBC drivers for database access, DDE and OLE for general purpose inter-program communication. Why settle for less?

J is packaged in a sophisticated Windows integrated development environment, which features a built-in editor and session manager with customizable menus, support for multiple script and execution sessions, a simple yet effective interface to Windows, and access to virtually any database. Core language features include control structures and advanced support for locales and scripts.

The commercial license includes royalty-free runtime use. J Release 2 is also available for most other small systems, including DOS, Mac, and UNIX workstations, and in low-cost personal and educational editions.

Discover how J can streamline your development efforts. Call today for more information!

Now available through Strand Software and authorized dealers. For more information, contact Anne Faust at Strand Software Systems • 2 Adelaide St. West, Suite #203 • Toronto • Ontario • Canada M5H 1L6 • Tel (416) 369-1630 • Fax (416) 369-9590 • Email anne.faust@rose.com

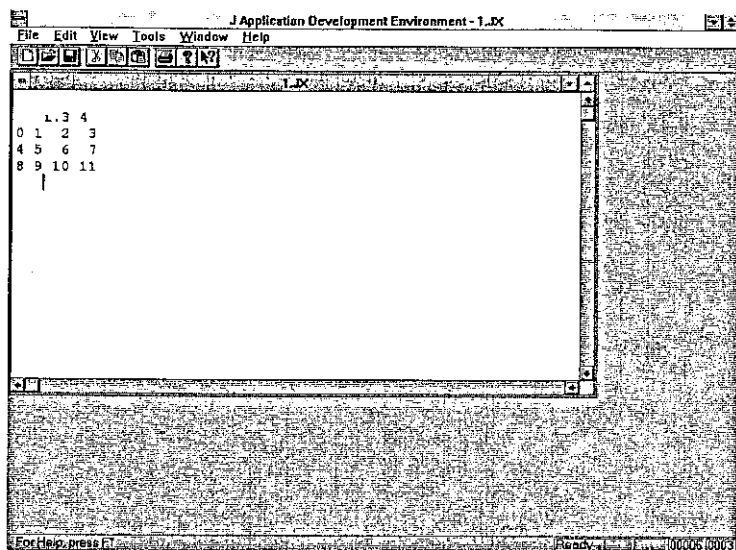
Windows is a trademark of Microsoft Corporation. The J logo is a trademark of Iverson Software Inc.

J Release 2 (Iverson Software Inc.)

reviewed by Dave Ziemann

By the time you read this, J Version 2 should be on the streets; I am reviewing an early beta-test copy of the product, which is at this time (mid-June) is still called J version 8.

For this reason, this review focuses on significant product features rather than on software deficiencies or apparent resource requirements, the characteristics of which are likely to change prior to release.



Screen Shot 1 — The J User Interface

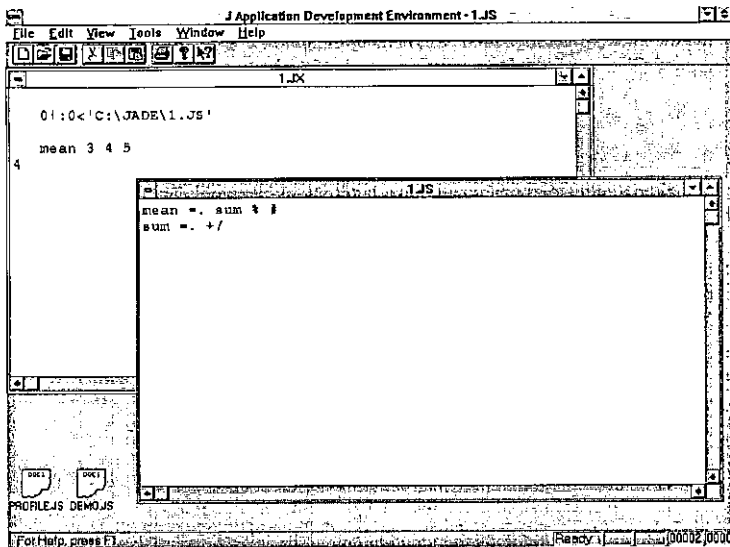
First Impressions

I had not looked at J since version 6.2 and was immediately surprised and impressed. J version 8 is a commercially oriented fully Windows-integrated product. The DOS version of J is included on the beta-test disk, but is clearly being de-emphasised. I don't know if it will be included in the product, and I didn't look at it. It looks as if ISI are concentrating their J development on Windows.

Installation is from Windows; you type 'A:SETUP' from the Run option in Program Manager — a reassuringly familiar activity. The standard Microsoft setup screen then takes over, and the files are decompressed and copied to the chosen target directory without fuss, and in only a few minutes.

The result of this is a new Program Manager Group which contains an icon for the J executable. Double-clicking this icon puts you straight into J8 — and what a difference!

What you see is an MDI parent window, which contains a single MDI child — a J execution window (with the label "1.JX"). Typing something into this window and then pressing the Enter key causes the entry to be executed as a J sentence, and the result displayed in the same window.



Screen Shot 2 — Using J Script Windows

The MDI User Interface

The abbreviation MDI stands for Multiple Document Interface, and is a standard interface protocol familiar to users of many Windows applications. Because this looked like an MDI application I hopefully assumed that it meant I could open multiple J script windows, all as children of the main window. I was not disappointed.

The MDI frame hosts a number of menus and some familiar icons. Clicking the first icon opens a new window with the label "1.JS". This is an empty window ready to receive J sentences that form part of a J script. Pressing the Enter key in a script window does not cause the sentence to be executed; we are building a script for deferred execution.

An active script window can be executed from a menu option or simply by pressing Ctrl-T. Each sentence in the script is transferred to the execution window and then executed, so that the script is "run". Alternatively, Ctrl-R will run the script without transferring each line to the execution window, which saves cluttering it up. Screen Shot 2 shows an open J script window and two minimised script windows.

Clicking the second icon presents a file-open dialogue, and an invitation to select a saved script file. Pressing OK then opens a script window on the selected script file. These features make defining, storing, retrieving, editing and executing J scripts a real joy. It just feels right.

The File menu also contains an option for opening a new J execution window. This means you can have more than one window in which J sentences can be entered and executed. You could use different J execution windows to reflect different lines of reasoning on the same problem — each execution window is a view on the same J session. The next three icons in the MDI frame let you cut, copy and paste, as one might expect. There are also icons for printing the contents of a window, for getting an "about" box, and for activating the Windows Help File associated with this product.

The File menu contains a rather splendid Word for Windows style Print Preview option, which can be used to preview the print form of a J execution or script window. In print preview mode three levels of magnification are supported, and you can page forwards and backwards through the preview. The menu also contains an option for setting the font to be used for the currently active child window. ISI supply their own TrueType font called ISIJ with the product, and the selectable sizes range from 8 to 72 point.

The nice ISI scaleable font and the funny icons in the MDI frame give me an idea. I wonder if it might not be possible to replace those one and two-character J mnemonics by special symbols whose graphical appearance reflects their function? Oh well, just a thought.

In the beta-test version the label in the main window was "J Application Development Environment", which ought to mean that the product should be called JADE, but I am told that it will appear as J Release 2.

Distributed Scripts

After playing a bit with the new interface I started to poke around the J scripts which were distributed with the beta release. There is already a lot of good material here, and it is clear that there is yet more to come.

The supplied PROFILE.JS script conditions the environment in a new and interesting way. Script files are apparently classified as belonging to one of three libraries, Private, Public or Project, in a manner which is reminiscent of certain mainframe system environments.

The z locale is populated with a host of more general utilities. To give just two examples, the verb `deb` deletes extraneous blanks from a list of characters, and the noun `LF` contains a line-feed character.

I found a lot of utility and demonstration scripts in the beta release, much of which was obviously still in development. There are demonstrations of how to program Windows controls such as list boxes, dialog boxes, font dialogues. There are a number of graphical demo scripts; a traffic-light simulator, a bouncing ball, a polygon painter, and a rather attractive implementation of a game called *enigma* which made excellent use of colour.

The Windows programming is performed via the Window Driver tool, which has been the subject of further development. This powerful windows scripting language is also available in ISI's APL product. Visual Basic controls are already supported as Window Driver child windows, and the final product will provide business graphics and a spreadsheet control.

The Windows Driver tool can also be used to dynamically add your own menu items to the main window. You can arrange for the selection of such menu items to result in the execution of an arbitrary J sentence. This is in fact how the active script window execution facility is supported; Ctrl-R just acts as the accelerator keystroke for its corresponding MDI menu option.

There are scripts for printing text via Windows Write, and for directly driving a Laserjet printer. I also found scripts for performing financial and actuarial computations.

Scripts providing ODBC access to a wide range of databases via the Q&E Database Library product are promised for the final product. This should provide access to local as well as network-based SQL databases.

A lot more is promised for the final release, and the distributed scripts will be documented in the *J Users Guide*.

Language Changes

As usual, there are a number of incompatible language changes of which J users need to be aware. The given names feature (abd:) has been decommitted. J workspaces are also decommitted. A number of changes have been made to the arguments of certain derivations of the foreign conjunction. In particular, the way that the script reader works has changed, so that it is now possible to read a script either from file or directly from a noun.

Most seriously perhaps, the name classes have been redefined as follows:

	was	is
noun	2	0
adverb	4	1
conjunction	5	2
verb	3	3

The vaguely numerological rationale here is that cases 0, 1 and 2 represent the valence of the name, and 3 represents the sum of the possible valences (monad + dyad = 3).

The Explicit Definition verb has been extended. For example, you can now enter a definition by opening the keyboard for input using `n : 0` as follows:

```
noun =. 0 : 0
how now
brown cow
)
noun
how now
brown cow
```

The `0 : 0` opens the keyboard for entry of a list of characters, and closes it when a singleton close parenthesis (followed by a return) is encountered. A line-feed character terminates each line. Varying the left argument controls the class of the object produced. A verb, for example can be directly defined as follows:

```
foo =. 3 : 0
2*y.
:
x.+y.
)
foo 3
6
2 foo 3
5
```

The singleton colon in the example above separates the definition of the monad from that of the dyad. The colon and the dyadic definition can be elided. Similarly, `1 : 0` and `2 : 0` can be used to define adverbs and conjunctions respectively.

The right argument can be text instead of the number 0. For example:

```
]v =. '2*y.',LF,':',LF,'x.+y.'
2*y.
:
x.+y.
(3 : v) 3
6
2 (3 : v) 3
5
```

The text can be a line-feed delimited string, a character matrix or a list of boxed strings. Adding 10 to the left argument will yield a tacit definition where possible, for example:

```
3 : '2*y.'
3 : 2*y.
13 : '2*y.'
2 : * ]
```

Locales

A good understanding of locales is now necessary in J, particularly as it looks like their use will pervade the system environment. If you think about what other APLs are doing with namespaces, then you will have a pretty good idea of what locales are about.

The name `mean_a_` is called a locative, and is said to refer to the name `mean` in the locale `a`. By contrast, an unqualified name such as `foo` is resolved by referencing the base locale.

For example, if the file 'stats.js' contains the following script:

```
mean =. sum % #
sum =. +/
```

then the following sentence defines a verb that will execute a script in locale a:

```
exec_a_ =. 0 1: 0      NB. execute script silently in locale a
```

and the following usage populates the locale a with the script contents:

```
exec_a_ <'stats.js'      N.B. so then ...
```

```
nl_a_ 3      NB. namelist of verbs in locale a
```

exec	mean	sum
------	------	-----

```
mean =. 34
sum =. 'hello'
mean_a_ 3 4 5
```

4

The name mean_a_ is resolved by executing the verb mean in locale a. The value of sum in that locale is the verb rather than the homonymous noun defined in the base locale. There is also a special locale, the z locale, from which names unresolved in other locales can be inherited. For example:

```
sum_z_ =. */
mean_a_ 3 4 5
```

4

but then

```
erase 'sum_a_'      NB. erase sum in locale a
```

1

```
nl_a_ 3
```

exec	mean
------	------

```
mean_a_ 3 4 5
```

20

Erasing sum in the locale a causes the value of sum (referenced in mean) to be inherited from the special z locale. The z locale can be populated with common utilities, which can be referenced anywhere, but do not clutter up the base locale. For example:

```
nl_z_ 3
```

erase	nl	sum
-------	----	-----

In addition, the *j* locale is typically populated with useful environmental utilities such as verbs that display the names of scripts in the libraries, or others that open windows on and permit the editing of named script files.

By the way, the name class of a locale is 6, so that:

```
n1 6
┌ ──┐
│ j  z │
└ ──┘
```

The leading empty box represents the anonymous base locale.

Try as I might, I could not succeed in nesting locales. Executing `a_b_c_ =. 42` only serves to create the noun `a_b` in the *c* locale. What I wanted of course, was to create a noun *a* in the *b* locale of the *c* locale. The most recent release of Dyalog APL will support nested namespaces.

Control Structures

Yes, it's true. ISI have put control structures into J. And yes, they are different from anyone else's. I feel strongly ambivalent about this; on one hand they pervert the functional simplicity of the language; on the other hand, if there ever was a language that could be made easier to understand with control structures, J is it. The real justification to my mind, is that they are required to make J acceptable in a commercial arena beyond the APL market place. (The same argument does not in my view apply to APL*PLUS III, but that's another story...)

J's control structures are provided via a set of dot-terminated control words, as follows:

```
break.
catch.
continue.
do.
else.
elseif.
end.
goto_<name>.
if.
label_<name>.
return.
try.
while.
whilst.
```

The control words `if.`, `try.`, `while.` and `whilst.` all mark the start of a block which is terminated by a corresponding `end.`. Usage of these can be summarised as follows:

```
if.  T do. B end.
```

```
if.  T do. B else. B1 end.
```

```
if.  T do. B elseif. T1 do. B1 elseif. Tn do. Bn end.
```

```
try. B catch. B1. end.
```

```
while. T do. B end.
```

```
whilst. T do. B end.
```

Words with `B` or `T` denote blocks of `J` sentences. The last sentence in the block determines the block to be executed next, depending on whether its leading atom is non-zero. The final result is the last sentence executed that was not in a `T` block.

The control words `while.` and `whilst.` differ only in that the latter skips the test the first time through, so that a `while.` structure may never execute its block, whereas a `whilst.` block is executed at least once.

The control word `break.` goes to the end of a `while.` or `whilst.` structure, whereas `continue.` goes to the top.

The `try.` and `catch.` structures permit recovery from an execution error. For example in the example in the paragraph above if block `B` executes successfully, then `B1` is not executed. If however the execution of block `B` fails, then block `B1` is executed.

The `goto_` and `label_` forms provide a family of control words which allow branching to arbitrarily labelled paragraphs. I think it is a shame that these have been included. They are at best unnecessary and at worst severely detrimental to code structure. Why should a highly functional language like `J` add them now, just as other language designers are recognising their disadvantages and are busy removing them?

The following verb demonstrates the use of control structures. It finds the root of a function *f* from a two-element list that brackets the root:

```

    root =. 3 : 0
mean =. +/ % #
while. ~:/y.
do. if. ~:/ *f b=. (mean,{.) y.
    do. y. =. b
    else. y. =. (mean,{:) y.
end.
end.
mean y.
)
f =. *: - +: NB. square minus double
root 0 100
2

```

The J Package

The product is priced at around US\$50 for personal use, and somewhere around US\$500 for a commercial licence. The documentation will apparently comprise three perfect-bound manuals, the *Introduction to J*, the *Dictionary of J*, and the new *J Users Guide*. The latter was not available at the time of this review. At the time of writing I did not have any information about commercial product support fees.

The Future

The speed with which ISI have moved J from a research tool to a viable commercial Windows product is impressive, and should put many an APL vendor to shame.

Pursuing a commercial goal for J will present ISI with two new difficulties. On the one hand they will have to enhance their product to keep up with the changes to the Windows environment; for example, it is hard to see how Windows 4 could support MDI in its current incarnation. On the other hand, they will have to resist the strong temptation to continue to make backward-incompatible changes to the J language. Commercial users do not take kindly to new software releases that cannot run their existing applications.

I wish ISI every success with their excellent product.

The APL*PLUS III GUI Implementation

reviewed by Duncan Pearson

Introduction

In the last issue of Vector and in other APL rags too numerous to mention Manugistics announced APL*PLUS/III as the most important APL release ever. Part of the reason for this monumental hubris may be the advent of control structures which are talked about elsewhere by Adrian Smith, but at least part of their pride will be in the new native Windows session and, most importantly, the APLGUI which has been changed from a monolithic set of APL functions (which reduced the available namespace by a 26th and the available workspace by considerably more) to a few system functions and variables working through an external DLL. The result is faster and cleaner, but they have also changed the interface in some ways that significantly improve its suitability for large system development.

What I intend to do over the next few pages is to examine whether the improvements in the APLGUI justify this claim and to compare it to its main rival, Dyalog APL/W v6.3. I will not go into how to code applications under +3 as this will doubtless be covered elsewhere. What you will get will be my impressions and my opinions but I will try to take you beneath the veneer of those smooth demonstrations that show you what will impress, and more importantly what will work.

I have spent most of the last eighteen months with Dyalog 6.3 developing, with Adrian Smith, the development environment CAUSEWAY [Smith, Vector 10.4, pp64-77]. I therefore have a pretty clear idea of what is important to me in a workable GUI interface. Since the arrival in the UK about a month ago of version 1.1 of +3, I have been working in my own time on the development of CAUSEWAY for +3 and have marked what has pleased me and what has annoyed me about it. What I have been doing with it is quite specific, in that it uses the APLGUI in a way that its developers never imagined. However my main concern is always to be able to produce fast, feature rich, reliable systems in the shortest amount of time possible and I imagine that the majority of you will share my aim.

The APLGUI

The APLGUI that comes with +3 is a development from that in +2 but it has overcome most of the faults that so crippled +2. Firstly it is fast. My impression is

that there is no noticeable difference in speed between it and Dyalog whereas the +2 APLGUI was at least ten times slower. The functions *Win* and *W_Call* which drove the +2 APLGUI and the variables *Wself*, *Warg* and *Wres* have been replaced by system functions and variables respectively. Whether you would agree with them that they have provided a clean upgrade path from the APLGUI in APL*PLUS/II is debatable. For the majority of UK readers this shouldn't be a problem though, as you could probably count the number of serious +2 APLGUI developers in the UK on the fingers of one hand.

The method of attaching callback code to events has changed and the means of identifying APLGUI objects has changed from system-generated numbers to developer-generated names. The most impressive change however is the way in which, with the interpreter and session being native Windows programs, the handling of any APLGUI objects created is concurrent with work in the session. This was ably demonstrated by James Wheeler in his demonstration when he had a ball bouncing in a form while he worked in the session changing the variable which determined the ball's size, and velocity. This may seem a bit of a gimmick but over the last couple of weeks it has made my life quite a lot easier as I have been able to get under the covers of my code as it is working.

The way in which a form interacts with others and in particular the form that was active when it was created is very flexible and gets over the problem that we have in Dyalog of having a choice of allowing user-input on only one form or on them all.

There has also been a new object, the *Timer* which generates special events at regular intervals. This is what kept the ball bouncing in the demo but it could also be of use in real-time applications and I have used it myself in CAUSEWAY to defer execution of some code.

Free Standing Menus

The Menu class appears only to provide menus that hang from menu bars. Although pop-up menus (menus that appear at the cursor position) were not initially fashionable they are nevertheless remarkably useful to have, and are becoming increasingly popular. In order to provide a usable form designer for CAUSEWAY I regard them as essential. In order that you don't spend your time waltzing from a new control to the property dialogue box, to the toolbar and back to the control I allow you to click the mouse button on the form and pop a menu asking what kind of object you want. You click on an object and up comes another menu asking how you want to change it. So useful are they that I have been forced to develop a routine to create, using `□wcall`, a pop-up menu and

return the user's choice from it. It worked but it involved placing a 'filter' between the APLGUI form on which I placed the menu, and Windows, to catch the WM_COMMAND message that Windows sent to report which menu item had been selected. This is like hooking DOS interrupts and cleaning out drains, messy and to be avoided at all costs.

Documentation

The documentation, while well organised, is in some places quite sparse and in many places just not rigorous enough. For example, in the documentation for the *Click* event we have:

Trigger:

When a mouse click occurs for the object. This event can occur either by user action or by invoking the *Click* method using *wi*.

It fails to mention that it can occur as the result of the user pressing the spacebar while a *Check* control has the focus. Nor does it mention here that it can occur as a result of the program setting the *value* of the check box (whether it is checked or not) — more about this later.

In contrast I found the documentation on the use of `□wcall` very useful. In fact had it been any less good I doubt that I would have been able to get my pop-up menu facility to work in the couple of hours it took. Filters and window classes are not easy to get across in a few paragraphs, but with a couple of well chosen examples they introduced it well.

Parentage

It is a shame that they have re-imposed the same restrictions as have Dyalog on what can be a child of what. When James Wheeler demonstrated the product in December he showed a bitmap as a child of a bitmap and a form as the child of a form. In release 1.1 '`q.□wi'New' 'Form'`' produces the rather bizarre '*PARENTAGE ERROR*'. I thought it very useful that a master form could have other forms as its children and when it was deleted all its children would also disappear. In CAUSEWAY we have a hierarchical form system with forms being conceptually children of other forms. We have to manage this with a table in the workspace (or under Dyalog, in the *data* of the root) and we have to be very careful that on deletion of forms the table is properly updated. It would have been much easier handled by the interpreter but obviously Manugistics found some very good reason why it was impossible.

Callbacks

The development of the callback methodology is interesting. This is about what to do when Windows informs APL that the user has done something on one of its controls, e.g. clicked on a button. The way that they chose to implement this in +2 was crippling. If the user clicked a button *BB* on a form *FF* then the system would execute the function *BB_FF_Click_handler*, no more and no less. This meant that to handle five events each on twenty controls on the thirty forms that might comprise a medium sized system you would need 3000 distinct functions, probably most of them the same.

In +3 each event valid for a control has an associated property that can be set to a text vector that will be executed on the occurrence of the event. This is very flexible, and an improvement on the methodology of Dyalog that only allows functions. In Dyalog you can use the same function for different events/control combinations, but many of these functions will be very simple and in +3 would not be needed at all. It is a pity that when looking at Dyalog the developers at Manugistics did not take note of the ability not only to associate a function with an event, but also an arbitrary APL array that would be passed as the left argument of the function. This is useful for exactly the same reason. If the callback code needs access to some data that is specific to that event on that object, then the workspace need not be cluttered with odd named variables to achieve that effect. It is possible to hold the data in the data property of the control, indexed by event name, and this works but is less clean.

As mentioned earlier there is an interesting feature of callbacks. When, under program control, I set the *value* of a *Check* box to change it, say, from being selected to not being selected, a *Click* event is generated on the box. I, the program, am being told that I have just changed the value. I can hear you say that if I had a more object-oriented mindset I would realise that it is the box that is being informed that it has just changed and might want to do something about it. So, what if it was a callback on the box that did the changing in the first place. Consider that I might have set, in some way, the box to 'watch' an APL variable. We do this all the time in CAUSEWAY. The variable has changed so I update the box. Then I get a *Click* on the box to say that it just changed, so I change the variable to match it. And then ... Actually there is a feature called *recursions* that will trap this kind of thing and (by default) if it happens more than 50 times it will stop. This used to be called STACK FULL but the neat thing is that you can, for debugging purposes, set the limit to be quite low, perhaps five or ten. This means that you catch the little bug(ger)s early.

Deferred Method or Callback Execution

When I execute a Method on a control or form using `□wi`, then any callbacks on that control that would be triggered as a result of the method, are executed during the call to `□wi` and before it returns to the part of the program that called it. A similar behaviour is possible in Dyalog using 1 `□nq object message`. This is not however necessarily desirable. If for instance, the code executed in the callback refers to a global variable, and there is a local of the same name in the function that executes the method, then the callback will use the value in the local rather than the global. In order to avoid name clashes of that sort a strict naming convention must be adhered to with perhaps different prefixes for locals, globals, labels and functions. This is fine until you start to use code provided by someone else who may not share your convention. This is bad for me who is writing a system that, as a result of callbacks, executes arbitrary code written by application developers who want to know nothing of the naming conventions in the 'system' code.

In Dyalog this problem does not arise. You `□nq object message`, the message is put on the Windows queue, and it is not processed until you have left the function to go back to the message loop (`□dq` in Dyalog). I have emulated this behaviour in +3 by having my own message queue and by using a callback on a Timer to fire a function to empty and process the queue. It is not ideal and some sort of deferred execution would be preferable.

Methods vs. Events

This problem illustrates a difference in approach between Dyalog and +3. In +3 if you want to do something to an object, for instance make it visible, you are encouraged to use a method, in this case `Show`. There is a parallel between methods properties and functions and variables. The methods are, as it were, an object's functions, and the properties are its variables. As indicated above the two are sometimes closely intertwined. The setting of the `value` property of a `Button` to -1 seems exactly equivalent to invoking its `Click` method. In Dyalog you may change an object property in the same way, but the only other way of changing the object is to queue a message to it that, had it come from Windows, would have had the desired effect.

A combination of the +3 approach and a hangover from the days of the +2 APLGUI provide a useful bit of functionality. First some background.

The main APLGUI function `□wi` takes as its left argument the name of an APLGUI object. If this is omitted then the value of the system variable `□wself` is used instead. In execution of a callback on an object `□wself` is automatically

set to the name of the object. This means that monadic calls of `⊡wi` act on the object.

The methods and properties of the forms and controls are part of the name hierarchy. Regardless of the left argument to `⊡wi` I can refer to a property of an object by prefixing to it the name of the object. There are certain shortcuts. If for instance a property, method or object name is preceded by a colon ':' then it is taken to be a property, method or object of the form. Thus if I am executing a callback on a button *B1* on a form and wish to know the *text* property of an edit, *E1*, on the same form I need only use `⊡wi ':E1.text'` not even knowing the name of the form.

Close and Destroy

In the methods *Close* and *Delete* and the events *Close* and *Destroy*, Manugistics have provided a good clean way to implement the behaviour, seen in so many multiple form applications, of allowing a child form to notice the intention to close the application and to take appropriate action, perhaps rejecting the close if, after consultation with the user, it is deemed inappropriate. This can be a minefield and is similar to the cascading delete in a relational database. In either case the wind-up requires two passes, one to check that everyone agrees and one to implement the deletion. The freeing of associated resource and the cleaning up of registration tables in the case of CAUSEWAY must be implemented from the *Destroy* event leaving the *Close* as a simple answer to a question.

Graphics

The main difference between graphics under +3 and under Dyalog is that under Dyalog the graphics are objects that can be moved, that report events and that retain their essence after being covered by other windows. Under +3 there is a *Draw* method. It is an operation to change temporarily the appearance of an object. There appear to be all the standard facilities there such as circles, lines, polygons and boxes. One thing that Windows doesn't provide in its Graphics Device Interface (GDI) and that likewise is not given here, is a Bezier curve. This is very simple and fast to implement in machine code or C and is a powerful drawing tool. It is what makes scalable TrueType fonts possible. Neither do they provide a simple way in APL to define bitmaps and icons. The manipulation and definition of bitmaps and icons, which are after all only matrices of numbers, is so suited to APL that it seems odd that Manugistics expect us to use inferior tools such as Paintbrush to produce them. With APL, for instance it is possible to take a black and white bitmap of some text and with a few rotations and comparisons produce a grey embossed version of the same.

Still I must emphasise that the most important feature that the +3 graphics are missing is persistence. If I were the type of person who could put up with having to write code to redraw a graph on a piece of window every time it was accidentally obscured, then I would probably put up with writing in C, or perhaps even COBOL.

scale property

The scale property provides a means of specifying a different internal co-ordinate system for an object than the default. On the whole it is well implemented and achieves its purpose well. If, for instance, you want to draw a graph in a frame and you use a PostScript-like co-ordinate system with the origin in the bottom left corner then set the *scale* property to 0 100 0 -100 100 (proportional, top, left, height & width). Then 0 0 will be the bottom left hand corner and 100 100 will be the top right. It has other units such as pixels, twips and points, however the default unit for position and size (unless set by scale) is the character unit. On a VGA screen this is 16 pixels deep by 8 across, on a SuperVGA with LargeFonts it will be larger.

There are two reasons that I believe this to be an unfortunate choice. Firstly, in a graphical environment there should be no need to be counting in different units on the two axes. In order to put a border of a uniform width around a control I should not have to make it so many character units deep and double that number wide. Secondly, if the average character size of the system is different in the target environment from that in the development environment then all the forms and their controls will change size to match the new character size. Typically this would mean that they would grow and take up more of the valuable screen real estate that the user just spent a couple of thousand pounds to get. While for simple dialogue boxes it is a good idea for the size to increase, there are many applications for which it is not, e.g. spreadsheets & word processors.

So, you don't like the default, then change it I hear you say. Two problems — it is impossible to change the default for the positioning of forms. The system object '# ' does not have a *scale* property (it is always characters) and so the position of forms must always be specified in character units. Furthermore if I change the *scale* of a form to pixels and create a frame in it, the position (the *where* property) of the frame is in pixels but the *'size'* is in character units as the frame did not inherit the scale of the parent but took instead the global default. This is in contrast to Dyalog where by default all objects inherit the *coord* of their parents, and where all you have to do is to set the *coord* property of ' . ', the root object, to pixels and everything subsequently created is in pixels.

DbClick

Here we have something that is symptomatic in some respects of Manugistics' attitude to its users, i.e. us. The *DbClick* event is only reported by the items on a List or a Combo control. These are the obvious things where you might think that a user should usefully double-click the mouse, both to select an item and to indicate acceptance of his selection in one fell swoop. However there are more situations where it might be useful. Take for example the Windows Control Panel. It appears to be a form in which are drawn a number of icons on which a double click causes a specific program to load. One might try this in +3. One might create the form and draw in it a number of icons using the Draw method. Then one would be stymied. The Form does not report a double click.

Why not? I can only assume that the paternal Manugistics designers did not think that we would want to know about it. They have made assumptions about the type of applications that we might develop in the same way that they have assumed that we might be shipping applications as complete runtime systems with resources such as bitmaps and icons bound into the interpreter. They are C developers and programmers and so that is how they think.

Missing

There are a few things that Dyalog provides that are useful and difficult to emulate. The first of these, the free standing menu has already been mentioned, another is the *LOCATOR*. This is a means by which we can use the mouse pointer to select an area of the screen by drawing out a rectangle whose size (and potentially position) are updated automatically by the system. This facility is used in *JWED* the screen editor but is not generally available.

Portability vs. Control

There are two definite advantages in +3 for the developer who is committed to Windows in one or more of its flavours and who therefore does not regard UNIX portability as necessary.

The facilities offered by the objects in the APLGUI are considerably richer than those offered in the corresponding controls under Dyalog. The *Menu* object can have associated an accelerator key that automatically selects the menu item whenever it is pressed (to achieve this behaviour in Dyalog it would be necessary to trap the keystroke event in all objects in which it were valid and compare the pressed key with a table stored by the developer on the form). The *Edit* object has a number of styles that make life so much easier for the

programmer. One in particular, the 'password' style is quite painful to emulate. The messages also have more detailed information. There are messages for key down and key up as well as for key press and they indicate, among other things, the current state of the Capslock key.

The second advantage is the way in which, if the developer needs to add functionality to a control, or to keep a closer eye on what is happening to it, he has access to the control's window handle, the unique identifier by which Windows identifies the control. This he can use in a call to a Windows DLL function which will then act upon the control. This is dangerous. It is getting in between the APLGUI and its objects, but in skilled hands it could be a powerful tool. Without it my pop-up menu would have involved a great deal more work than it did.

Undoubtedly one of the main reasons for the lack of most of the above functionality in Dyalog has been the insistence that it must be possible to implement the interface under UNIX. In Toronto Dyadic demonstrated some APL/W code running unchanged on a UNIX box under the development version of their Motif product. Because the +3 APLGUI is very much more closely bound in with Windows, this kind of portability would just not be possible.

Conclusions

If the overall impression I give is of dissatisfaction it is perhaps because I have gone into detail about the things with which I have had problems and glossed over some of the really useful features of the product. There is no doubt that it is a workmanlike product and immeasurably better than APL*PLUS/II. It provides a similar capability to Dyalog v6.3 and while it is richer in some areas it is poorer in others. It gives finer control over many objects but it has only primitive graphics. There are areas of design that I found frustrating, such as the odd callback behaviour, and areas which I found a blessed relief, such as the flexibility on Wait and the Close & Destroy events. On balance I would still choose Dyalog v6.3 because of the greater simplicity and predictability of the callback structure and because of the graphics. By now it may be an academic choice as Dyalog v7 is promised for July 4th. It would be impossible to assess it from the brochures, but namespaces could be equally as important a development as control structures and the extra features that they promise to pack into the GUI should knock APL*PLUS/III 1.1 into a cocked hat. The humble application developer can only benefit.

APL*PLUS III Control Structures

reviewed by Adrian Smith

Background

Remember that I started life as a Basic programmer, moved to PL/1, and then had my first encounter with APL back in November 1978. Since then I have learned C, SQL and PostScript, and may soon encounter ABAP/4 in earnest (yet another 4GL/query language). Of these, only SQL and APL have been structureless, as neither was intended for writing the sort of procedural code that PL/1 excels in. PostScript is unusual, in that it has no 'goto', so you have to work with the 'if-else' and 'repeat' constructs. In practice, the same is almost true of C, and I usually tried to write 'goto-less' PL/1 where I could.

As you can see from this brief life-history, I am a slightly biased observer, particularly as I have always tended towards the 'keep-it-simple-stupid' style of APL, which has meant avoiding deeply nested structures and using loops in place of fancy 3-D reshapes and diagonal transposes! With this in mind, please read sceptically, and hope for a balancing article in the next Vector from a less structured author.

Philosophical Stuff

If there is a wasp in the room, I like to know where it is! The same applies to loops in APL, which is why I have always used an old STSC 'whizzbang' technique to make them very obvious:

```
Loop:→LAB+(pdata) iterate End,ct+1
      process data[ct]
End:→LAB[ct+ct+1]
```

... of the various OR scientists who have used APL over the years, about half detest this idea (and stoutly refuse to use it), and the other half think it is quite sensible, and take to it immediately. I think the split will almost certainly be reflected in the APL world as a whole, but at least the APL*PLUS III approach has the merit of being totally ignorable (and takes a mere 14 pages of manual to explain), so the less-structured APLers amongst us cannot get too upset over it.

It is also interesting that the latest J release has introduced a very similar set of structures (see David Ziemann's review and Gene McDonnell's "*At Play with J*"),

and no-one seems too bothered that these undermine the clean simplicity of the notation. I think that we need to see the languages evolving at three levels:

- the fundamental notation is simple mathematics plus some structural stuff to do with constructing and reshaping arrays.
- control structures are an outer level, which select the particular bit of APL notation to execute under particular circumstance.
- namespaces/locales are an application management tool which only become relevant for large APL sites and complex systems, typically those being worked on by more than one programmer.

I think I would like to start a drive towards the global elimination of *each*, on the grounds that:

- (a) it looks incredibly ugly and makes code really hard to read
- (b) it kids you into thinking that you aren't looping
- (c) it confuses the nice classification shown above, by falling midway between levels 1 and 2.

This becomes a practical proposition only if the control structures are easy to work with, and execute efficiently.

I think I only have one serious worry — control structures may stop people bothering to think through the problem and result in simplistic iterated code rather than simple array code. However *each* is just as bad, if not worse, and I have seen enough awful APL even at the VS APL stage to realise that FORTRAN programmers will write bad APL anyway, so we may as well make them a little more comfortable whilst doing it.

When Would I use this Stuff?

There are some places in APL where you cannot escape a 'branch if' construction — one of the most obvious is defaulting a missing left argument to an ambivalent function:

```

      ∇ R←CHAR AMBI1 TXT
[1]  R Remove CHAR from TXT
[2]  ⚡(0=⊖NC 'CHAR')/'CHAR←' '
[3]  R←TXT~CHAR
      ∇

```

... which might generate a *STYLE ERROR* due to its gratuitous use of *Execute* and the doubled quotes, so maybe we could also code:

```

▽ R←CHAR AMB12 TXT
[1]  ⍝ Remove CHAR from TXT
[2]  →(2=⊞NC 'CHAR')⊕⊞LC+1 ⋄ CHAR←' '
[3]  R←TXT~CHAR
▽

```

... or even infest our code with some kind of *DOIF* construct. The alternative offered by APL*PLUS III is to write it like this:

```

▽ R←CHAR AMB13 TXT
[1]  ⍝ Remove CHAR from TXT
[2]  :IF 0=⊞NC 'CHAR'
[3]    CHAR←' '
[4]  :END
[5]  R←TXT~CHAR
▽

```

... or to pack it all on one line if you prefer ...

```

▽ R←CHAR AMB14 TXT
[1]  ⍝ Remove CHAR from TXT
[2]  :IF 0=⊞NC 'CHAR' ⋄ CHAR←' ' ⋄ :END
[3]  R←TXT~CHAR
▽

```

... but woe betide those who think they should get away with:

```

▽ R←CHAR AMB15 TXT
[1]  ⍝ Remove CHAR from TXT
[2]  :IF 0=⊞NC 'CHAR' ⋄ CHAR←' '
[3]  R←TXT~CHAR
▽

```

They will get no further than...

```

'f' AMB15 'fat cat'
OUTER SYNTAX ERROR, LINE 4, STATEMENT 1
'f' AMB15 'fat cat'
^

```

in other words, their function will not even start executing unless the control structures are all balanced up nicely. In a trivial example like this, no problem, but I wonder how APLers will take to this very 'compile and run' approach to finding syntax errors.

Does it make the code easier to read? In as trivial an example as this, I don't really think it does, but I was looking through some code yesterday which unscrambles EDI messages and does interesting things with the product codes and variant tags. The main workhorse function is a set of 6 almost identical blocks of code, one for each message type. It is begging for a 'Select ... Case' construction, and would benefit greatly from the indented code which this encourages.

```
:select msgtype
:case 1
    lots of quite
    horrid code
:case 2
    and so on

:else
    'Unknown message type'
:end
```

I really can't see any objection to this — it adds a degree of structural readability to the function as a whole, without in any way interfering with the actual APL code. The APL*PLUS III editor helps matters with its automatic indenting. It also seems possible to stop in the middle of such a structure and add lines of code without an *SI DAMAGE*, but I haven't had time to explore this fully.

The other obvious application of control structures is in replacing my old *iterate* code with something that reads as an explicit *each*:

```
Loop:→LAB+(pdata) iterate End,ct+1
    process data[ct]
End:→LAB[ct+ct+1]
```

... becomes

```
:for ct :in (pdata)
    process data[ct]
:end
```

... a very obvious simplification, and on almost any terms, a good thing.

Is it Fast Enough to be Useful?

I can't see that there are any timing issues around the Select-Case or While-Until constructions — the APL code inside the loop almost always heavily outweighs the trivial 'check and branch' outer syntax. What is more interesting is the speed

of iteration, as there are times when you want to process record by record, so the choice here is between:

- the new iterative structure
- nesting the data and using the *each* operator
- doing some serious thinking and avoiding the loop altogether.

I thought a good example would be a text search through a reasonably large address list, such as the delegate list from Toronto. I might want to find everyone with a CompuServe account, so that I can add them to my address book:

:whinge

how to get the file into the PlusIII workspace? I loaded it into my favourite text editor and copied it to the clipboard. Would the editor accept it — too big at 39396 bytes. OK, let's `▯NREAD` it and partition-enclose on the `▯TCNL`. `NONCE ERROR` it says — dive for the manual and use `▯PENCLOSE` instead. Now I have a nested text vector — double click it to edit and all I get is 'object not editable'. *This is NOT ON*. I have never had PLUSII on my machine, and this junk is going straight back to Anthony until it works with vectors of vectors. So there.

:endwhinge.

Now for those timings I set out to do 20 minutes ago ...

```
a←0
vvv←5000p'fat' 'cat' 'sat'
t+▯ai[2] ◊ process"vvv ◊ ▯ai[2]-t
0.77
t+▯ai[2] ◊ loopy vvv ◊ ▯ai[2]-t
1.49

loopy x
:for ct :in (1px)
  process x[ct]
:end

process x
a←a+1
```

If you were starting off with a text matrix, you could add around .1 sec to the time for `process` as you would need to split the data into vectors, and maybe a little more time to junk trailing blanks. Assuming that you wanted to do some significant processing within the loop, I would suggest you make an explicit loop and tolerate the small loss of performance.

:whinge

I just tried some more simple stuff with nested vectors and came upon:

```

      pvvv
5000  www+5000p'ere' 'we' 'go' 'again'
      t+ai[2] o p+vvv\www o ai[2]-t
195.04

```

... that time you see is in seconds — that's right: it took over 3 minutes on a fast DX2. What's more it totally locked the machine while it did it, so that I was on the point of giving up and hitting <Reset> and hence losing several of the above (unsaved) paragraphs in the process. Dyalog/W executes the above in .22 seconds, which is more reasonable.

:endwhinge.

An Idea Brought on by Reading Dave's Review

As you can tell, I feel really positive about both the idea and most of the implementation of control structures in APL*PLUS III. It seems to me that there is one thing Manugistics could usefully add — the J constructs of try. and catch. which are similar in concept to IBM's `□EA`. For example:

```

      v R+CHAR AMB16 TXT
[1]  a Remove CHAR from TXT
[2]  :TRY R+TXT-CHAR
[3]  :CATCH R+TXT~' '
[4]  :END
      v

```

... looks logical and clean. It would also make sense for the very commonly coded "create this file; if it's already there tie it and truncate it" logic which is very messy and a little dangerous to code with `□ELX`.

Conclusion

I stick by my view that Manugistics have done the right thing, and introduced control structures in a remarkably non-invasive way which allows the programmer to add structural information to functions without interfering with the basic notation at all. It executes fast enough to be a usable *each* replacement, and improves heavy *Select-Case* oriented functions enormously.

I worry about portability, and I would like to see some code to 'de-structure' a function into plain VS APL — there was a proposal at Toronto for a similar style of structuring which did offer this possibility, so perhaps some dedicated hack would like to have a go? This apart, I can see no problems with the basic idea, so the sooner everyone else copies it, the happier I will be.

Full Mainframe APL2 on a PS/2

Software:

- APL2/370 Version 2 Release 2
- VM/ESA Release 1.5 – 370 Feature
- GDDM/XA, GDDM/PGF, GDDM/OS2
- AFM 4.0, IEDIT, Enhanced Format, FSM124, PowerCode, WSORG
- OS/2 2.11, Communications Manager/2 1.1

Hardware:

- PS/2 (Micro Channel Architecture)
- Personal/370 (P/370) Co-processor Adapter/A card
- SCSI desktop 9-track or 3480 cartridge tape drives

The **APL2 Development Workstation** provides a complete APL2 production and development environment on a PS/2 – 100% compatible with mainframe APL2. One workstation supports up to 16 concurrent users via LAN or dial-in. DASD capacity is up to 32 Gbytes per workstation. Plus, the same workstation supports non-APL applications running under VM, OS/2, Windows, and DOS. From \$11,635 (US). Custom PS/2 configurations available.

All trademarks
acknowledged



Interprocess Systems

11660 Alpharetta Highway, Suite 455, Roswell, Georgia 30076-3880 USA
Tel +1 (404) 410-1700 Fax +1 (404) 410-1773 Email 70373.2676@compuserve.com

RECENT MEETINGS

This section of Vector documents all British APL Association meetings, and any other events of interest to the APL world. If you have recently attended any gathering which you feel would be interesting to Vector readers, please let the Editor have a brief note, and we will include it here.

In this issue, we document James Wheeler's demonstration of APL*PLUS III, and the three presentations at the May AGM: David Piper on a scheme for making APL into a fully object-aware language; John Scholes on the addition of 'namespaces' to release 7 of Dyalog APL/W, and finally Peter Donnelly who demonstrated as many of the new features of Dyalog APL as time allowed.

James Wheeler presents APL*PLUS III

reported by Anthony Camacho

The presentation was well backed with a series of messages projected onto a screen and was followed by a short demonstration of the product. In the first part the presentation closely followed the bullet points on the screen, so I have done my best to reproduce these.

James Wheeler began with Manugistics' goals for this new product:

1. The look and feel of a real Windows product
2. Very easy and powerful GUI features
3. Enhance programmer productivity
4. Open APL product which co-operates with other Windows products
5. Accelerate change in the APL language

He said it is more an attitude than a product; these goals will guide further work.

The product architecture is related to Windows:

- It uses Win32s, the Windows 32-bit architecture and won't run where this isn't available
- It runs under Windows 3.1, Windows for Workgroups 3.11, Windows NT
- In future it will run under Chicago, Cairo and Windows 4 (if that is the name for the Windows 32-bit upgrade).

The key features of the product are:

- A new Windows session manager and editor
- A debugger
- A user command processor
- Object-oriented Graphical User Interface
- A Visual Forms Editor so you can draw your interfaces
- Interface to Windows DLLs
- Dynamic Data Exchange
- Built-in graphics and printing
- New control structures
- A royalty-free run time system included with the product

It is a high-performance product. It had surprised even its authors by running computationally heavy work faster under Windows than APL*PLUS II ran the same work under DOS alone.

It has GUI features built-in and so is faster than the agent of APL*PLUS II.

The current workspace size limit is two gigabytes.

The built-in control structures are faster than branching.

About the control structures James said that there are many algorithms which cannot be elegantly expressed where the data has to be in flat arrays; with nested arrays some loops may be eliminated or, using each, removed at least from repeated interpretation. Even so real-world APL is still full of spaghetti code and the new control structures make it possible to match other languages, and may help overcome the prejudice of some DP professionals. APL has an implicit structure in that after line 3 it will obey line 4 unless told not to.

The new structures:

- are conforming extensions to standard APL
- provide a sort of 'outer syntax'
- improve the readability and structure of APL code
- are familiar to C and Pascal programmers

They are:

```
:if      :else   :endif
:while   :endwhile
:repeat  :until
:select  :case    :caselist  :endselect
:for     :in      :endfor
```

They all begin with the colon and must be on the beginning of a line (which therefore cannot be labelled). And as his final knock-down point in favour James said that Adrian Smith approves of this innovation so it must be right!

In a short diversion on the benefits of the Graphical User Interface (for the mainframers who hadn't yet encountered one) James said that the GUI gives the user a better intuitive understanding of how to work with software. By allowing exploration and allowing mistakes to be recovered it makes complex software much less intimidating, and it allows users to discover how an application works without reading the manuals. By contrast the old-fashioned fascist approach characteristically required the user to make valid entries in the order the programmer decided until the programmer deemed the user had earned the

chance to make a choice. The old approach hardly encouraged the user to feel in charge.

The GUI in APL*PLUS III provides forms (which are windows) and means to control the elements on a form. The control is via system commands `⎕wi` `⎕wself` `⎕wevent` `⎕warg` `⎕wres` `⎕wcall` `⎕na`.

Every part of a window is an object: an object is a self-contained lump of code and data. Other objects or software can interact with the object but cannot see inside it; all they can do is pass it arguments and maybe get results back. Objects can only be accessed through a well defined interface, with the overall result that GUIs are much simpler to build and operate.

There are two kinds of object in APL Graphical User Interfaces. They are:

Methods: think of these as functions inside the object which can be called with arguments.

Properties: think of these as variables in an object that can be set and referenced.

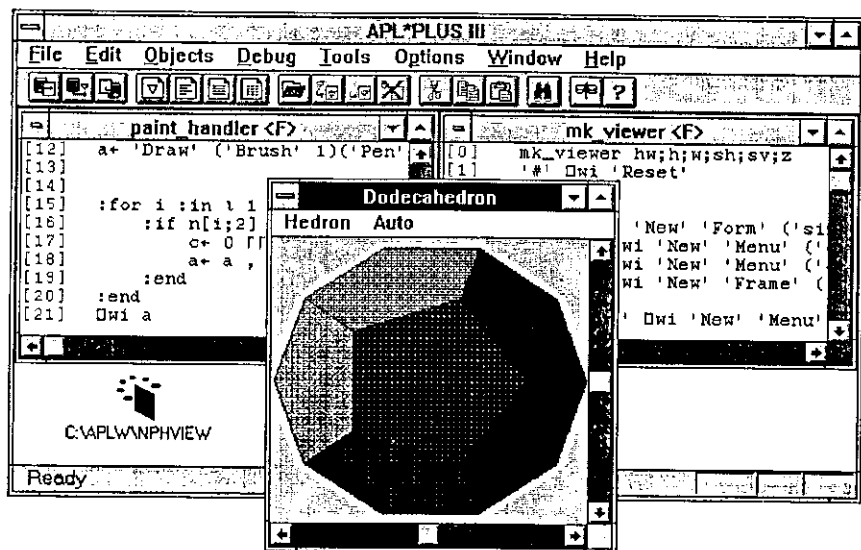
Objects take action when a property is set (for example by keying into a field) or a method is invoked (for example to close a form) or when an event is signalled (for example by pushing a button). An event is the system notifying the program that something has happened that requires or may require a response. The system uses events to tell the program what the user has done to the interface. James concluded by saying that he regards APL*PLUS III as an event-driven interpreter for writing event-driven programs.

Then he fired up the interpreter and, after checking that `2+2` and `19` produced the expected results, showed that `⎕clear 5m` gave a workspace of five megabytes by assigning a million random numbers from 1 to 10 to a variable and then summing it to give 5500000.

He then showed the options available on the menu bar, demonstrated that one could cut in any of APL, MSWord and Excel and paste into any of them. A workspace called *bouncer* showed a ball bouncing around the inside of a window. James showed how it was constructed and that he could change the colour of the ball or its diameter on the fly without ill effect.

Actually he was unlucky; he increased the diameter just as the ball was about to bounce and the change in diameter caused the horizontal movement to stop. I suspect that the change perhaps took the edge of the ball past the edge of the window without triggering the event that should have reversed its horizontal

increment. The ball then slid up and down the left side of the window. To my surprise, James released it by dragging the left edge of the window smartly left well beyond the ball position and it cheerfully resumed its bouncing as before.



In response to questions James said that `)clear` would not be enough to reset the windows objects. Once created they have an existence independent of the APL workspace. They must be cleared with a reset command.

He hoped to add user-defined GUI objects next and then to provide Object Linking and Embedding but was unable to give much idea of a timescale for these additions.

Questions only died down after the display equipment had been packed away in its box and numerous members of the audience had had to leave to catch trains. This new APL product certainly is of great interest to BAA members and James was heartily thanked for coming to show it to us.

AnOOPL — An Object Oriented Programming Language

Talk by David Piper; notes by Adrian Smith

Introduction

David's talk fell naturally into two main sections: firstly a summary of the benefits to the developer of the Object Oriented approach, and secondly some very speculative suggestions as to how APL could take the Object-oriented road with surprisingly little change to the appearance of the language. He clearly felt that it was wrong that companies like Eagle Star are being pushed towards C++ and Smalltalk just so that they can get the benefits of Objects; APL offers something quite different, so a fusion of object concepts with APL notation really should add power to both.

"This is what I want! Current status is "grey matter" only — only 2 days' experimenting so far"

Why Objects?

Partly because it is the fashion. A lot of people talk about it, so if APL has it they will talk about APL!

Partly because there are real benefits (see Adrian's *Technical Editorial* in Vector 10.4) to the developer, particularly when you are working on large, complex applications. You can expect: enhanced quality; more code re-use; easier and faster development; more "function points per £K"; easier maintenance. Perhaps you've heard this all before ...

- enhanced quality comes mainly from better *encapsulation*. We have all used name prefixing to mimic this, but (as with all manual methods) this is vulnerable to mistakes.
- *polymorphism* means you don't need to worry what sort of data you have, and *inheritance* means you only need to worry about what's new. You know that certain key functions are always available. This keeps functions short, so they are easy to write and fix.

APL has had code re-use since the beginning with)COPY, but it is too often ignored. Code-management tools have helped, as have APL2 packaged WS — OO makes this part of the language, and hence makes it automatic.

APLers are also used to short functions, and know that these are easier to write and to maintain. They are also fairly blasé about polymorphism (pity the poor C programmer who needs separate functions to do the same job for integers, unsigned integers, long integers, reals) so they should adapt well to an environment which offers "Point and Grunt" editing and makes modularity a design necessity with a "message passing" paradigm.

So — having established that an AnOOPL should help us to do things better, quicker and more accurately ...

What AnOOPL?

Should we be designing APL++ in the mould of C++? This implies patching an object layer on top of the existing structure, but there are some very real and fundamental problems with a hybrid solution. C++ has "Instance variables" which are private to each instance of a class (no problem here) but it also has "Class variables" which should be local to each class, but C has no mechanics for handling these, so any class can get at them and (potentially) tweak the behaviour of an unrelated class. We are back to manual discipline as the only means of ensuring proper encapsulation.

There are also lots of nasty complexities lying in the gungy stuff which sticks together the new object-based bits of an application with the older procedural bits. "Complexity gathers in the twilight zone", as you have to design a very different style of interface on each side of the horizon.

Conclusion — it would be much better to go for a SmAPLtalk which offers true object orientation with no $\square$ KITCHENSINK kludges, but somehow retains the look and feel of APL. Can it be done? Here are some preliminary ideas to set you thinking ...

How AnOOPL

Can we make a truly pure OOP that still looks like APL? Start by looking at a familiar syntax with different semantics:

α	FN	ω
<i>parameter</i>	<i>message</i>	<i>object</i>

There is still work to be done to implement the change in semantics, but at least by keeping the syntax, we keep APL!

A simple example ...

APL	AnOOPL
<code>date+ΔDGD 31 3 1993</code>	<code>date+toDate 31 3 1993</code>
<code>date</code>	<code>date</code>
<code>727653</code>	<code>31-03-1993</code>
<code>ΔDGC date</code>	<code>dayNumber date</code>
<code>31-03-1993</code>	<code>727653</code>
<code>2+date</code>	<code>2+date</code>
<code>727655</code>	<code>02-04-1993</code>
<code>ΔDGC 2+date</code>	<code>dayNumber 2+date</code>
<code>02-04-1993</code>	<code>727655</code>
<code>2×date</code>	<code>2×date</code>
<code>1455306</code>	<code>Domain Error (no method)</code>

APL reveals the internal data type to us — dates are stored as integers, so that's how we see them. In AnOOPL we are sending the `toDate` message to a numeric vector and getting back an *encapsulated* date which (among other things) knows how to display itself, which it duly does when we send it the 'see you' message (in APL we have always had the convention that 'no message' means 'display yourself') on the next line.

If we really want the day-count, we need to send our date object a `dayNumber` message. It has a method for this, which returns an integer. Similarly, we can send it a `+` message (with a parameter of 2) as it knows how to add numbers to dates, but we cannot sensibly send it a `×` message — whoever heard of multiplying a date by something! It does not have a method for this, so naturally it fails with a *Domain Error*.

Can arrays be fitted into this scheme? Given that arrays are the foundation of APL, they must also be the foundation of AnOOPL, so how might this be done ... here is another example:

APL	AnOOPL
<code>nested+date 'char'</code>	<code>nested+date 'char'</code>
<code>nested</code>	<code>nested</code>
<code>727653 char</code>	<code>31-03-1993 char</code>
	<code>classOf nested</code>
	<code>ArrayOfObjects</code>
<code>2p"nested</code>	<code>2p"nested</code>
<code>727653 727653 ch</code>	<code>31-03-1993 31-03-1993 ch</code>
	<code>classOf "nested</code>
	<code>Date ArrayOfCharacters</code>
<code>2+date 22</code>	<code>2+date 22</code>
<code>727655 24</code>	<code>02-04-1993 24</code>

Any nested or heterogeneous array is a member of class *ArrayOfObjects* — these closely resemble nested arrays in APL. Polymorphic methods are applied like “scalar pervasive functions”, and “Each” is available to apply methods item by item.

AnOOPL vs APL — Differences

So far, this talk has stressed the high degree of similarity between APL and AnOOPL, but there are four key differences, some of which arise out of object orientation, and some of which are just the removal of excess baggage (APL should have been like this anyway):

- static scoping. Variables are strictly local or global in scope. Variables which are local to an object are available to all methods of that object.
- no bracket indexing or axis specification. The ‘message passing’ paradigm requires a clean syntax.
- no niladic functions. All messages must be passed to an object.
- no distinguished names (system functions).

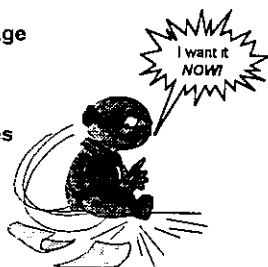
And as a final bonus — no more execute! Instead we can have the remarkable J-like concept of an ‘undespatched message’:

```
Rank←pp
Rank 2 2ρchar
2
  classOf Rank
UndespatchedMessage
  Incr←1+
  Incr 5 2 0 ~2
6 3 1 ~1
```

to conclude
(with David’s
final overhead
foil):

AnOOPL

- Pure Object Oriented Language
 - not APL++ with kludges
- It is APL
 - only 4 differences
- Extensions into J-like features
 - undespatched messages
 - possibly “tacit definition”???
- Power of arrays and objects





The Definitive APL for Windows

The screenshot shows a window titled "Grid, ToolBar and StatusBar Objects". Inside, there's a menu bar with "File" and "Edit". Below it is a toolbar with various icons. A status bar at the bottom shows "Ready ...", "Ins", "Caps", and "Num". The main area contains a table with the following data:

	May	Jun	Jul	Aug	Sep
Chateau Cote-Daugay-ex-Made	212	1000	154	631	4
Chateau Cautet	681	206	837	709	
Chateau Cure-Bon-La-Madeleine	389	953	948	390	
Chateau Faurie-de-Soutard	12	192	984	245	
Chateau Fonplegade	258	234	102	220	
Clos Fourtet	599	228	319	700	
Chateau Franc-Mayne	162	486	861	814	
Chateau Franc-Pourret	431	259	371	394	
Domaine du Grand-Faurie	677	514	729	721	
Chateau Grand-Mayne	606	987	151	671	5

Experience counts

Since we launched Dyalog APL/W at APL92, nearly two years ahead of our nearest competitor, our customers have developed hundreds of successful industrial-strength GUI applications. With the benefit of their experience, we have enhanced and refined Dyalog APL/W into the mature, stable and above all useful Windows development tool that it is today.

Not only does Dyalog APL/W provide the most comprehensive set of GUI facilities available for any APL, but you can be confident that your workspaces will run *unchanged* on Unix workstations under OSF/Motif.

With Version 7.0 we have added support for Visual Basic Custom Controls, a powerful built-in Grid object, Numeric, Currency, and Date fields, ToolBar, StatusBar and TabBar objects, automatic context-sensitive Hints and Tips, Metafiles, MDI, 3-D Forms and Controls, a fully customisable Session, an ODBC interface, *Namespaces* for encapsulation, and a host of other improvements.

That's why Dyalog APL/W remains the professional choice. For a free trial copy, contact Dyadic or your local distributor today.

Namespaces and Dyalog APL Version 7

Talk by John Scholes and Peter Donnelly; notes by Adrian Smith

Introduction

This was the first chance any of us had had to see the new Dyalog environment, but perhaps more significant for the future of APL was the implementation of 'namespaces' which Dyadic have pioneered in version 7 of Dyalog/W.

John Scholes worked gently through a series of examples of Namespaces, and I felt at the time that he was answering many of Dave Piper's requirements from the previous talk. Vector will need to look very hard at the details of this idea in future issues — for the moment all we have time to do is report John's talk and leave you to judge for yourselves if Dyadic have got it right.

Peter Donnelly showed us a splendid pot-pourri of the new GUI features in Dyalog/W, and trumped James Wheeler's bouncing ball with the (inevitable?) bouncing duck. Again, there is far too much stuff here to review in detail, so all we have here is a few screen shots and some very brief comment to give you the flavour.

John Scholes on 'Namespaces'

John began by acknowledging his debt to Adrian Smith and Duncan Pearson for sowing the seeds of these ideas, back in January of this year. The notion of *namespaces* has been about in the APL world for a long time — it is really a very simple idea! Namespaces do for APL what directories did for DOS: up to now the workspace has been a flat structure (like A: before DOS 2.0), but with directories you had the chance to organise your files into sensible groups (like C:\word\docs\vector) which kept like things together, and also allowed you to have more than one thing with the same name!

"It's nice if the number of functions in a WSDOC don't outnumber the functions in the workspace you are documenting"

At a stroke, namespaces have the ability to resolve for ever the name-clash problems that have always constrained our ability to re-use code, and they eliminate the need for nasty kludges like local `⌶FX` to hide support functions within the utility that requires them.

John started with a clear workspace, and defined a couple of variables:

```
vec+14
)copy util DISP
DISP 1"vec
```

1	1	2	1	2	3	1	2	3	4
---	---	---	---	---	---	---	---	---	---

```
DISP [NL]"19
```

mat	DISP								
vec									

The next stage was to leave the 'root' namespace by creating a 'subdirectory' called `conv`, like this ...

```
[NS 'conv'      a Create namespace
#.conv
)cs conc       a ... and move to it
```

... at this point the WS name in the session's titlebar changed helpfully to 'CLEAR WS[conv]' to let us know where we were.

Now there were no functions and no variables to be seen. You can't see in or out of a namespace, it behaves just like a mini-workspace — `)COPY` operates as you would expect — and is very quick with very low overhead (a couple of pointers get switched). Note the use of '#' which is the name of the root namespace — this is different from the DOS convention where the '\' is used both as the separator and the name of root. Note also that there is only one `[WA` — in just the same way as DOS always reports the space on the whole disk.

```
)ed enc
m+enc v
m+b base t v

enc 13      a but of course base is missing!
VALUE ERROR .... just like being in a mini WS

base+4p2    a Creates base in this context so ...
->[lc       a off we go again!
1 1 0 1

)cs         .... goes back to root
#
)objects    .... who have we here?
conv
```

```

conv.enc 13      A switch to conv - run enc - switch back
1 1 0 1
base+'hello'     A mess up the number base
conv.enc 13      A ... but this still works fine!
1 1 0 1
conv.base++1     A Persistent locals!

```

The rule is that when you execute a qualified function name such as `conv.enc` it collects its argument and then switches context to the namespace given. This allows you to have persistent local variables (`enc` might refer to a 'global' in its working namespace) as well as collecting functions together.

Of course, system functions can also be qualified:

```

conv.NL "19      A runs a namelist in the conv namespace
conv.ED 'decode' A edits in the requested namespace
#.ED 'foo'       A edits in the root namespace
##.ED 'foo'      A edits in the parent namespace

```

... as can many system variables, such as `IO`.

Finally, the session itself is a namespace (with one or two system variables such as `PW`), so you can define variables and utility functions within it, and have them persist across `load`.

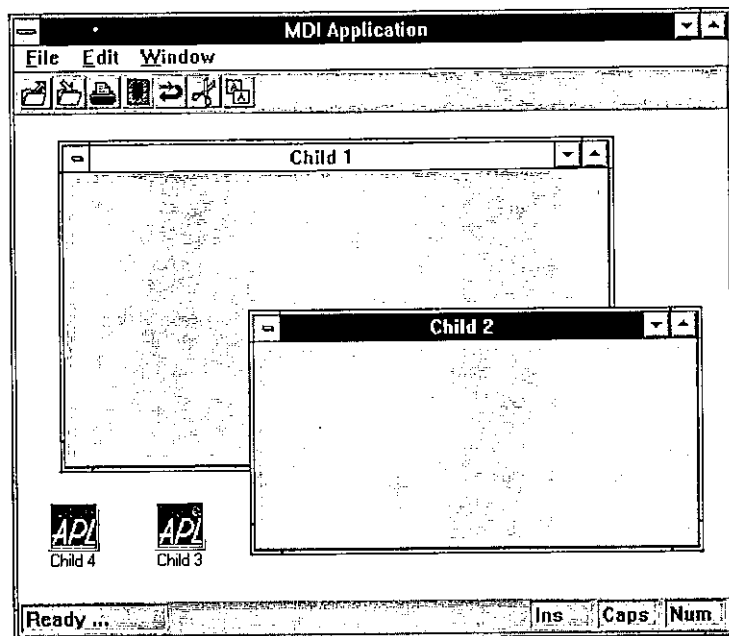
Mr. Donnelly and his Amazing Bouncing Duck

Peter started by illustrating two significant enhancements to the Dyalog environment: timers and the elimination of the need for an explicit `DD` to start objects listening. This allows you to make objects which respond immediately to events (which makes for much easier testing and debugging, as you can throw events to things from the session and see how they catch them), as well as having objects running in the background while you carry on working in the APL session.

Naturally, he showed a bouncing ball, and then upped the stakes significantly with a bouncing duck. Of course the duck could be modified 'in flight' with `duck+2*duck`, or its behaviour could be changed as it bounced, by editing the callback code. Peter also pointed out that the utility code for the callbacks all resided (as it should) in the 'duck' object (as all the GUI objects are now namespaces), so that the demonstration was quite a powerful example of the benefits of encapsulation, as well as being fun to watch!

Peter continued with a crash course on the .VBX extensions — these are a huge resource of 3rd-party tools which Visual Basic programmers have found essential. He demonstrated a 'cheap and cheerful' graph, and a file-manager tree as examples of the kind of tools which are readily available. The definition and operation of these looked clean and straightforward, and they operated easily enough, and without a hint of a GPF.

The next major topic was the definition of MDI frames, and the management of 'child windows' within them. For those readers who have never met Winword (or Excel or Jigsaws or any of a rash of new Windows toys) MDI is a neat way of having several 'documents' open within one application (for 'document' read 'spreadsheet' or 'drawing' or 'puzzle' as appropriate), such that they all share one main menu and toolbar. The main form is the MDI frame, and has some built-in actions, for example it knows how to cascade or tile the child windows.



MDI also tries to be helpful to the user — for example as you run up and down the menus (or allow the mouse to dwell over a tool in the toolbar) you can have a 'hint' appear in the status field at the bottom, or (horror of horrors) a little yellow 'speech bubble' will pop-up to advise you what the tool is meant to do. Microsoft have a lot to answer for!

Anyway — if you want to make your APL application look like MS Office, you can do it without lots of hairy APL code to look for mouse-moves. This stuff is much better supported by the interpreter, which can do it faster and more reliably than an application ever could.

We had time for just two more topics before the doors closed and we were thrown out to catch our trains. The 'grid' object is surely the answer to many an APLer's prayers. Getting a numeric matrix editor to work reliably was a tough challenge in Dyalog/6, and linked scrolling columns (which is what you need to edit any kind of relational table) was possible, but too slow to be comfortable for the user. The grid looks excellent — it is visually very effective, and you can attach more or less any 2-dimensional variable to it. The behaviour can be set at the cell level (so you can have one column read-only, the next one blue, and the third one set to accept dates). It has proper row and column titles (so it handles the linked scrolling for you) and generally looks to take Windows programming out of flatland into a genuinely multi-dimensional space.

	20 Years	21 Years	
7.0%	£930.36	£910.17	£
7.5%	£966.71	£947	£
8.0%	£1003.73	£984.51	£
8.5%	£1041.39	£1022.69	£
9.0%	£1079.67	£1061.5	£
9.5%	£1118.56	£1100.92	£
10.0%	£1158.03	£1140.94	£
10.5%	£1198.06	£1181.52	£
11.0%	£1238.63	£1222.65	£
11.5%	£1279.72	£1264.29	£
12.0%	£1321.3	£1306.44	£
12.5%	£1363.37	£1349.06	£
13.0%	£1405.89	£1392.14	£

Finally, Peter showed us how to place a Windows 'metafile' on a form (or a button, or an image — in which case you could make an array of them), and how you could take it apart within APL. In the simple case, this just gives us the ability to save graphics to the clipboard in a scalable, compact format, but there were clearly many more interesting things you could do once you had decomposed the metafile in the workspace.

Commentary

by Adrian Smith

This was just a first look at late beta code, but I have to say that it showed a lot of promise. I think that the addition of 'namespaces' to the GUI objects must go a long way towards combining the functional syntax of APL with all the benefits of encapsulation and modularity that Dave Piper was seeking in his talk. It would be rather nice to have the benefits of the Smalltalk environment without the weirdness of the syntax!

The analogy with DOS is a good one. When I owned a twin-floppy 256K PC, I 'loaded' applications from the floppy disks which I kept in a stack of plastic boxes under my desk. The applications were small (e.g. Turbo Pascal ran from a single .EXE which fitted easily into 360K) and self-contained, so why would I need a directory? I did once experiment with `mkdir`, but really, what was the point?

When hard disks first became common (the original 10Mb kind), people still tended to dump everything into a single root directory, and rapidly lost track of things in the resulting mass of files. We soon learned that not only did things become more manageable with directories, but they ran rather faster into the bargain, because DOS had far fewer names to hunt through.

And what of the bouncing duck? I think Dyalog missed a trick by omitting timers from the very first version of the Windows product. How else do you write decent ~~games~~ simulations? It is also good to have a live APL session while your objects run — this makes it so much easier to test callbacks under controlled conditions. The MDI stuff is probably necessary, particularly in the big corporates who tend to lay down tight interface standards along the lines of "if it doesn't look like Word we're not having it around here". It also couples well with namespaces, as it becomes far easier for an application to open several files simultaneously when the data variables can be localised within each child window.

Metafiles are essential (at least for me they are), and I shall explore the .VBX additions with great interest. I spent a couple of days mocking up a tree-editor in Dyalog/6 — if the .VBX is any good at all I shall gladly throw mine away and use one that works. The grid looks brilliant — I have been using Excel and DDE for 'quick and dirty' data input for too long, hoping that something like this would relieve me of the burden of writing one of these with the low-level controls. I really can't wait to get my hands on this one!

GENERAL ARTICLES

This section of Vector is intended for articles about the applications of APL, and non-technical (i.e. having a minimal content of actual program code) material about the development of the language.

We start with a challenge from Peter Merritt.

There follows the first section of a two-part tutorial on Windows programming with APL*PLUS III by Eric Lescasse. In the first part, he introduces the session and development tools, and builds a simple GUI application. In the second part he goes on to develop the application with the introduction of fonts and colour, as well as the use of a 'timer' to make it live.

We also have a description of the BAA's membership database, which provides a skeleton system for anyone wanting to build a simple membership system in APL*PLUS.

To Be, Or Not To Be — That is the Gazodenplatz

by Peter Merritt

A number of years ago I was asked to investigate the feasibility of an email system on an IBM mainframe using APL for the end-user control software. During this project, the idea of actually protecting the data came up (OK, I suggested that senior officers sending "Councillor X, what a waste of space, eh?.....", in clear was a rather, ah, short-term activity). This article is therefore by way of an introduction to some simple, rather 'off-the-wall' techniques for data encryption which I doodled at the time. There is nothing new about them (I've just used some APL primitives to make things easier), but I've been pleasantly surprised on three separate occasions since then by the reactions of newcomers to APL when I showed them the results. Unfortunately, the last person to whom I mentioned this work was a member of the editorial committee of VECTOR - baaaddddd mistake.....

Anyway, having avoided the evil moment for a few months, I now offer the following text (in 5x character groups) as a form of competition, the solution (if it's not too obvious) being in the main article which will be in the next issue (which can be interpreted as either a 'taster', for regular subscribers, or as a form of annoyance for those who will only get this copy as a 'freebie!').

```
SRWL8 9xjdf 1GJA2 CafQf mnwBz 2Dce0 ZXq6a 2rRTy vRprd AT7h6 jmePT
cuYto FizQU 1KSmp Mqr3L DjsH1 YaECF sYmgP uHlnr GvM12 jON9l XgWqX
hjeB6 S4VwK o7Zkp qn0WL xjtoS R89xj WLwBe OJATy vRWld ASRep 2Cpr1
Ga2uY TcmnQ iq6df to89T 7Dcxj rRjmZ Xz2h6 afWLP qT7pq QfuHp qQfWL
ppqQH iZXuH T7pqh 6Qfpq 12ton 0Qfpq pqn0p qq6Hi n0LDM Q12T7 12xj2
CQrWL uHppL DECWL ZXQfH iWLMQ Dcxjp qpqTc pqpqn 0Tc2C pqQfp qwBMQ
jmWL1 GuHpp QfT7a 2HiMQ Hia2p qtopq jmLDu HpqTc topqS Rpqjs ZXQfY
MqrWL ZXQfZ XpqmQ LDWLL DpqHi Qfpqh 1T7WL WLpqn 0LDpq HiuHp qpqpq
h6t07
```

First prize will be a free copy of VECTOR; second prize two copies etc. etc. Good hunting.

Windows Development in APL*PLUS III

by Eric Lescasse (CompuServe 70731,3233) Part I of II

Introduction

The goal of this article is to show how to approach building Windows applications with APL. To do this, we will use APL*PLUS III for Windows, version 1.1, the new product from Manugistics, Inc. You will see that programming Windows applications, a task reputed to be more difficult overall than DOS application programming, is in fact

- within the reach of all APL developers, even the most occasional ones
- much more simple than developing in APL under DOS.

As one example of this simplicity, the French APL keyboard is handled automatically without your having to do anything. Printing APL code is equally simple: you need only select a part of the APL session with the mouse (or the keyboard) and choose Print in the File menu. This means no more APL character downloads, or utility programs to enable the French APL keyboard!

The resulting applications involve much less APL code, and are much more reliable and easier to maintain. Moreover, you still continue to have 100% of the traditional characteristics of APL at your disposal, in particular its unequalled interactive development environment, its great power for modelling and data processing, and its unsurpassed speed for processing large amounts of data.

The APL*PLUS III Working Environment

APL*PLUS III is a pure Windows application, having been developed with the collaboration of Microsoft, with a modern, Windows-style development interface.

APL*PLUS III for Windows is a Windows MDI (Multiple Document Interface) application, in the tradition of major Windows products such as Microsoft Word and Excel. This means that there can co-exist on the screen as many "documents" as you like. In the case of APL*PLUS III, these documents will be: the APL session, the editing sessions (functions and variables) and the numeric editor.

The ability to use multiple documents simultaneously is particularly useful: it gives you separate windows to view your APL session and multiple APL

functions and variables, and the ability to cut and paste between the sessions using the mouse.

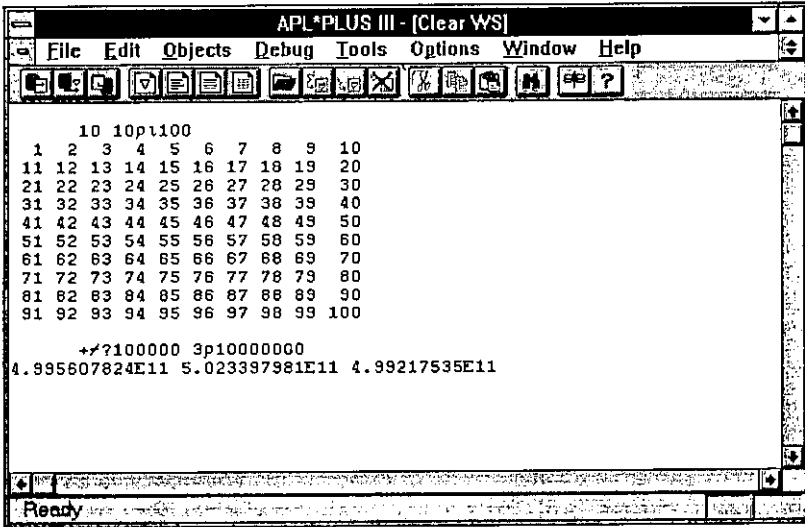


Figure 1—The APL*PLUS III working environment

Choosing menu options in APL*PLUS III causes dialog boxes like the following to appear:

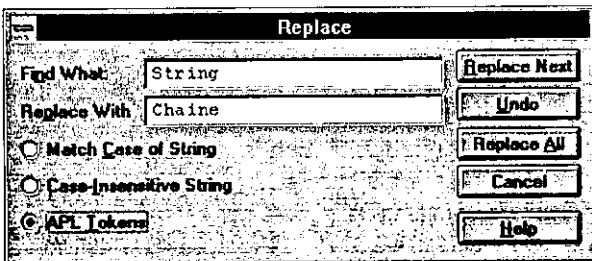


Figure 4—Dialog box for replacing a character string

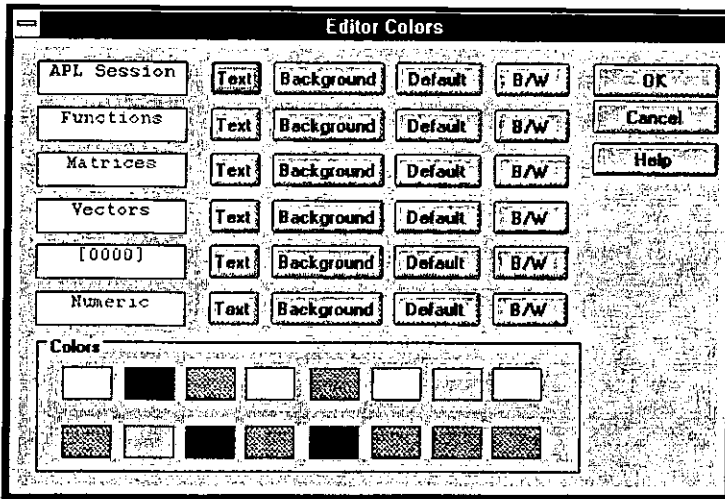


Figure 5—Colour choices for APL*PLUS III editing sessions

Or, more simply, to load a workspace:

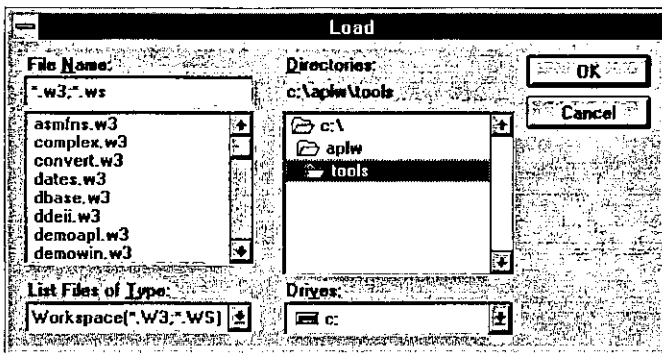


Figure 6—Dialog box for loading a workspace

These few examples should suffice to give you an idea of the APL*PLUS III development environment.

Note that with the Tools... dialog in the Options menu you can dynamically modify the APL*PLUS menus to install your own most frequently used commands in the Tools menu. This facility is particularly enjoyable to use.

Developing an Application under Windows

Now we are going to develop a complete Windows application with APL*PLUS III. The example that we will use is intended more as an illustration than actually to be useful. The following are provided for developing a Windows application:

- the system functions `⌶wi`, `⌶give`, and `⌶call`
- the system variables `⌶wself`, `⌶warg`, `⌶wres`, and `⌶wevent`

Let's begin in a clear APL workspace:

```
)clear  
CLEAR WS
```

Let's create a Windows window, or "form":

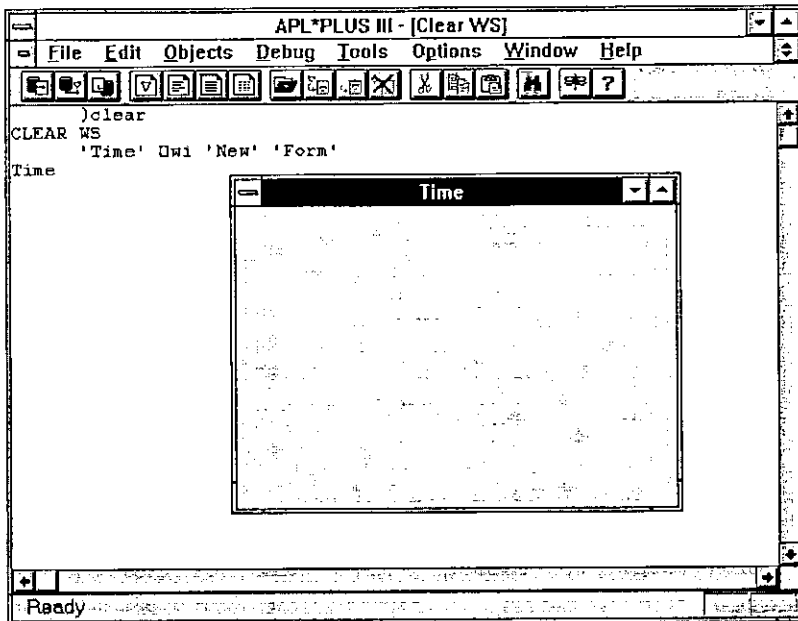


Figure 7—Creating a Window

The instruction `'Time' [N]w 'New' 'Form'` creates a new window named *Time*. *New* is the name of the method used. *Form* is the name of the object class.

The window created has a default size equal to half of the width and height of the screen and is centred on the screen. It has a maximise and minimise button as well as a system menu. Its title, or "caption", is by default the same as the name of the window, which is "Time".

Windows programming in APL*PLUS III is true object-oriented programming, called APLGUI. It lets you manipulate the following classes, or types of objects

Object Class	Meaning
#	System object
Button	Push button
Check	Check box
Combo	Combo box, an input field with drop-down list of choices
Edit	Input field
Form	Window
Label	Text label
List	List of choices
Menu	The form's main menu
Option	Radio button
Printer	Printer object
Scroll	Scroll bar
Timer	Period interval timer
Toolbox	Button bar or tool bar

These objects are organised in a hierarchy, and can inherit certain “properties” from their “parents”. In the same way that a pencil has properties (colour, weight, length,...) the APLGUI objects also have properties that you can easily modify using the system function `⌈wi`.

Let's add an object to our Time form:

```
'Time.time' ⌈wi 'New' 'Button'
Time.Time
```

Our screen now looks like this:

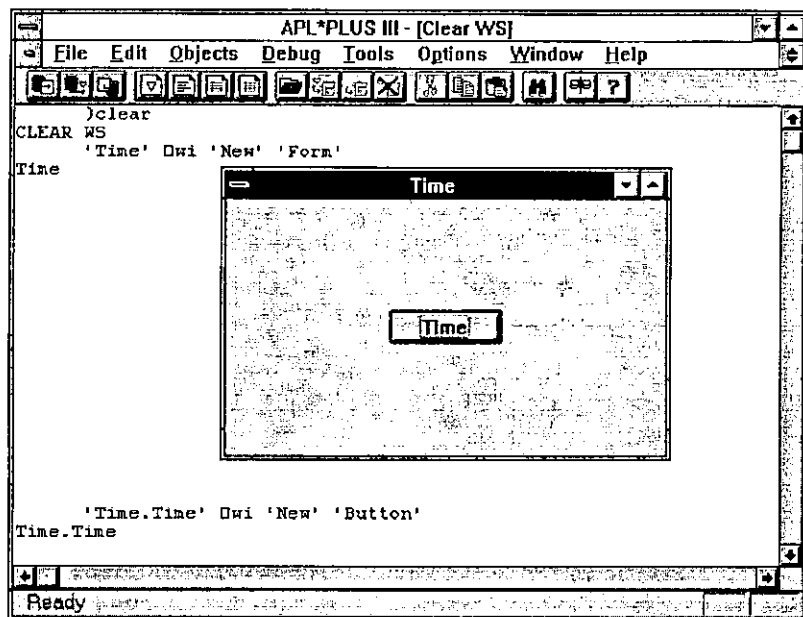


Figure 8—Creating a button in a form

The button is named Time (like the form, but we could have given it any other name) and it is by default placed in the centre of our window. The result of the function `⌈wi` is the name of the object created: `Time.Time`. The period indicates the hierarchy: the button `Time` is a “child” of the form `Time`.

Now, let's associate an action with the time button, with the following instruction:

```
'Time.Time' ⌈wi 'onClick' '⌈ts'
```

This instruction indicates that when the event *onClick* (that is, a click with the mouse) is produced with the object *Time.Time*, the character string '*⌈ts*' should be executed. *⌈ts* is an APL instruction that returns the current date and time in the form of a numeric vector.

Now, let's push the button *Time* three times. Our APL*PLUS III screen will then have the following appearance (after having moved the window *Time* over and down a little with the mouse):

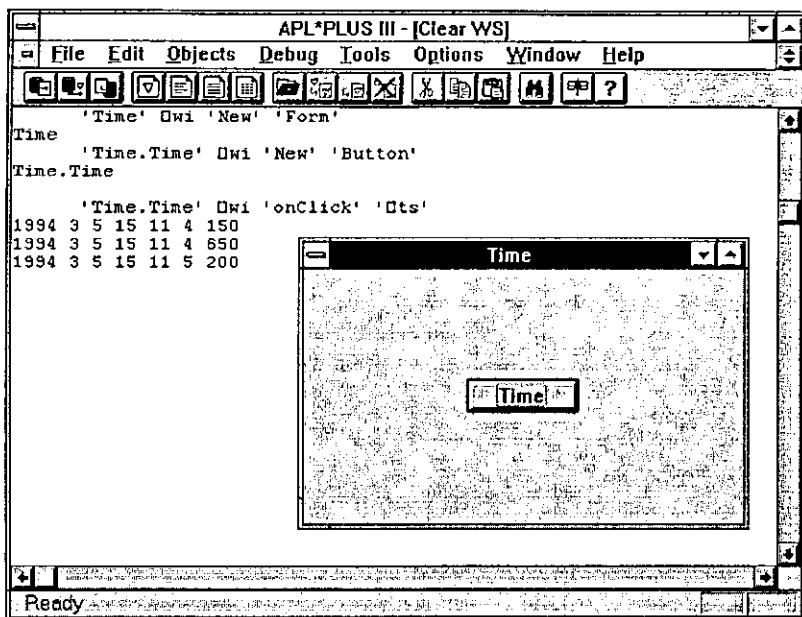


Figure 9—Clicks on the button *Time*

Note that we could return to the APL session simply by clicking with the mouse in the last example: the window *Time* then disappears, covered by the APL session. Typing Alt-Tab makes it reappear.

We now have a little Windows application, complete and functional. We only needed to use the following three instructions:

```
'Time'      □wi 'New' 'Form'
'Time.Time' □wi 'New' 'Button'
'Time.Time' □wi 'onClick' '□ts'
```

Note that we could have achieved the same result with a single call to the function □wi, by writing this:

```
'Time' □wi 'New' 'Form' ('Time.New' 'Button' ('onClick' '□ts'))
```

or this:

```
var+ 'Time.New' 'Button'
var+ var, c 'onClick' '□ts'
'Time' □wi 'New' 'Form', c var
```

Independence between Forms and Workspaces

The forms that you create are independent of the workspace, which is a major advantage. It means that you can freely change the active workspace without affecting the functioning of your forms.

To illustrate, let's load a new workspace, for example C:\APLW\TOOLS\UTILITY

```
)xload c:\aplw\tools\utility
C:\APLW\TOOLS\UTILITY SAVED 02/17/1994 18:15:00
```

Press Alt-Tab to make our *Time* form reappear. Push the *Time* button. The time is displayed in the APL session and our window is still functional!

Note that if you close the window *Time* by choosing Close from its system menu, you need only enter 'Time' □wi 'Wait' to reactivate our little Windows application. Close the *Time* window, then enter:

```
)clear
CLEAR WS
'Time' □wi 'Wait'
```

The *Time* window reappears; moreover, it still works correctly.

Improving our Windows Application

Instead of displaying the time in the APL session, let's make it appear in our form. To do this, we will create a **Label**, which we will position in the upper left of the form and in which we will display the time.

Let's create the label:

```
'Time.lTime' ⍳wi 'New' 'Label' ('where' 1 1 1.2 20)
```

Notice that this time, in creating our label, we changed its where property (its position in the window) by putting it one unit (=1 character) from the upper left corner.

Our form *Time* now looks like this:

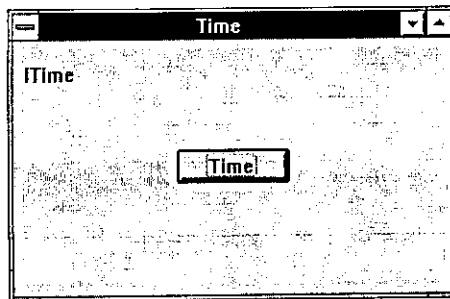


Figure 10—The form *Time* with a button and label

The name of the label (*lTime*) is written in it by default. We will clear it, and also add a frame to the label:

```
'Time.lTime' ⍳wi 'caption' ''
'Time.lTime' ⍳wi 'border' 1
```

This could have been done in one instruction:

```
'Time.lTime' ⍳wi 'Set' ('caption' '')('border' 1)
```

using the method *Set*, which lets you change several properties of an object at once. Next, let's modify the expression associated with clicking the button *Time*, so that it will now display the time in our label:

```
'Time.Time' ⍳wi 'onClick' '' 'Time.lTime' ⍳wi 'caption' ('⍴ts')
```

Or, alternatively and more simply, let's change the *onClick* property of our button as follows and write a function *Time*, containing the following APL code:

```
'Time.Time' ⍋wi 'onClick' 'Time'

▽ Time
[1] 'Time.lTime' ⍋wi 'caption' (▼⍋ts)'
▽
```

Such a function is called a "callback" function, which means a function that is automatically executed in response to an event. Make your form *Time* reappear by typing Alt-Tab, then click on the button *Time*. Your window will look like:

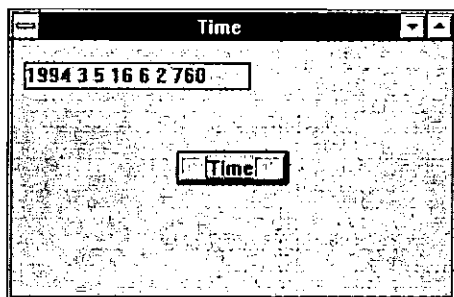


Figure 11—Clicking the button *Time* writes the time in the label

Now our application has become a real Windows application that no longer needs the display of the APL*PLUS III working environment to function. You can iconify APL*PLUS III by clicking on its minimise button (at upper right).

To summarise: This is all that we have done so far:

```
'Time' ⍋wi 'New' 'Form'
'Time.Time' ⍋wi 'New' 'Button'
'Time.Time' ⍋wi 'onClick' '⍋ts'
'Time.lTime' ⍋wi 'New' 'Label' ('where' 1 1 1.2 20)
'Time.lTime' ⍋wi 'Set' ('caption' '')('border' 1)
'Time.Time' ⍋wi 'onClick' 'Time'

▽ Time
[1] 'Time.lTime' ⍋wi 'caption' (▼⍋ts)'
▽
```

You will note that this complete little Windows application uses only one APL primitive, ▼ (format), which is necessary to transform the numeric date and time ⍋ts into characters.

The BAA Membership Database

by Nicholas Small

Introduction

Our membership database system was re-written in APL*PLUS/PC some little while ago. As the techniques used would be suitable for any simple database, it was thought that an outline of its operation might be of interest to members.

The primary purposes of the membership database are the maintenance of details relating to members and the printing of labels for meeting notices, Vector mailings, and membership cards. In addition, addresses are accessed for the sending of invoices and processing of credit card payments. There are a number of potential additional uses for the database, for example to record those members who are willing to answer queries from people learning APL.

The use of personal data in the database is governed by the Data Protection Act, our registration under which is described in Vector 9.3, p.9. Your Committee has resolved that members' details shall not be revealed to third parties for mailing purposes.

History

The original database was run in a proprietary database package that was not owned by the Association, nor was it a package that was familiar to the (then) new administrative assistant or her technical support team (myself). The only software on the administrative assistant's PC (an 80286 machine) that could be applied to the task was APL*PLUS/PC (version 7) and by chance I happened to have a database package in that very language, written to process time-sheets and monitor projects for a company long since defunct; I could see that by throwing away about 95% of the application code we would be left with something that could readily be converted to the task in hand.

My database package had used component files to store data, but for the membership system I opted for a native DOS file with fixed length records for the main data (component files are retained for control data). This format was chosen for two reasons: such a file can be used by a program as a direct access file, to allow fast input and output; also, it is possible to read and manipulate the data using one's favourite text editor.

When deciding on data handling techniques for a database, the options available depend on the size of the database, both in terms of number of records and in the size of individual records. Our membership database is currently rather more than 250k bytes (there are nearly 900 records of length 284 bytes, including CR/LF). If the database were half this size there would be scope to manipulate it as a single object within the APL workspace. At the other extreme, if the file was much more than twice its current size then a simple text editor would not read it in its entirety either. (Even with a PC that had extended memory and an APL interpreter and text editor to make use of it, these limitations would apply for a database of more than a certain size.)

Description of the Data

The main fields in the database are:

- a zone code, used to group labels for posting;
- a unique membership number (previously, this had only given a unique identifier when combined with the zone code, so some re-numbering was needed);
- a sort field (based on Name or organisation);
- name and address (this is three-quarters of the record);
- membership details and contact dates.

There are no separators between fields; this reduces the amount of processing required, at the cost of using global variables to index fields in a record.

Four component files are used, three of which are log files — for membership cards, mailing label runs, and invoice details. The fourth file contains semi-permanent data, for example correspondence and remittance addresses for invoices, subscription rates, the membership numbers of the current committee, and a template for the display of a membership record.

Program Structure

The system has a simple hierarchical structure, with selection by position of the menu bar. The main menu has just three sub-menus — one to access membership records, one to print labels, and Utilities to do everything else. On the first of these, there is an option to add a new record and two options to select a record: either by membership number, for which there is a prompt, or via the sort field,

with a prompt that leads to a display of the (sorted) sort fields as a menu, starting at an appropriate point.

The labels menu starts with two toggle items, one setting the number of lines to be printed per label (depending on what stationery is loaded in the printer) and the other determining whether or not expiry dates are printed on the labels. Most of the other options actually print labels; the list that is referred to is a vector of membership numbers which is built up by pressing a function key when editing the membership record.

The Utilities menu includes options to call functions such as *MCOUNTS*, which provides an analysis of membership numbers and Vector circulation.

There is a start-up routine which defines constants, reads the semi-permanent data from the component file, and gets all membership numbers and sort fields from the database (the sort field is 30 characters, so these take up about 32k bytes, which is not a problem in terms of workspace size). The defined constants include all application-specific variables used in the workspace, ranging from menu text to the location in a membership record of, for example, the membership class field; consequently, when a change has to be made, things are all in one place.

Workspace Functions

The main functions in the workspace may be divided into three groups: screen handling functions; functions to read from or write to the database file; and functions for printing.

The functions for screen handling — menus and record editing — are based on the utilities supplied with APL*PLUS/PC in the *INPUT* workspace. Only minor modifications were made to these functions, for example, to accommodate the chosen screen layout — a couple of lines at the top for titles and a few lines at the bottom for instructions and messages, with the works in between. The function *sFIELDS* takes a mask of the form shown in Figure 1, returns the parameters for the fields defined by the +s, and puts the text up on the screen; this function was modified to put the text into a variable, which, after dropping the first two rows, I could then write to the screen myself, as and when required.

Another modification was to the function *sINPUTFIELDS*, which allows data to be entered in fields on the screen. Here some user-defined input types were introduced, to inhibit the input of negative or exponential format numbers where these were inappropriate.

The only non-trivial modification made was to the function that writes data to the screen: the supplied routine (*sPUTFIELDS*) writes data to fields individually, which was painfully slow in the original applications which had four times as many fields, and was running on a PC with an 8086 chip. It is much faster to read the basic screen image into a variable, insert text as necessary, and re-display — see the listing of *qPUTFIELDS* in Figure 2. Of course, one could improve the speed even more by working out beforehand the indices of the characters in all the fields in the ravel of the screen image and putting the data directly there, so eliminating the loop.

To get data into and out of the database, there are functions to read, write, and append fixed length records, plus one to read specified columns for all records. These are all straightforward and are shown in Figure 3.

Of the print functions, the one most frequently used is that to print a label, which is called by a number of functions that select the records for which labels are required. As mentioned above, the membership numbers for each label run are stored on a component file for reference. Most of the other print functions are for the benefit of the Treasurer — for the production of invoices or statements, and to process credit card payments. The last of these is a great boon: previously, the Treasurer had to write out credit card slips by hand, which was tedious and error-prone; now, the computer checks the card number (which should satisfy a modulus 10 Luhn formula and generates a print-out that is submitted in place of the more familiar voucher. (The processor makes a surcharge for electronic submissions, else we would avoid the use of paper.)

The modulus 10 Luhn formula that credit card numbers satisfy is the following expression, where DIGITS is a vector of the 13 or 16 digits in the number:

$$0 = 10 \mid + / Y + 9 < Y + (\phi DIGITS) \times (\rho DIGITS) \rho 1 \ 2$$

Conclusion

Using the functions in the *INPUT* workspace as supplied, plus functions to access the data, as in Figure 3, a workable database system can easily be created. (A better starting point would be a similar family of functions, written by Adrian Smith, that are available from APL-385 on receipt of a blank disk and return postage.)

With some relatively minor modifications to those functions, one would have a base from which a database application generator could be created.

```

MEMBER'S DETAILS

ZONE + IDENTIFIER ++++
SORT FIELD ++++++

NAME ++++++
ADDR1 ++++++
ADDR2 ++++++
ADDR3 ++++++
ADDR4 ++++++
POSTCODE ++++++ COUNTRY ++++++

yy mm dd yy mm dd
DATE JOINED ++/+/+ LAST CONTACT ++/+/+

COPIES +++ MEMBERSHIP CLASS ++ EXP ++ AMOUNT O/S ++++
(or: Ayy)

FLAGS + + + + + + + + +
1 2 3 4 5 6 7 8 9 10

```

Figure 1 - Membership Details

```

v DATA qPUTFIELDS FIELDS;SCRN;I;J;K;X;W;WIN
[1]  a Put data into various (one-line) fields on the screen
[2]  WIN← 0 0 25 80
[3]  SCRN←WIN □WGET 1
[4]  X←1↑ρFIELDS
[5]  I←J+0
[6]  L1:→{X<I+I+1}/L2 ◊ W←FIELDS[I;]
[7]  SCRN[W[1]+W[3];W[2]+K]←DATA[J+K←,W[4]] ◊ J←J+W[4] ◊ →L1
[8]  L2:WIN □WPUT SCRN
v

```

Figure 2 - Faster Screen Output

Figure 3 - The APL Code

```

V Z←DAREAD PARMS

[1]  ⍺ Read a record from direct access native file.
[2]  ⍺ PARMS is: tieno, record length, record number

[3]  Z←⍺NREAD PARMS[1],82,PARMS[2],PARMS[2]×PARMS[3]-1
V

V Z←DATA DAWRITE PARMS

[1]  ⍺ Write a record to direct access native file.
[2]  ⍺ PARMS is: tieno, record length, record number

[3]  DATA ⍺NREPLACE PARMS[1],PARMS[2]×PARMS[3]-1
[4]  Z←1
V

V Z←DATA DAAPPEND PARMS

[1]  ⍺ Append data to (direct access) native file.
[2]  ⍺ PARMS is: tieno
[3]  ⍺ Overwrite eof with new record followed by CR, LF, eof.

[4]  (DATA,⍺TCNL,⍺TCLF,⍺AV[27])⍺NREPLACE PARMS,~1+⍺NSIZE PARMS
[5]  Z←1
[6]
V

V Z←COLS DAGETCOLS PARMS;BITE;S;X;TT;ROWS

[1]  ⍺ Get a matrix of columns from direct access native file.
[2]  ⍺ PARMS is: tieno, record length
[3]  ⍺ COLS are columns required

[4]  COLS←,COLS
[5]  Z←(0,⍺COLS)⍺' '
[6]  TT←0 ⍺ Bytes read so far
[7]  →(PARMS[2]>S+⍺NSIZE PARMS[1])/0
[8]  S←PARMS[2]×⌊S÷PARMS[2] ⍺ Round down in case of EOF marker
[9]  BITE←PARMS[2]×ROWS+⌊32000÷PARMS[2] ⍺ Chunk size
[10] LP:→(0=BITE+BITE⌊S-TT)/0
[11]  ROWS←BITE÷PARMS[2]
[12]  Z←Z,[1]((ROWS,PARMS[2])⍺NREAD PARMS[1],82,BITE,TT)[;COLS]
[13]  TT←TT+BITE
[14]  →LP
V

```

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hackers' Corner: Native Files without tears	Adrian Smith	126
Technical Correspondence		129
Take it or Leave it	Jan Karman	
The Slings and Vectors of the BAA	Ajay Askoolum	
Adverb/Conjunction Combinations in J	Gordon Sutcliffe	
At Play with J - Control Structures	Gene McDonnell	136
A Windows-driven Menu Driver	Jan Karman	139

Articles already placed in Vector 11.2 ...

Understanding Statistical Theory Through Simulation	David Appleton
No SX please, we're APLers	Ray Cannon
A Common Approach to the Windows GUI for Dyalog/W and APL*PLUS III	Duncan Pearson
A Standard Font for Dyalog/W and Plus III	Adrian Smith
Axis Simulation	Allan Gay

Hackers' Corner: Native Files in Dyalog APL (without Tears)

by Adrian Smith

Introduction

One of the things you get used to in APL*PLUS/PC is fast, reliable access to DOS native files; indeed many developers have tended to use these in preference to component files for function filing (see for example my notes on *Δpack* in Vector 8.2 pp.125-126) because the APL*PLUS component files are somewhat slow by comparison.

In Dyalog/W, the opposite is true! Component files go like greased lightning and the *NFILES* auxiliary processor is (in my opinion) something of a dog. It has the unpleasant habit of locking out completely if the AP cannot find space to load, and it litters your workspace with a bunch of unhelpfully-named functions like *open*, *close* etc. If you would like the pleasure of wiping NFILES.EXE for ever from your hard disk, read on.

Raw Input and Output

This investigation began when I got interested in making noises with APL! I discovered that the Windows .WAV files are simply a short header, followed by a stream of byte values (say 11000 per second) in the range 0-255 (where 128 represents the midpoint) which outline the waveform. APL has these funny circle functions, which are adept at generating sine waves, so a suitable numeric vector will generate tones and harmonies very easily indeed. Exponential decay superimposed on random numbers gives a very authentic gunshot!

In order to investigate the header structure, I clearly needed to read the file as a stream of untranslated byte values, so turning to the SDK help-file I found enough to hack together the following:

```

V r←rget name;_lread;_lclose;_llseek;_lopen;fh;size;
  sink;blk;ct;bytes;get
[1]  A Get Raw file contents as a string of bytes.
[2]  A Returns 0 if file not found.
[3]  □NA'I kernel.P16|_lopen <0T I'
[4]  □NA'U4 kernel.P16|_llseek I U4 I'
[5]  □NA'I kernel.P16|_lclose I'

```

```

[6]   □NA'U4 kernel.P16|_lread I >U1[] U2'
[7]   r←0 ◊ fh←_lopen name 0 ◊ +(-1=fh)↑0
[8]   size←_llseek fh 0 2
[9]   sink←_llseek fh 0 0
[10]  a Maximum length is 64K per read, so get blocks ...
[11]   blk←64000|size ◊ r←0 ◊ sink←□WA
[12]  a Don't read lots of garbage with the last block (faster)
[13]  More: get←blk|size-pr
[14]   ct bytes←_lread fh get get
[15]   +(ct=0)↑Done a should never happen!
[16]   r,←ct↑bytes
[17]  a Have we got enough yet?
[18]   +(size>pr)↑More
[19]  Done: sink←_lclose fh
      ▽

```

The functions are ANSI-standard C, and closely match the syntax offered by *nfiles*. Because they are built in to Windows, they appear to run rather fast — here is a trivial example:

```

      rget '\\apl\dyalog\serial'
48 48 48 49 53 57 10

```

... note that this has a line-feed (10) but no carriage-return (13), indicating its Unix heritage! The other half of the transaction is filled by:

```

      ▽ {r}←name rput bytes;_lwrite;_lclose;_lcreat;fh;size;
        sink;blk;ct;bytes;put;len
[1]  a Put a string of bytes to file (no translation).
[2]  a Returns 1 if OK; 0 if failed.
[3]   □NA'I kernel.P16|_lcreat <0T I'
[4]   □NA'I kernel.P16|_lclose I'
[5]   □NA'U4 kernel.P16|_lwrite I <U1[] U2'
[6]   r←0 ◊ fh←_lcreat name 0 ◊ +(-1=fh)↑0
[7]  a Maximum length is 64K per write, so make blocks ...
[8]   blk←64000
[9]  More: len←blk|pbytes ◊ put←len↑bytes
[10]   ct←_lwrite fh put len
[11]   bytes←len↑bytes
[12]  a Have we finished yet?
[13]   +(0<pbytes)↑More
[14]  Done: sink←_lclose fh ◊ r←1
      ▽

```

... which will happily write out several K of meaningless noise in less time than it takes to blink. Note that you will need to sort out the CR/LF pairs if you want the file to appear to DOS as records. It is sensibly behaved, in that it will open (and truncate) an existing file, or start a completely new file without complaining.

Text Files

Having mastered the raw case, it occurred to me that I could probably use the same means to read and write text files as well — sure enough I can still beat the *NFILES* AP for speed, even though I am doing the ASCII translation in APL! Here are the two functions that do the job, without further comment:

```

▽ r←nget name;_lread;_lclose;_llseek;_lopen;fh;size;
  sink;blk;ct;bytes;get;asc;[]IO
[1]  ⍝ Get Native ASCII file contents as a vector of vectors
[2]  ⍝ Returns '' if file not found.
[3]  []NA'I kernel.P16|_lopen <0T I'
[4]  []NA'U4 kernel.P16|_llseek I U4 I'
[5]  []NA'I kernel.P16|_lclose I'
[6]  []NA'U4 kernel.P16|_lread I >U1[] U2'
[7]  ⍝ Make translate table
[8]  asc←256p'.'
[9]  asc[32+;32]←'!'#$%&'()*+,-./0123456789;:<=>?'
[10] asc[64+;32]←'@ABCDEFGHIJKLMN O PQRSTU VWXYZ[\]^_'
[11] asc[96+;32]←'`abcdefghijklmnopqrstuvwxyz{|}~'
[12] asc[9 11 14]←TC
[13] r←0 ⋄ fh←_lopen name 0 ⋄ →(1=fh)+0
[14] size←_llseek fh 0 2
[15] sink←_llseek fh 0 0
[16] ⍝ Maximum length is 64K per read, so get blocks ...
[17] blk←64000|size ⋄ r←' ' ⋄ sink←[]IO⋄
[18] ⍝ Don't read lots of garbage with the last block (faster)
[19] More←get←blk|size-⋄r
[20] ct bytes←_lread fh get get
[21] →(ct=0)+Done ⍝ should never happen!
[22] bytes←ct+bytes
[23] ⍝ Translate and append ...
[24] r,←asc[bytes]
[25] ⍝ Have we got enough yet?
[26] →(size>⋄r)+More
[27] Done:sink←_lclose fh ⋄ []IO⋄1
[28] ⍝ Eliminate LF and partition on CR ...
[29] r←1⋄r←TC[3]
[30] r←1+''(r←TC[2])<r
▽

▽ (r)←name nput txt;_lwrite;_lclose;_lcreat;asc;fh;size;
  sink;blk;ct;bytes;put;len
[1]  ⍝ Put a vector of vectors to file (ASCII translation).
[2]  ⍝ Simple vectors are tolerated!
[3]  ⍝ Returns 1 if OK; 0 if failed.
[4]  []NA'I kernel.P16|_lcreat <0T I'
[5]  []NA'I kernel.P16|_lclose I'
[6]  []NA'U4 kernel.P16|_lwrite I <U1[] U2'
[7]  r←0 ⋄ fh←_lcreat name 0 ⋄ →(1=fh)+0
[8]  ⍝ Make translate table
[9]  asc←256p'.' ⋄ asc[9 11 14]←TC
[10] asc[32+;32]←'!'#$%&'()*+,-./0123456789;:<=>?'
[11] asc[64+;32]←'@ABCDEFGHIJKLMN O PQRSTU VWXYZ[\]^_'
[12] asc[96+;32]←'`abcdefghijklmnopqrstuvwxyz{|}~'
[13] ⍝ Look up each element and add CR/LF pairs
[14] a(2>|=txt)/'txt←ctxt'

```

```

[15]  txt+>,/txt,"<BTC[3 2]
[16]  bytes+~1+255|asc:txt
[17]  * Maximum length is 64K per write, so make blocks ...
[18]  blk+64000
[19]  More:len+blk\rbytes * put+len+bytes
[20]  ct+_lwrite fh put len
[21]  bytes+len+bytes
[22]  * Have we finished yet?
[23]  +(0<\rbytes)*More
[24]  Done:sink+_lclose fh * r+1

```

▽

... obviously, Scandinavian readers will need to patch the translate table, depending on where in their $\square V$ they buried $\emptyset$ and other exciting things.

As a final bonus, I found that I could write stuff direct to the printer (by using 'prn' as the file name) without getting jumped on by Windows — just what my *RAIN* graphics needs to get its PostScript past the Print Manager queue!

TECHNICAL CORRESPONDENCE

↑ (it or leave it) in Dyalog APL/W and MS Windows

From: Jan Karman

May 9, 1994

I apologize for not having made use of your VectorAPL font, but please, read on.

This text was made in Windows Write. I feel normal, I look normal, so I think I'm normal, in spite of the facts that I've studied actuarial sciences and that I'm using APL for my work. For a normal person the normal way to get the ↑ on your screen, and — thanks to TrueType — on your print apart from APL is by Windows Write. You can't get it in Word for Windows (they say because there is one position in the character set in Word for Windows, which is reserved for a fixed space) and there is no way to trick Word for Windows. You can't get your ↑ in WordPerfect for Windows either (a conspiracy?), but there is a way to trick WordPerfect for Windows: paste your ↑ in Windows Write (here you must first pick your DyalogAPL font before pasting, otherwise you're losing your ↑), cut or

copy your $\dagger$ in Windows Write and finally paste your $\dagger$ in WordPerfect for Windows. Some people suggest you make your own font (some are surprised how easy that is) and others are suggesting you request a special VectorAPL font (then you definitely will lose some other character; and even if that character is your APL space you still lose it) and have a runtime Dyalog APL interpreter of about 1M bytes for free. Is that the way to do it?

I have to endure crashes during Microsoft Windows sessions once or twice a week, if not a day, because of its instability. Yes, Microsoft is bigger than we are. If you click on 'Font' or 'Character' in one of your favourite word processors for Windows you'll find an abundance of the most crazy and — to me — silly characters, while we aren't able to design one single properly working font for our daily work in one of the oldest computer languages. How patient or how gullible do computer designers think a human being can be?

I had expected the following: PostScript, the end of the most persistent obstacle, and then I thought: TrueType, even better, the real end of the nastiest problem. After 27 years we still have to beg for conformity (Gfeller & Kromberg) in not only one of the oldest computer languages, but in my view still the most promising one ... were it not that we can't even display our $\dagger$. The time I've spent on these exercises (for some people a feast — for me just a nuisance) is humiliating.

Or is it the sound of the apocalyptic cavalry behind the APL-hills: $\dagger$ it or $\ddagger$ it?

I often feel like Bill Cosby's portrait of Noah when he let him answer his neighbour's question about what an Ark might be: "How long can you tread water?" I'm not drowning yet!

I think APL is still there thanks to its mathematical consistency and in spite of ... well, let's see what happens.

Jan Karman
Billie Holidaystraat 13
1311 PN Almere
The Netherlands

[Vector Production — use <Insert Symbol> from normal text having first selected the DyalogAPL font — this is the only way I know to fool Word!]

To Suffer The Slings and Vectors of the British APL Association

From: Ajay Askoolum

May 94

I have read VECTOR regularly since 1986. During this time, the Journal has consistently echoed the ambivalent sentiment that 'we must convince the outside world that APL is the most powerful and most productive programming language' (VECTOR Vol 10 No 3 Page 6) and that the APL community is rapidly becoming 'a gathering of veterans from the FORTRAN wars! — so the language is threatened with extinction within the current generation.' (VECTOR Vol 10 No 3 Page 121).

Indeed this is not new : 'After APL 91, I upset a few people by wondering to myself if it had been a conference or a reunion.' (VECTOR VOL 10 No 2). I have used the verb 'echoed' because on the very rare occasions that the computing press mentions APL these days, it does so purely to decry the fact that the language still exists! Is this a conspiracy? If not how is VECTOR 'showing its [APL's] facilities in practice, illustrated by successful applications.'?

Since VECTOR does not explicitly state its objectives, I cannot reasonably take issue with the coverage it gives to J and Windows. However is it not presumptuous of VECTOR to join the fashionable crusade of Windows, OOPS and control structures while failing what most readers of VECTOR would regard as its most critical mission: 'To show that APL is the most powerful and most productive programming language by illustrating its facilities within successful applications in industry.' Given the absence of readily available reference material on APL, should not VECTOR capitalise on the opportunity to establish itself as the *de facto* standard for APL?

An extremely useful addition to VECTOR would be a keyboard map showing the APL symbols and a glossary of terms relating to them - on the inside back cover, perhaps? IBM APL2 must be the most widely used interpreter given its environment and supplier, especially with its)LANGUAGE facility. This interpreter is text based. APL*PLUS/PC must be a close contender for the second most widely used interpreter — as APL*PLUS II works better as a protected DOS application (no DDE, ODBC or OLE support), it too must be included in the category of text based interpreters. VECTOR itself has not published any findings on the usage of the various types of interpreters. If VECTOR is to contribute to the effort to demonstrate that APL is the most powerful and productive language (I strongly believe that it is), it has to represent APL as it is:

- a language with peculiar symbols
- a language in which the processing of vectors and arrays is intuitive
- and a language without control structures found in other languages.

DB McIntyre's contribution 'Language as an intellectual tool : From hieroglyphics to APL' (IBM Systems Journal Vol 30 No 4 1991) eloquently justifies symbols: "APL's concise notation helps us grasp the intellectual content of an algorithm without the distraction of extraneous and irrelevant matters prescribed by a machine."

My point is that APL symbols provide an undeniable advantage — it is free of (spoken) language barriers much as the other language-independent means of communications such as international ground to air signals, semaphore using two flags, road traffic signs etc. These have a consistent international meaning. Arrays and vectors are the most natural concepts in which programmers and non-programmers think of data : *CostOfShopping* is *PriceOfEachItem* times *NumberOfEachItem* not as the sum of the cost of the first item plus the cost of the second item plus .. until the last item. What of control structures? Surely the attraction of APL is that it does NOT have the formal control structures found in other languages.

I would find the introduction of these structures acceptable only in return for tangible benefits. Among these, I rank the following as the top two:

- increase in speed, perhaps arising from compilation of APL code at run time
- APL is no longer workspace bound, as encapsulated in the following type of structure:

```

WHILE NOT EOF(n) ◊
  ⍝ While the end of the file tied to N is not reached processes
  ◊
  ⍝ the application code WEND ◊
  ⍝ end processing

```

where prior knowledge of the shape of the data is not a prerequisite. A more pertinent consideration is the sort of control structures which APL will appropriate. It must be quite obvious to proponents of control structures that these structures have their own variations and limitations and that it is not a case of adopting the behaviour of these structures in any one specific product.

It is my strong belief that control structures exist in other languages because, unlike APL, their dynamism is restricted to scalars (one at a time, from the first to the nth where n is known in advance) and because of the inadequate use of

Boolean logic. In APL, Boolean logic is used to great advantage, thereby removing the need for IF THEN ELSE IF THEN .. structures. An added advantage of this approach is that it overcomes the need for hardcoded algorithms, and is more productive or tolerant of programmer omissions, depending of your point of view.

APL has survived for over 30 years in spite of all the adversity it has encountered. However, in the absence of strong sponsorship, its continued survival is endangered beyond the current generation as suggested.

VECTOR is eminently placed to promote APL in a more active way, by showing its relevance to business and by recruiting new entrants to the realm. How? My suggestions are:

- a section in VECTOR dedicated to the considerations of APL application development which will provide utilities which can simply be transcribed into commercial applications, sponsor and discuss legitimate coding practices etc aimed at the readership with working experience of APL
- a section in VECTOR dedicated to beginners, illustrating the comparative advantage of APL.

For instance, showing how the diagonal of a matrix can be extracted with ease i.e. $1 \ 1 \ 10 \ 10 \ 100$ yields $1 \ 12 \ 23 \ 34 \ 45 \ 56 \ 67 \ 78 \ 89 \ 100$ or how APL can generate all five possible results of a comparison of any scalar or $n \times 1$ matrix with a minimum and maximum i.e. $\bar{2}$ if less than the minimum $\bar{1}$ if equal to the minimum 0 if in the range of the minimum and maximum values 1 if equal to the maximum 2 if greater than the maximum.

Thus: with $\square IO \leftarrow 1 \ (5 \ 1 \ 10 \ 100) \ INRANGE \ 2 \ 4 \ \bar{2} \ \bar{1} \ 0 \ 1 \ 2$ where the function looks like:

```

      ∇ R←L INRANGE R
[1]   R←(×L-1↑R)+×L-1↑R
      ∇

```

- a tutorial service which is free to the student/school but where the resources are provided by APL vendors/sustaining members and experienced volunteer APL users
- making APL visible to the outside world by means of free demonstration disks, school competitions (for sixth forms, say) where the prizes can be tangible (copies of APL interpreters, courtesy of sustaining members) or complimentary (coverage in VECTOR, even free annual subscription)

- consolidating the suppliers of APL into a forum and organising an annual commercial conference (delegates from industry pay a fee) in the UK where the platform is dedicated to the demonstration of APL in business critical applications
- a survey amongst the readers of VECTOR to establish what the readers, (most of whom would be users of APL?) put APL to and how VECTOR can best cater for their needs

The promotion of APL on this scale can be undertaken by VECTOR as it has a wide circulation and is independent of a profit motive. Personal attempts to do so albeit on a much smaller scale is, in my experience, akin to providing a stick to be assaulted with. APL deserves better!

Ajay Askoolum
42 Hanworth Road
Redhill
Surrey RH1 5HT

Explanation of Adverb/Conjunction Combinations in J

From: Gordon Sutcliffe

May 9, 1994

I hope the following problem is of interest to J enthusiasts and look forward to a solution.

The explanation of the combination of adverbs and conjunctions in J, is only partially covered in Ref 1. Consider, for example, four interesting dyadic cases of the conjunctions & and @:

- | | | | |
|----|-------------|-----------|------------|
| 1. | x u & v & w | y u-v-w-x | (Dyadic u) |
| | | \v-w-y | |
| 2. | x u & v @ w | y u-v-w-x | (Dyadic w) |
| | | \y | |
| 3. | x u @ v & w | y u-v-w-x | (Dyadic v) |
| | | \w-y | |
| 4. | x u @ v @ w | y u-v-w-x | (Dyadic w) |
| | | \y | |

where *u*, *v*, *w* are verbs and *x*, *y* are the arguments of the dyadic verb resulting from the execution of the conjunctions. The empirical results are expressed in terms of the original verbs.

The problem posed is this: *How can these results be derived analytically?* As *J* has a simple structure [1,2], there should be a simple explanation of how multiple adverbs/conjunctions combine to modify their verb arguments. As a start, consider an explanation for long verb trains:

Long Verb Train Execution: In a long (four or more) verb train [3], the right verb of the initial fork or hook is substituted by the fork to the right. In the case of an initial fork, the right verb of the fork has the same valence as the train, and hence, the substituted fork(s) have the same valence as the train. In the case of an initial hook, the right verb of the hook is defined always as monadic, and hence, the substituted fork(s) are monadic regardless of the train valence.

A similar explanation might explain the action of the conjunction examples above. That is, the initial dyadic environment being modified as the conjunctions are executed from left to right, and determining the valence of the substituted verbs, when they are subsequently executed from right to left. However, this is not how *J* works. While case 1, 3 and 4 would be explained; case 2 would have the same result as case 1.

So, back to the beginning, the original unsolved problem. What are the principles of *J* [4] required for synthesis of an explanation of the combined action of adverbs and conjunctions? What is the simple explanation for the conjunction examples?

References

Ken Iverson, *J Introduction & Dictionary* Version 7, ISBN 1-895721-06-7

- [1] p1, Introduction, first paragraph: 'Although it [*J*] has a simple structure, ...'
- [2] p1, Introduction: '5. The systematic use of adverbs and conjunctions to modify verbs, ...'
- [3] p69, F. Trains
- [4] p10, Vocabulary: '(C) The appended *Dictionary of J* provides a complete and concise dictionary and grammar.'

At Work and Play with J: Control Structures in J version 8

by Gene McDonnell

There have been many proposals for control structures in APL systems before, and there are now current several APL systems which have them. This article will describe the control structures which are made available in the new release of J (version 8).

The material is largely drawn from the latest edition of the "*J Introduction and Dictionary*", available from Iverson Software Inc, 33 Major Street, Toronto, Ontario, Canada M5S 2K9. The documentation which I have seen was in a preliminary form at the time of writing, so be sure to get a copy of the official document if you would like to use these new facilities.

This article assumes that you, like me, are not overly familiar with the general notion of control structures. If in fact you are a long-time user of Fortran or Algol or C, please forgive these callow comments.

The control words introduced are:

```
break.  
catch.  
continue.  
do.  
else.  
elseif.  
end.  
goto_<name>.  
if.  
label_<name>.  
return.  
try.  
while.  
whilst.
```

The four control words `if.`, `try.`, `while.`, and `whilst.` mark the beginnings of control structures that are each terminated by a matching `end.` control word.

The control words `while.` and `whilst.` differ in that the test block in a `whilst.` statement is skipped the first time (the "st" in `whilst.` can be thought of as meaning "skip test") whereas in a `while.` statement it is always executed. As a

consequence, the execution block in a `whilst.` statement is always executed at least once, but in a `while.` statement, it may execute zero times.

The words `do.` and `else.` and `elseif.` occur within control structures, separating them into blocks. The control word forms `goto_<name>.` and `label_<name>.` represent an infinite family of possible control words, for each of which `<name>` is a different text. For example, one may write:

```
goto_ahead.  
...(statements)  
label_ahead.
```

or:

```
label_back.  
...(statements)  
goto_back.
```

A block consists of zero or more control words and sentences that are grouped together by control words occurring within a control structure. The role of blocks is summarized as follows:

```
if. T do. B end.  
if. T do. B else. B1 end.  
if. T do. B elseif. T1 do. B1 ... elseif. Tn do. Bn end.  
try. B catch. B1. end.  
while. T do. B end.  
whilst. T do. B end.
```

Words with B or T denote blocks. If the first (or only) atom of the result of the last sentence executed in a T block is zero, the B block following is not executed, otherwise it is executed.

In a series of `elseif.` `T1 do. B1`, if the `T1` are not exhaustive, it is good practice to put a final `elseif. 1 do. Bz`, where `Bz` is a block covering the default procedure when all else has failed, so that `Bz` is executed when no other test has succeeded.

Perhaps an example will make some of these details more concrete. The program p23 represents a crude but effective process for determining `x` as the two-thirds power of `y.` for `y.` any positive cube.

The statement numbers are not part of the program. They are shown only for reference purposes.

```

1  p23=.3 : 0
2  v=.0
3  w=.1
4  while. y.~:z=.v*x=.v*v do.
5      if. z>y. do. v=.v-w=. -:w
6          else. v=.v+w=. +:w
7      end.
8  end.
9  x
10 )
```

Statements 1 and 2 give initial values to the local variables *v* and *w*. Statements 3 through 7, inclusive, are a *while.* statement. Statements 4 through 6 inclusive are an *if.* statement. The T block in the *while.* statement compares the argument (*y.*) for inequality with *z*, which is the cube of *v*. If they are unequal, the result of the T block will be 1 (nonzero) and the *if.* statement will be executed. The T block in the *if.* statement determines whether *z* is greater than *y.*, and if it is the block following *do.* will be executed. Otherwise, the block following *else.* will be executed. The block following *do.* halves *w* and subtracts this from *v*; the block following *else.* doubles *w* and adds this to *v*. Continuing this process will eventually create a *z* which is equal to *y.*, making the result of the test zero, and when this occurs the *if.* statement will no longer be executed. Line 8 will then be executed, giving as the program's result the value of *x*, since the result of a program is the result of the last sentence executed that was not in a T block.

The purpose of the *try.* and *catch.* blocks is to permit recovery from a failure in execution. In a statement such as *try. B catch. B1 end.* if block B executes successfully, then B1 is not executed. If the execution of block B fails, then block B1 is executed.

The behaviour of the remaining control words can be summarized as follows:

- break.* Go to the end of a *while.* or *whilst.* control block
- continue.* Go the top of a *while.* or *whilst.* control block
- goto_<name>.* Go to the statement beginning with *label_<name>.*
- label_<name>.* Target of *goto_<name>.*
- return.* Exit the program

I must add a warning: I have not yet had access to the latest released J system, so that the information above is based on a reading of a preliminary version of the latest "*J Introduction and Dictionary*" and not on an actual machine execution. I have, however, submitted the program to two people who had access to the latest J system, and am told that it executed properly.

A Windows-driven Menu Driver

by Jan Karman

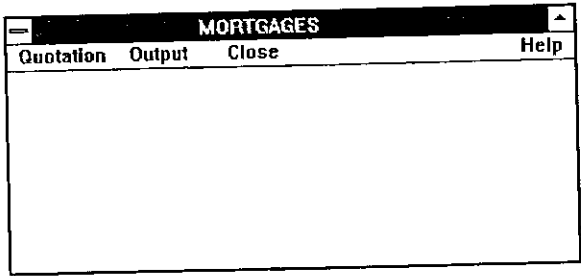
In Stephen B. Jaffe's paper at the APL86 Congress in Manchester a 'menudriven menudriver' was discussed. The driver consisted of a character matrix containing the menus and menu items, a pointer, pointing to the successive menu items and two functions, one of them displaying the correct menu items, and the other one pointing to the next 'group' of items or executing the lowest level menu item.

Now to do the same for Windows, we need to convert the same character array and menu pointers into a Window driven device. Suppose we have an application on making quotations for mortgages in our workspace *MORTGAGE*. The menu items with their pointers are as follows:

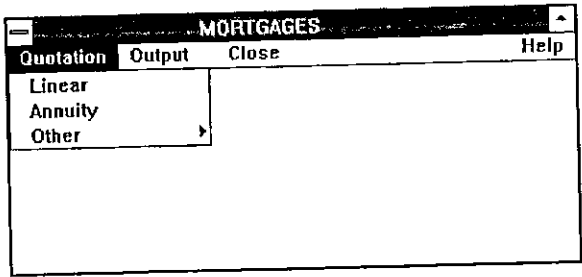
Row	Menu(item)	Pointer
	<i>MENU</i>	<i>IMENU</i> (variable name)
01	MORTGAGES	0
02	Quotation	1
03	Output	1
04	Linear	2
05	Annuity	2
06	Others	2
07	Print	3
08	Screen	3
09	File	3
10	Spreadsheet	3
11	Life Insurance	6
12	Variable rate	6
13	Low rate incremental	6

For instance *Linear*, *Annuity* and *Others* point to *Quotation*, etc. In this example *Quotation* is a *MENU* and *Linear* and *Annuity* are *MENUITEMs*. *Others* is a *MENU* itself, because it is pointed to by *Life Insurance*, *Variable rate* and *Low rate incremental*. The latter are *MENUITEMs* in their turn. Now, we want to convert the list of items in a convenient Windows menu. *MENUDRIVER* is doing this, regardless of depth of pointer levels, using Dyalog APL/W.

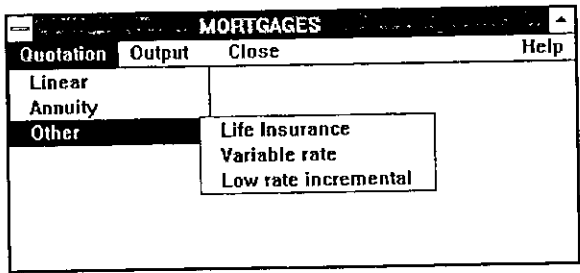
Our first menu display looks like this:



If we click on *Quotation* our display becomes



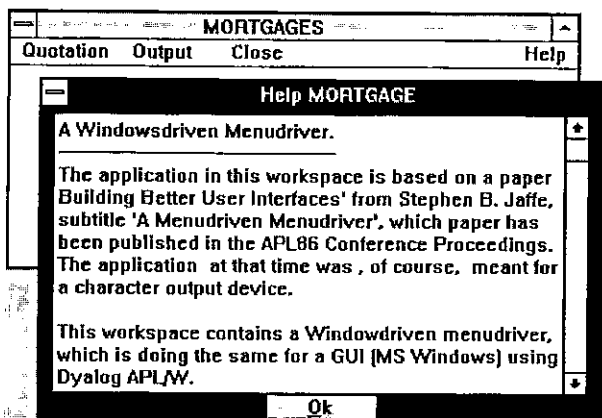
and we may click further on *Other*:



and also the *Output* MENU, etc.

Close gives control to the workspace in a development environment and to Windows Program Manager in a Runtime application.

Help is generated by the *MENUDRIVER* function. The *Help* Window looks like this, and is editable by the user. It uses an external file for saving, so it can be used in Runtime applications.



This Window-driven menudriver is generated by

- a function *MENUDRIVER* (see Appendix)
- a character array with the names of the *MENU*s and *MENUITEM*s
- a pointer controlling the relation of *MENU*s and *MENUITEM*s

The *MENUDRIVER* function takes any character array and pointers with the appropriate names. So, the only thing you have to do is edit these variables. This is done by a function *EDITMENU*, which takes the result of *MENULIST* as a character array as shown above. The last item in any relation (which is a *MENUITEM* in the terminology of Dyalog) should carry the name of a cover (callback-) function and clicking should take care of execution.

MENULIST and *EDITMENU* are simple functions which need no further explanation. *BLANKOUT* is a function published and explained in the quoted reference. *SORT* is an ambivalent function which sorts a list of variables in the right argument, according to the key formed by the variables in that argument or an optional list of variables in the left argument.

Reference

- [1]Jaffe, S.B. Building better user interfaces
APL86 Conference Proceedings (ACM 1986)

Appendix.

```

v MENUDRIVER:[TRAP;PT;SW;A;FX0;FXF;OBJ;PRO;PRI;HELP
[1]  a ---- Jan Karman, fax 0003135775186
[2]  a tel 2191
[3]  a ---- Menudriver for Windows environment -----
[4]  [TRAP=0 'E' 'ERR'
[5]  a ---- Make form -----
[6]  PT=MENU:0
[7]  ('MOO')[WC'FORM'(MENU[PT;])(15 5){40 65}('EVENT' 33 -1){MINBUTTON' 0}
[8]  a ---- Make menubar -----
[9]  ('MOO.MOI')[WC'MENUBAR'
[10] a ---- All MENUS and MENUITEMS -----
[11] SW=0 o FX0+''
[12] PT=(ipIMENU)-IMENU
[13] LOO:M=+'MENU'/(~SW)/'ITEM'','','
[14] OBJ=+'G<.M99' [WC 'M.'>']FMT PT
[15] OBJ=OBJ, 'A+''',MENU[PT;],''
[16] ~SW/LO
[17] A=+'((-1+A+'' ')+A+1A)-'' '','','
[18] OBJ=OBJ, '(',('EVENT' 30 '),A,')'
[19] LO:OBJ=+'G<.M99' [FMT PT-IMENU[PT]],OBJ
[20] ~LO+~O&PT
[21] FX0=FX0,+(PT=0)/[1]OBJ
[22] OBJ=(PT=0)/[1]OBJ o PT-PT-0
[23] ~LO+10*PT
[24] SW~SW
[25] PT=(ipIMENU)nIMENU-1
[26] ~SW/LOO
[27] FX0=+' 'v.>FX0)/FX0+''',0 1+BLANKOUT"FX0
[28] a ---- Function script is editable if wanted (remove 'a' next line)
[29] a [ED'FX0'
[30] SORT'FX0'
[31] a[FX+('FXF'),+FX0
[32] a ---- MENUITEM for closing and help -----
[33] ('MOO.MOI.Close')[WC'MENUITEM' 'Close'('EVENT' 30 1)
[34] ('MOO.MOI.Help')[WC'MENUITEM' 'Help'('ALIGN' 'RIGHT')
[35] ('EVENT' 30 'SHOW_HELP')
[36] a ---- FORM for HELP -----
[37] [WSID [IT'helptxt'
[38] HELP=helptxt
[39] PRO=+'MAXBUTTON' 0){MINBUTTON' 0}{VISIBLE' 0}('EVENT' 33 -1)
[40] PRO=+'BCOL' 160 0 100,PRO
[41] 'Help'[WC'FORM'('Help 'ws){30 44}((60)[0.025*p,+HELP],50),PRO
[42] PRO=+'STYLE' 'MULTI'('VSCROLL' -1){HSCROLL' 0}('EVENT' 22 0)
[43] 'Help.ED'[WC'EDIT'('TEXT'HELP)(2 2){90 95}{FCOL' 40 10 180),PRO
[44] 'Help.Ok'[WC'BUTTON' 'Ok'(94 40){20}{CANCEL' 1}('EVENT' 30 1)
[45] QDQ:choice+=QDQ'MOO'
[46] HELP~Help.ED[WC'TEXT'
[47] L3:~Exit
[48] a ---- FORM for Errors -----
[49] Err:
[50] PRI=+'MAXBUTTON' 0){MINBUTTON' 0}{VISIBLE' 1}('EVENT' 33 -1)
[51] 'Error'[WC'FORM' 'Error message'{50 20}{18 40}{BCOL' 0 140 80),PRI
[52] 'Error.ED'[WC'EDIT'('TEXT'no program)(2 2){70 95}
[53] ('FCOL' 140 0 10){STYLE' 'MULTI'
[54] 'Error.Cancel'[WC'BUTTON' 'Ok'(80 40){20}{CANCEL' 1}('EVENT' 30 1)
[55] a ---- Invoke windows and exit-section -----
[56] [QDQ'Error' o [EX'Error' o ~QDQ
[57] Exit:[EX'
[58] ~DOIF HELP#helptxt o helptxt+HELP
[59] [EX'helptxt' o [OFF a Use in Runtime application
v

```

Index to Advertisers

Adaytum Systems	6
Dyadic Systems Ltd	100
Interprocess Systems	90
Mercia Software (half)	11
MicroAPL	12
Strand Software Systems Inc	64
Soliton	2
APL Booklist	60
Vector Back Numbers	28

Advertisers should contact Gill Smith, on 0439-788385, CompuServe: 100331,644

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue $n+1$ and discuss themes for issues $n+2$ onwards. Please send the text of submitted articles (with diskette as appropriate) to the Editor:

Anthony Camacho,
11 Auburn Road, Redland,
BRISTOL, BS6 6LS
Tel: 0272-730036

Authors wishing to use Windows Write should contact Vector Production for a copy of the Vector APL TrueType font and Vector APL typebox.

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK. Tel: 0439-788385 (any time) CompuServe: 100331,644.

BAA: Membership Application / Renewal

Membership of the British APL Association is open to anyone interested in APL. The membership year runs from 1st May to 30th April. The British APL Association is a special interest group of the British Computer Society, Reg. Charity No. 292,786

Name: _____
 Address: _____

 Post or zip code: _____
 Country: _____
 Telephone Number: _____

Membership category (please tick box):
 UK private membership £12 ☐
 Overseas private membership £14 ☐
 Airmail supplement (not needed for Europe) £4 ☐
 UK Corporate membership £100 ☐
 Corporate membership overseas £135 ☐
 Sustaining membership £430 ☐
 Non-voting UK member (student/OAP/unemployed only) £6 ☐

PAYMENT — in Sterling only

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard or Visa number.

I authorise you to debit my Visa/Mastercard account

Number: _____ Expiry date: ____|____

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

Data Protection Act:
The information supplied may be stored on computer and processed in accordance with the registration of the British Computer Society.

(please tick the required option above)

Signature: _____ Send the completed form to:

British APL Association, c/o Rowena Small, 8 Cardigan Road, LONDON, E3 5HU

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1994/95 Committee

Chairman:	Dr Alan Mayer 0792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Secretary:	Sylvia Camacho 0272-730036	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Treasurer:	Nicholas Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU
Journal Editor:	Anthony Camacho 0272-730036 acamacho@cix.compulink.co.uk	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Activities:	Duncan Pearson 0483-33329 100265.1564@compuserve.com	7 Lynne Court Chesham Road GUILDFORD, Surrey
Education:	Dr Ian Clark 0388-527190 100021.3073@compuserve.com	9 Hill End, Frosterley Bishop Auckland Co. Durham DL13 2SX
Technical:	Jonathan Barman 0488-648575 100116.1030@compuserve.com	Hill Top House, East Garston, NEWBURY, Berks RG16 7HD
Projects:	George MacLeod 0442-878065	Greymantle Associates Ltd., Bartrum House, Ravens Lane, BERKHAMSTED, Herts HP4 2DY
Publicity:	David Eastwood 071-922 8866 microapl@applelink.apple.com	MicroAPL Ltd., South Bank Technopark, 90 London Road, LONDON SE1 6LN
Recruitment:	Mark Harris 0438-310155	Kestrel Consulting, Business & Technology Centre, Bessmer Drive, STEVENAGE, Herts SG1 2DX
Administration:	Rowena Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU

Journal Working Group

Editor:	Anthony Camacho	0272-730036
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (0439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (0439-788385)
Support Team:	Jonathan & Bridget Barman (0488-648575), Ray Cannon (0252-874697), Richard and Adam Weber (0302-539761), Sylvia Camacho, Duncan Pearson, John Searle (081-858 6811), David Ziemann (071-267 8032), Jon Sandles (0904-411635)	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" — an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Compass R&D Ltd
5 & 7 Frederick Sanger Road
Surrey Research Park
GUILDFORD Surrey GU2 5YD
Tel:0483-302249
Fax:0483-302279

HMW Trading Systems Ltd
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel:071-353 4212
Fax:071-353 3325

Soliton Associates Ltd
Groot Blankenberg 53
1082 AC Amsterdam
Netherlands
Tel:+31 20 646 4475
Fax:+31 20 644 1206
Email:ijh@soliton.com

Manugistics
2115 East Jefferson St
Rockville
MARYLAND 20852 USA
Tel: (301) 984-5412
Fax: (301) 984-5094
Email:aplSales@manu.com (US)
Email:ntl@manu.com (International)

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE,
Hants, RG24 0AL
Tel:0256-811125
Fax:0256-811130

MicroAPL Ltd
South Bank Technopark
90 London Road
LONDON SE1 6LN
Tel:071-922 8866
Fax:071-928 1006

Dutch APL Association
Postbus 1341
3430BH Nieuwegein
Netherlands
Tel:03474-2337

Insight Systems Aps
Nordre Strandvej 119A
DK-3150 Hellebæk
Denmark
Tel: +45 42 10 70 22
Fax: +45 42 10 75 74
Email: info@insight.dk